

JEU D'ECHECS QUANTIQUE

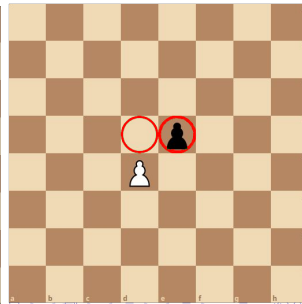
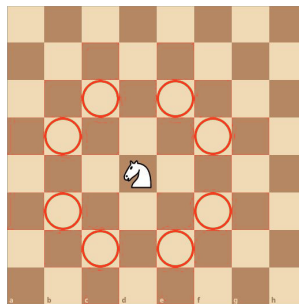
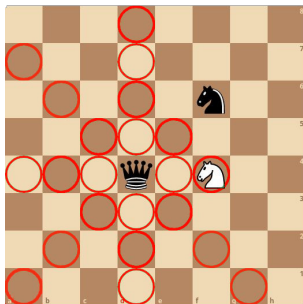
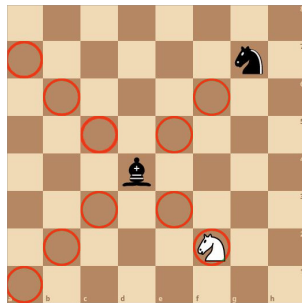
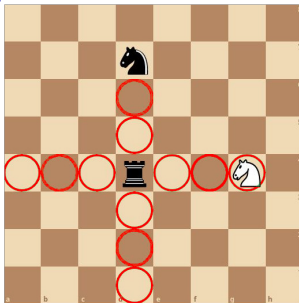
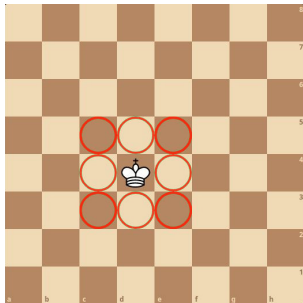
Maël Petit n°32502



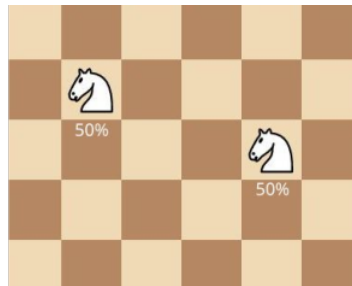
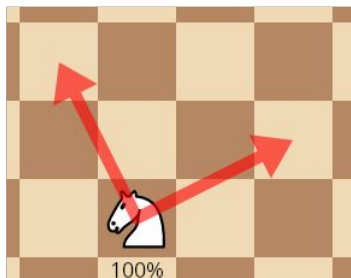
Présentation des règles du jeu

- 2 joueurs qui jouent à tour de rôle
- Sur un carré plateau de 64 cases
- Différents types de pièces
- Un tour = un mouvement
- Objectif : capturer le roi adverse

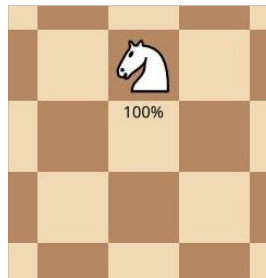
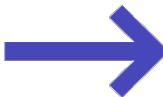
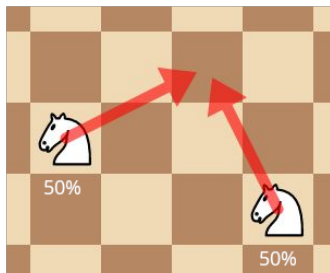
Mouvements classiques



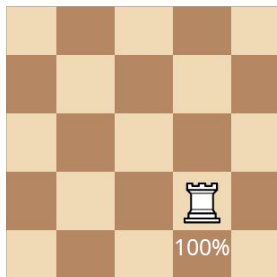
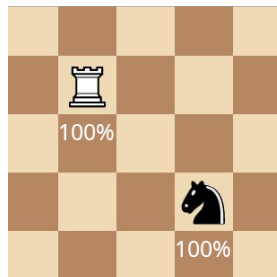
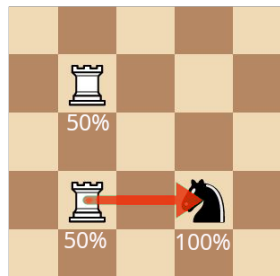
Mouvement split



Mouvement merge

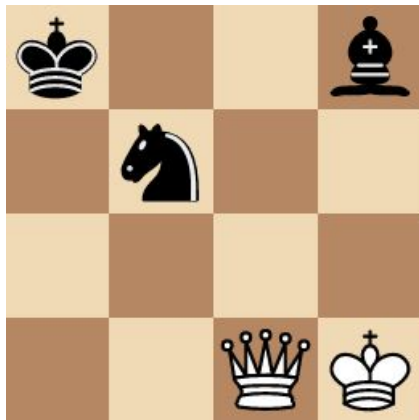


Principe des mesures



Objectif : Etude de la finale fou-cavalier contre dame

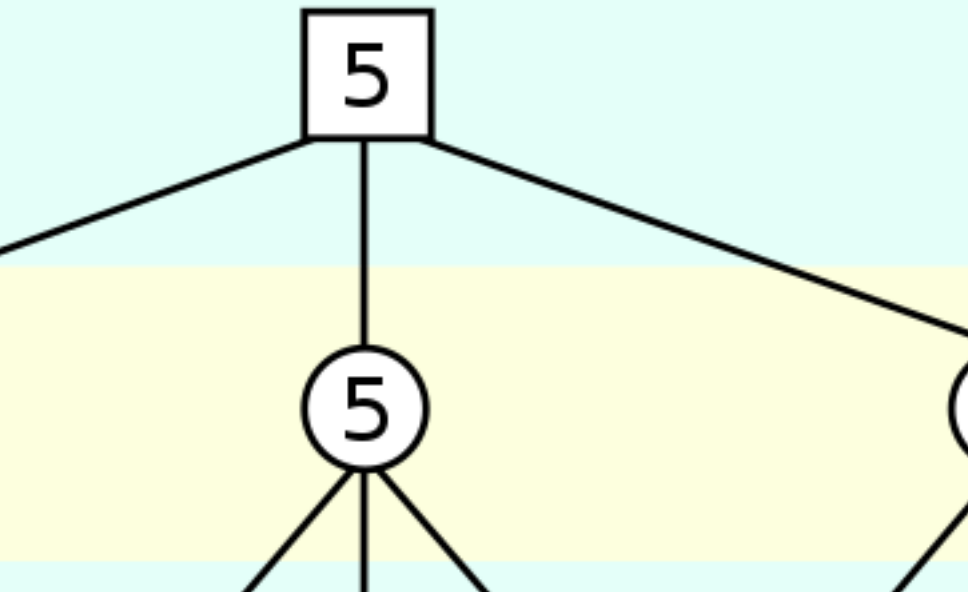
- Diminution de la taille du plateau
- Jeu d'échecs classique : finale gagnante pour les blancs mais difficile à gagner
- Jeu d'échecs quantique : stratégie simple pour avoir une chance de gagner rapidement



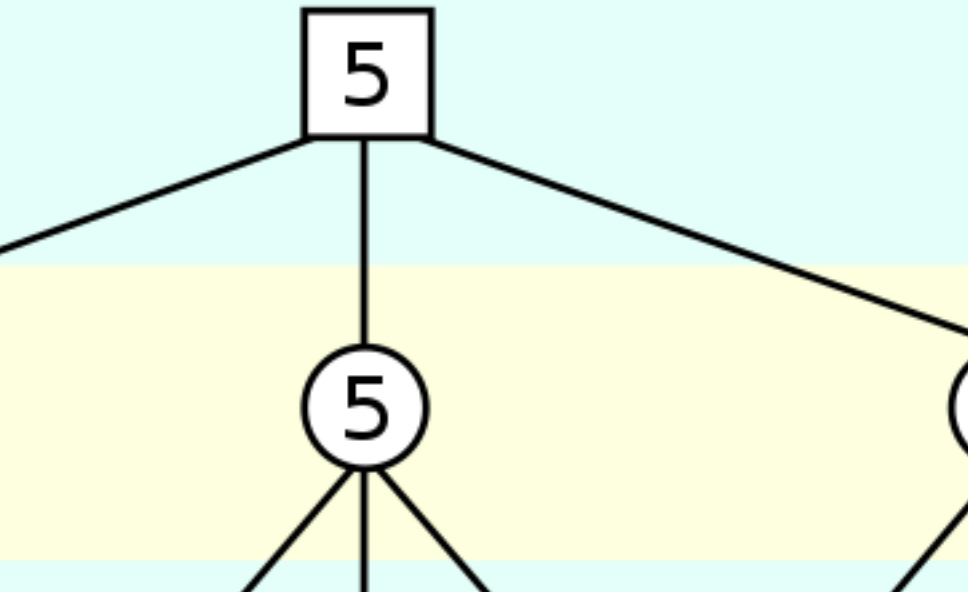
Implémentation

- Représentations des différentes structures (plateau, pièces, matrice, qubit, ...) à l'aide du papier du chercheur Christopher Cantwell
- Récupération des mouvements légaux
- Les mesures
- Réalisation des mouvements

Algorithme min-max : principe



Elagage alpha-beta : principe



Algorithme min-max

- Besoin d'être adapté lors des mesures
- Fonction d'évaluation des feuilles de l'arbre :

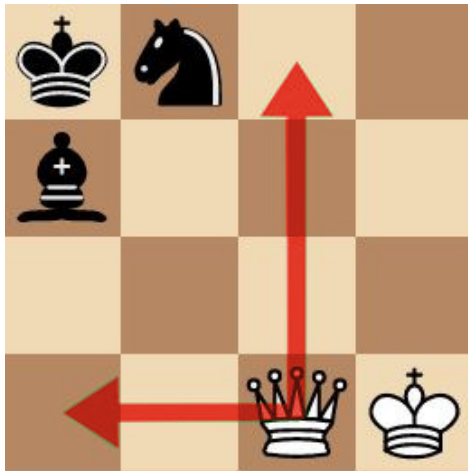
$$eval(\mathcal{B}) = \begin{cases} 1 & \mathcal{B} \text{ est gagnant pour le joueur blanc} \\ -1 & \mathcal{B} \text{ est gagnant pour le joueur noir} \\ 0 & \text{sinon} \end{cases}$$

- Calcul de la valeur d'un mouvement :

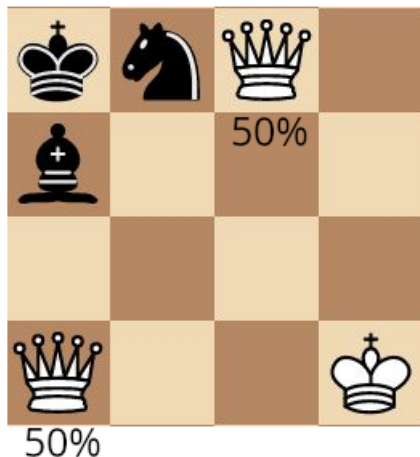
$$proba_1 * board_1 + proba_2 * board_2$$

Résolution sur un exemple

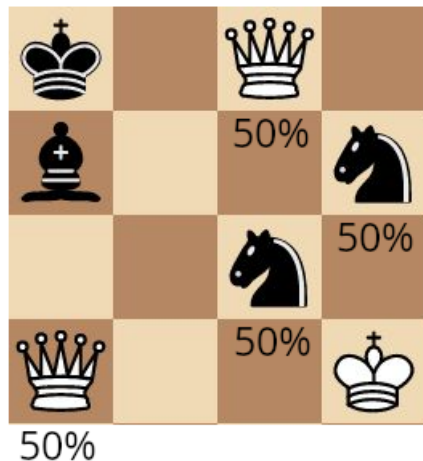
- Profondeur = 4



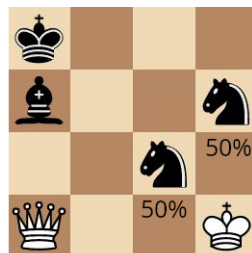
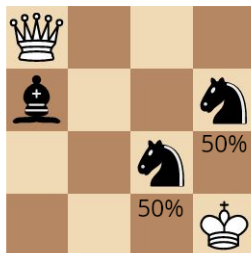
Résolution sur un exemple



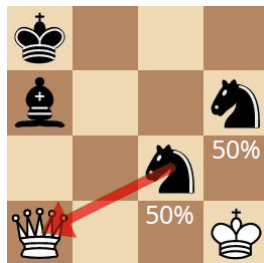
Résolution sur un exemple



Résolution sur un exemple



Effet horizon



- Changement de la fonction d'évaluation :

$$eval(\mathcal{B}) = \begin{cases} 1 & \mathcal{B} \text{ est gagnant pour le joueur blanc} \\ -1 & \mathcal{B} \text{ est gagnant pour le joueur noir} \\ -P & \text{si la dame blanche est capturée et que le joueur noir a} \\ & \text{encore son cavalier et son fou} \\ 0 & \text{sinon} \end{cases}$$

- Cette fonction d'évaluation semble favoriser les noirs

Test sur des exemples générés aléatoirement

- Les pièces sont positionnées aléatoirement
- 1000 plateaux générés
- Le joueur blanc joue en premier
- Profondeur = 4

Nombre de plateaux passés	722
Moyenne	0.08
$-1 \leq val < -0.75$	5
$-0.75 \leq val < -0.5$	0
$-0.5 \leq val < -0.25$	2
$-0.25 \leq val < 0$	0
$val = 0$	239
$0 < val < 0.25$	1
$0.25 \leq val < 0.5$	4
$0.5 \leq val < 0.75$	0
$0.75 \leq val \leq 1$	27

Test sur des exemples générés aléatoirement

- Les pièces sont positionnées aléatoirement, sauf les rois qui sont placés sur des bords opposés
- 1000 plateaux générés
- Le joueur blanc joue en premier
- Profondeur = 4

Nombre de plateaux passés	524
Moyenne	0.01
$-1 \leq val < -0.75$	1
$-0.75 \leq val < -0.5$	0
$-0.5 \leq val < -0.25$	1
$-0.25 \leq val < 0$	0
$val = 0$	458
$0 < val < 0.25$	4
$0.25 \leq val < 0.5$	9
$0.5 \leq val < 0.75$	0
$0.75 \leq val \leq 1$	3

Test sur des exemples générés aléatoirement

- Les pièces sont positionnées aléatoirement, sauf les rois qui sont placés sur des bords opposés
- 1000 plateaux générés
- Le joueur noir joue en premier
- Profondeur = 4

Nombre de plateaux passés	743
Moyenne	0.00
$-1 \leq val < -0.75$	0
$-0.75 \leq val < -0.5$	0
$-0.5 \leq val < -0.25$	0
$-0.25 \leq val < 0$	0
$val = 0$	256
$0 < val < 0.25$	0
$0.25 \leq val < 0.5$	0
$0.5 \leq val < 0.75$	0
$0.75 \leq val \leq 1$	1

Conclusion

- Les mouvements quantiques favorisent l'attaque
- Le joueur qui a le trait est avantageé comme au jeu d'échecs classique
- Profondeur faible
- L'effet horizon empêche d'avoir des résultats précis

Index

- main.cpp
- Board.hpp
- Board.hpp
- Board.hpp
- mesure.hpp
- move.hpp
- get-proba-move.hpp
- Piece.hpp
- Piece.hpp
- get-list-move.hpp
- TypePiece.hpp
- Coord.hpp
- Coord.cpp
- Move.hpp
- Move.cpp

- Qubit.hpp
- Qubit.hpp
- Random.hpp
- Random.cpp
- math-utility.hpp
- math-utility.hpp
- check-path.hpp
- check-path.hpp
- Unitary.hpp
- CMatrix.hpp
- Complex-printer.hpp
- Constexpr.hpp
- ConsoleInterface.hpp
- ConsoleInterface.hpp
- final-solver.hpp

- final-solver.hpp
- Matrix.hpp
- Matrix.hpp
- test-move-promotion.cpp
- test-move-enpassant-with-capture.cpp
- test-move-pawn-two-step.cpp
- test-move-promotion-with-capture.cpp
- test-move-promotion-with-split-before.cpp
- test-move-split-with-mesure.cpp
- test-split-in-non-empty-case.cpp
- test-slide-merge-move-through-a-split-piece.cpp
- test-get-proba-move-slide-capture.cpp
- test-get-proba-move-slide-on-same-color.cpp
- test-get-proba-move-slide-without-target.cpp
- test-get-proba-move-jump-same-color.cpp

- test-get-proba-move-jump-capture.cpp
- test-capture-slide-move-true.cpp
- test-capture-slide-move-false.cpp
- test-move-merge-after-split.cpp
- test-split-after-split.cpp
- test-split-after-split2.cpp
- test-multiple-split.cpp
- test-split-takes.cpp

main.cpp

Retour au début

```
1  #include <iostream>
2  #include <complex>
3  #include <Complex_printer.hpp>
4  #include <CMatrix.hpp>
5  #include <Unitary.hpp>
6  #include <Qubit.hpp>
7  #include <Board.hpp>
8  #include <Piece.hpp>
9  #include <Move.hpp>
10 #include <observer_ptr.hpp>
11 #include <check_path.hpp>
12 #include <Constexpr.hpp>
13 #include <TypePiece.hpp>
14 #include <ConsoleInterface.hpp>
15 #include <final_solver.hpp>
16 #include <functional>
17
18
19 int main()
20 {
21     Board<4> B{
22         {{B_KING, B_KNIGHT, Piece(), Piece()},
23          {B_BISHOP, Piece(), Piece(), Piece()},
24          {Piece(), Piece(), Piece(), Piece()},
25          {Piece(), Piece(), W_QUEEN, W_KING}}};
26     Board<4> B1{
```

```

27      {{B_KING, Piece(), Piece(), B_BISHOP},
28       {Piece(), B_KNIGHT, Piece(), Piece()}},
29       {Piece(), Piece(), Piece(), Piece()}},
30       {Piece(), Piece(), W_QUEEN, W_KING}}};
31
32      Board<4> B3{
33      {{B_KING, Piece(), B_KNIGHT, B_BISHOP},
34       {Piece(), B_KING, Piece(), Piece()}},
35       {Piece(), Piece(), Piece(), Piece()}},
36       {Piece(), Piece(), W_QUEEN, W_KING}}};
37
38
39      Board<4> Bt{
40      {{B_KING, B_BISHOP, Piece(), Piece()}},
41       {Piece(), Piece(), Piece(), Piece()}},
42       {Piece(), Piece(), Piece(), W_KNIGHT}},
43       {Piece(), Piece(), W_KNIGHT, W_KING}}};
44
45      Board<3> Bt1{
46      {{B_KING, Piece(), Piece()}},
47       {Piece(), W_ROOK, Piece()}},
48       {Piece(), Piece(), W_KING}}
49      };
50      Board<3> Bt2{
51      {{B_KING, Piece(), Piece()}},
52       {Piece(), W_KING, Piece()}},
53       {Piece(), Piece(), Piece()}}
54      };
55
56

```

Retour au début

```
57     Board<1,6> B2{
58         {
59             {W_KING, W_ROOK, Piece(), Piece(), B_KING, Piece()}}
60         }
61     };
62     /*bool b = Final::brut_force_quantum_chess(B2, 1, Color::WHITE);
63     std::cout<<std::boolalpha<<b<<std::en
64     dl;*/
65     auto[res, res2] {Final::res_pos(B,4)};
66     std::cout<<res<<std::endl;
67     Final::print_stack<4>(res2);
68     //bool b {Final::brut_force_classic_chess (B1, 10, Color::WHITE)};
69     //std::cout<< std::boolalpha<< b<< std::endl;
70
71
```

Board.hpp

Retour au début

```
1  #ifndef GAME_BOARD_HPP
2  #define GAME_BOARD_HPP
3
4  #include <vector>
5  #include <array>
6  #include <utility>
7  #include <optional>
8  #include <complex>
9  #include <cstdint>
10 #include <initializer_list>
11 #include <functional>
12 #include <CMatrix.hpp>
13 #include <Piece.hpp>
14 #include <TypePiece.hpp>
15 #include <Color.hpp>
16 #include <Coord.hpp>
17 #include <forward_list>
18 #include <observer_ptr.hpp>
19 #include <Move.hpp>
20 #include <Constexpr.hpp>
21 #include <check_path.hpp>
22
23 class Piece;
24
25 /**
26  * @brief La classe représentant le plateau de jeu
```

```

27  *
28  * @tparam N Le nombre de ligne du plateau
29  * @tparam M Le nombre de colonne du plateau
30  */
31  template <std::size_t N = 8, std::size_t M = N>
32  class Board final
33  {
34  public:
35      // Constructeur
36      CONSTEXPR Board();
37      /**
38       * @brief Initialise le plateau à partir d'une liste de pièce,
39       * les pièces ne sont pas divisées initialement.
40       *
41       * @param[in] board Double initializer_list sur des pointeurs sur Piece
42       */
43      CONSTEXPR Board(std::initializer_list<
44                      std::initializer_list<
45                        Piece>> const &
46                      board);
47
48      // Copie
49      CONSTEXPR Board(Board const &) = default;
50      CONSTEXPR Board &operator=(Board const &) = default;
51
52      // Mouvement
53      CONSTEXPR Board(Board &&) = default;
54      CONSTEXPR Board &operator=(Board &&) = default;
55
56      // Destructeur

```

```

57     CONSTEXPR ~Board() = default;
58
59     /**
60      * @brief Retourne le nombre de ligne du plateau
61      *
62      * @return std::size_t Le nombre de ligne
63      */
64     CONSTEXPR static std::size_t numberLines() noexcept;
65
66     /**
67      * @brief Retourne le nombre de colonne
68      *
69      * @return std::size_t Le nombre de colonne
70      */
71     CONSTEXPR static std::size_t numberColumns() noexcept;
72
73     /**
74      * @brief Retourne un pointeur sur la piece à l'emplacement cible
75      *
76      * @param[in] n L'indice de la ligne
77      * @param[in] m L'indice de la colonne
78      * @return Un pointeur observateur sur une piece
79      * ou nullptr si la case est vide
80      */
81     CONSTEXPR Piece const &
82     operator()(std::size_t n,
83               std::size_t m) const noexcept;
84
85     /**
86      * @brief Renvoie la liste dans tous les mouvements

```

```

87      * classiques légaux d'une pièce
88      *
89      * @param pos Position de la pièce
90      *
91      * @return std::forward_list<Coord> La liste de coordonnées des positions
92      * d'arrivées de chaque mouvement possible
93      */
94  CONSTEXPR std::forward_list<Coord>
95  get_list_normal_move(Coord const &pos) const;
96
97  /**
98   * @brief Renvoie la liste de toutes les cases qu'une pièce
99   * peut atteindre lors d'un mouvement split, il faut choisir
100  * deux éléments de la liste pour créer un mouvement split.
101  *
102  * @param[in] pos Position de la pièce.
103  *
104  * @return std::forward_list<Coord> La liste de coordonnées
105  * des positions d'arrivées possible sachant que chaque élément
106  * représente une seule des deux cases d'arrivées d'un split move
107  */
108  CONSTEXPR std::forward_list<Coord>
109  get_list_split_move(Coord const &pos) const;
110
111  /**
112   * @brief Teste si les mouvement de la piece sont si la
113   * pièce est un pion un mouvement de promotion
114   *
115   * @note peut etre utiliser sans verifier si la pièce est un pion
116   *

```

```

117     * @param[in] pos La position de la pièce
118     * @return true si le mouvement possible est un mouvement
119     * de promotion
120     * @return false sinon
121     */
122     CONSTEXPR bool
123     check_if_use_move_promote(Coord const &pos) const noexcept;
124
125     /**
126     * @brief Renvoie la liste de toutes les promotions
127     *
128     * @warning Ne verifie pas la validité du mouvement,
129     * de la position ou du type de la pièce
130     *
131     * @param[in] pos La position du pion
132     * @return La liste de tout les mouvements de promotion
133     */
134     CONSTEXPR std::forward_list<Move>
135     get_list_promote(Coord const &pos) const noexcept;
136
137     /**
138     * @brief Applique une fonction à l'entièreté des mouvements possible
139     *
140     * @param[in, out] func La fonction à appliquer sur tout les mouvements
141     * Prototype : bool f(Move const&)
142     * Si renvoie true alors le parcourt est interrompue
143     * @param[in] color La couleur du joueur auquel on
144     * récupère les mouvements
145     */
146     template <class UnitaryFunction>

```



```

147  CONSTEXPR void
148  all_move(
149      UnitaryFunction func,
150      std::optional<Color> color_player = std::nullopt) const noexcept;
151
152  /**
153   * @brief Test si un mouvement est réalisable
154   *
155   * @param[in] move Le mouvement à tester
156   * @return true si le mouvement est légal
157   */
158  CONSTEXPR bool move_is_legal(Move const &move) const;
159
160  /**
161   * @brief Réalise un mouvement d'une pièce quelque soit le type
162   * du mouvement (classic, split, merge)
163   *
164   * @warning Ne procède aucune vérification sur la validité du mouvement
165   *
166   * @param[in] mouvement Le mouvement à réaliser
167   * @param[in] val_mes Optionel peut déterminer la mesure de
168   * la présence d'une pièce ou non
169   */
170  CONSTEXPR void
171  move(Move const &movement, std::optional<bool> val_mes = std::nullopt);
172
173  /**
174   * @brief Test si un plateau est gagnant pour une couleur
175   *
176   * @param[in] c Couleur

```

```

177     * @return true si la position est gagnante pour la couleur c
178     * @return false sinon
179     */
180     CONSTEXPR bool winning_position(Color c) const noexcept;
181
182     /**
183     * @brief Recupère la couleur du joueur au tour de jouer
184     *
185     * @return Color la couleur du joueur
186     */
187     CONSTEXPR Color get_current_player() const noexcept;
188
189     /**
190     * @brief Change le joueur actuel et passe la main à l'autre joueur
191     */
192     CONSTEXPR void change_player() noexcept;
193
194     /**
195     * @brief Renvoie la probabilité qu'il y ait une pièce à une position.
196     *
197     * @param pos La position
198     */
199     CONSTEXPR double get_proba(Coord const &pos) const noexcept;
200
201     friend class Piece;
202
203     template <std::size_t _N, std::size_t _M>
204     friend CONSTEXPR bool
205     check_path_straight_1_instance(
206         Board<_N, _M> const &board,

```

```

207     Coord const &dpt,
208     Coord const &arv,
209     std::size_t position,
210     std::optional<Coord>);
211
212     template <std::size_t _N, std::size_t _M>
213     friend CONSTEXPR bool
214     check_path_diagonal_1_instance(
215         Board<_N, _M> const &board,
216         Coord const &dpt,
217         Coord const &arv,
218         std::size_t position,
219         std::optional<Coord>);
220
221     template <std::size_t _N, std::size_t _M>
222     friend CONSTEXPR bool
223     check_path_queen_1_instance(
224         Board<_N, _M> const &board,
225         Coord const &dpt,
226         Coord const &arv,
227         std::size_t position,
228         std::optional<Coord>);
229
230     /**
231     * @brief Fonction qui permet de faire un mouvement de pion quelconque avec une promotion
232     *
233     * @param s Coordonnées de la source
234     * @param t Coordonnées de la cible
235     * @param p Le type de la pièce de la promotion
236     * @param[in] val_mes Optionel peut determiner la mesure de

```

```

237     * la présence d'une pièce ou non
238     */
239     CONSTEXPR void move_promotion(
240         Move const &move,
241         std::optional<bool> val_mes = std::nullopt);
242
243     /**
244     * @brief Renvoie la probabilité que le mouvement soit réalisé
245     *
246     * @param move Le mouvement à réaliser
247     * @return Une probabilité comprise entre 0 et 1
248     */
249     CONSTEXPR double get_proba_move(Move const &move) const noexcept;
250
251 private:
252
253
254     /**
255     * @brief Renvoie la probabilité que la fonction mesure renvoie true,
256     * c'est-à-dire la proba que le mouvement soit "réussi"
257     *
258     * @param p Coordonnées de la case
259     * @return La probabilité
260     */
261     CONSTEXPR double
262     get_proba_mesure(Coord const &p) const noexcept;
263
264     /**
265     * @brief Renvoie la probabilité que la fonction mesure_capture_slide renvoie true,
266     * c'est-à-dire la proba que le mouvement de capture_slide soit "réussi"

```

```

267      *
268      * @param s Coordonnées de la source
269      * @param t Coordonnées de la cible
270      * @param check_path Fonction qui teste la présence d'une pièce
271      * entre une source et une cible
272      * @return La probabilité
273      */
274  CONSTEXPR double get_proba_mesure_capture_slide(
275  Coord const &s,
276  Coord const &t,
277  std::function<bool(Board<N, M> const &,
278                      Coord const &,
279                      Coord const &,
280                      std::size_t,
281                      std::optional<Coord>)>
282      check_path) const noexcept;
283
284  /**
285   * @brief Renvoie la probabilité que la fonction mesure_castle renvoie true,
286   * c'est-à-dire la proba que le mouvement de roque soit "réussi"
287   *
288   * @param king Coordonnées du roi
289   * @param rook Coordonnées de la tour
290   * @return La probabilité
291   */
292  CONSTEXPR double get_proba_mesure_castle(
293      Coord const &king,
294      Coord const &rook) const noexcept;
295
296  /**

```

```

297 * @brief Vérifie si la case à une possibilité de contenir une pièce,
298 * et si elle n'en a pas modifie m_piece_board en nullptr.
299 *
300 * @param pos Les coordonnées de la position de la case a vérifier.
301 */
302 void update_case(std::size_t pos) noexcept;
303 /**
304 * @brief Fonction qui permet de mettre à jour le plateau,
305 * car ce mouvement entraine l'apparition de plusieurs instance de board
306 * identique, il faut donc les concaténer en ajoutant les probas, si la
307 * proba vaut 0, on supprime l'instance
308 *
309 * @warning Complexité en  $k^2 \times N \times M$ , avec  $k$  la taille de  $m\_board$ 
310 * c'est à dire le nombre d'instance du plateau,  $N$  et  $M$  les dimensions
311 * du plateau
312 */
313 void update_board() noexcept;
314 void update_board_classic() noexcept;
315
316 /**
317 * @brief Mouvement classique d'une pièce qui "saute"
318 * (cavalier, roi)
319 *
320 * @warning Ne procède aucune vérification sur la validité du mouvement
321 *
322 * @tparam N Le nombre de ligne du plateau
323 * @tparam M Le nombre de colonnes du plateau
324 * @param[in] s Coordonnées de la source du mouvement
325 * @param[in] t Coordonnées de la cible du mouvement
326 * @param[in] val_mes Optionel peut determiner la mesure de

```

```

327     * la présence d'une pièce ou non
328     */
329     CONSTEXPR void move_classic_jump(
330         Coord const &s,
331         Coord const &t,
332         std::optional<bool> val_mes = std::nullopt);
333
334     /**
335     * @brief Mouvement "split jump"
336     *
337     * @warning Aucun tests sur la validité du mouvement (cible vide, ect)
338     *
339     * @tparam N Le nombre de lignes du plateau
340     * @tparam M Le nombre de colonnes du plateau
341     * @param[in] s Coordonnées de la source du mouvement
342     * @param[in] t1 Coordonnées de la cible 1 (qui doit être vide)
343     * @param[in] t2 Coordonnées de la cible 2 (qui doit être vide)
344     */
345     CONSTEXPR void move_split_jump(
346         Coord const &s,
347         Coord const &t1,
348         Coord const &t2);
349
350     /**
351     * @brief Mouvement de merge
352     *
353     * @warning Aucun test sur la validité du mouvement
354     * (cible vide, pièce identique sur les sources, ect)
355     *
356     * @tparam N Le nombre de lignes du plateau

```

```

357      * @tparam M Le nombre de colonnes du plateau
358      * @param[in] s1 Coordonnées de la source 1
359      * @param[in] s2 Coordonnées de la source 2
360      * @param[in] t Coordonnées de la cible
361      */
362      CONSTEXPR void move_merge_jump(
363          Coord const &s,
364          Coord const &t1,
365          Coord const &t2);
366      /**
367       * @brief Mouvement classique d'une pièce qui "glisse" (fou, dame, tour)
368       *
369       * @warning Ne procède aucune vérification sur la validité du mouvement
370       *
371       * @tparam N Le nombre de lignes
372       * @tparam M Le nombre de colonnes
373       * @param[in] s Coordonnées de la source
374       * @param[in] t Coordonnées de la cible
375       * @param[in] check_path Fonction qui permet de vérifier la présence
376       * d'une pièce entre la source et la cible sur une instance du plateau
377       * @param[in] val_mes Optionel peut déterminer la mesure de
378       * la présence d'une pièce ou non
379       */
380      CONSTEXPR void
381      move_classic_slide(Coord const &s, Coord const &t,
382          std::function<
383              bool(
384                  Board<N, M> const &,
385                  Coord const &,
386                  Coord const &,

```



```

387         std::size_t,
388         std::optional<Coord>>
389         check_path,
390         std::optional<bool> val_mes = std::nullopt);
391 /**
392  * @brief Mouvement "split slide"
393  * @warning Aucun tests sur la validité du mouvement (cible vide, ect)
394  * @tparam N Le nombre de lignes du plateau
395  * @tparam M Le nombre de colonnes du plateau
396  * @param[in] s Coordonnées de la source du mouvement
397  * @param[in] t1 Coordonnées de la cible 1 (qui doit être vide)
398  * @param[in] t2 Coordonnées de la cible 2 (qui doit être vide)
399  * @param[in] check_path Fonction qui permet de vérifier la présence d'une
400  * pièce entre la source et la cible sur une instance du plateau
401  */
402 CONSTEXPR void move_split_slide(
403     Coord const &s,
404     Coord const &t1,
405     Coord const &t2,
406     std::function<
407         bool(
408             Board<N, M> const &,
409             Coord const &,
410             Coord const &,
411             std::size_t,
412             std::optional<Coord>)
413         >
414     check_path);
415
416 /**

```

```

417 * @brief Mouvement de merge
418 * @warning Aucun test sur la validité du mouvement
419 * (cible vide, pièce identique sur les sources, ect)
420 * @tparam N Le nombre de lignes du plateau
421 * @tparam M Le nombre de colonnes du plateau
422 * @param[in] s1 Coordonnées de la source 1
423 * @param[in] s2 Coordonnées de la source 2
424 * @param[in] t Coordonnées de la cible
425 * @param[in] check_path Fonction qui permet de vérifier la présence d'une
426 * pièce entre la source et la cible sur une instance du plateau
427 */
428 CONSTEXPR void move_merge_slide(
429     Coord const &s1,
430     Coord const &s2,
431     Coord const &t,
432     std::function<
433         bool(
434             Board<N, M> const &,
435             Coord const &,
436             Coord const &,
437             std::size_t,
438             std::optional<Coord>)>
439     check_path);
440
441 /**
442  * @brief Le mouvement de pion d'une case fonctionne comme
443  * un mouvement de jump classique, à la différence qu'un
444  * pion ne peut pas capturer une pièce en avançant.
445  * On mesure de la même façon qu'un jump classique en considérant
446  * toutes les pièces comme des pièces alliées

```

```

447      *
448      * @warning Ne procède aucune vérification sur la validité du mouvement
449      *
450      * @tparam N Le nombre de lignes du plateau
451      * @tparam M Le nombre de colonnes du plateau
452      * @param[in] s Coordonnées de la source
453      * @param[in] t Coordonnées de la cible
454      * @param[in] val_mes Optionel peut determiner la mesure de
455      * la présence d'une pièce ou non
456      * @return true si le mouvement à etait effectué
457      * @return false sinon
458      */
459  CONSTEXPR bool
460  move_pawn_one_step(Coord const &s, Coord const &t,
461                    std::optional<bool> val_mes = std::nullopt);
462
463  /**
464   * @brief Le mouvement de pion de deux cases fonctionne
465   * comme un mouvement de slide classique, à la différence
466   * qu'un pion ne peut pas capturer une pièce en avançant,
467   * on mesure de la même façon qu'un slide classique en considérant
468   * toutes les pièces comme des pièces alliées.
469   *
470   * @warning Aucun test sur la possibilité de faire un mouvement de deux
471   * cases du pion, pas de mise a jour du board sur la prise en passant
472   *
473   * @tparam N Le nombre de lignes du plateau
474   * @tparam M Le nombre de colonnes du plateau
475   * @param[in] s Coordonnées de la source
476   * @param[in] t Coordonnées de la cible

```

```

477      * @param[in] val_mes Optionel peut determiner la mesure de
478      * la présence d'une pièce ou non
479      * @return true si le mouvement à etait effectué
480      * @return false sinon
481      */
482  CONSTEXPR bool
483  move_pawn_two_step(Coord const &s, Coord const &t,
484                    std::optional<bool> val_mes = std::nullopt);
485  /**
486   * @brief Mouvement de capture dun pion. A la différence
487   * d'un mouvement de "capture jump", on mesure la présence
488   * de la cible car le pion a besoin de la cible pour se déplacer
489   *
490   * @warning Ne procède aucune vérification sur la validité du mouvement
491   *
492   * @param[in] s Coordonnées de la source
493   * @param[in] t Coordonnées de la cible
494   * @param[in] val_mes Optionel peut determiner la mesure de
495   * la présence d'une pièce ou non
496   * @return true si le mouvement à etait effectué
497   * @return false sinon
498   */
499  CONSTEXPR bool capture_pawn(Coord const &s,
500                             Coord const &t,
501                             std::optional<bool> val_mes = std::nullopt);
502
503  /**
504   * @brief Permet d'effectuer un mouvement de prise en passant
505   *
506   * @warning Aucun test sur la validité de la cible et de la cible

```

```

507      * en passant, ni sur le type de la pièce
508      *
509      * @tparam N Le nombre de lignes du plateau
510      * @tparam M Le nombre de colonnes du plateau
511      * @param[in] s Les coordonnées de la source
512      * @param[in] t Les coordonnées de la cible (l'endroit où arrive le pion)
513      * @param[in] ep Les coordonnées du pion capturer "en passant"
514      * @param[in] val_mes Optionel peut determiner la mesure de
515      * la présence d'une pièce ou non
516      * @return true si le mouvement à etait effectué
517      * @return false sinon
518      */
519  CONSTEXPR bool
520  move_ennemi(Coord const &s, Coord const &t, Coord const &ep,
521              std::optional<bool> val_mes = std::nullopt);
522
523  /**
524   * @brief Mesure la présence d'une pièce
525   *
526   * @tparam N Le nombre de lignes du plateau
527   * @tparam M Le nombre de colonnes du plateau
528   * @param[in] position La position de la pièce mesurée
529   * @param[in] val_mes Optionel peut determiner la mesure de
530   * la présence d'une pièce ou non
531   * @return true Si la pièce est présente sur la case position
532   * @return false Sinon
533   */
534  CONSTEXPR bool mesure(Coord const &p,
535                        std::optional<bool> val_mes = std::nullopt);
536

```

```

537  /**
538   * @brief Fonction qui permet de faire une mesure dans le cas
539   * d'un mouvement "capture slide"
540   *
541   * @tparam N Le nombre de lignes du plateau
542   * @tparam M Le nombre de colonnes du plateau
543   * @param[in] s Coordonnées de la source du mouvement
544   * @param[in] t Coordonnées de la cible du mouvement
545   * @param[in] check_path Fonction qui permet de vérifier si il y a une pièce
546   * entre la source et la cible sur une instance du plateau
547   * @param[in] val_mes Optionel peut determiner la mesure de
548   * la présence d'une pièce ou non
549   * @return true Si la mesure indique de faire le mouvement
550   * @return false Sinon
551   */
552  CONSTEXPR bool
553  mesure_capture_slide(Coord const &s, Coord const &t,
554                      std::function<bool(Board<N, M> const &,
555                                         Coord const &, Coord const &,
556                                         std::size_t,
557                                         std::optional<Coord>)>
558                      check_path,
559                      std::optional<bool> val_mes = std::nullopt);
560
561  /**
562   * @brief Mouvement classique d'une pièce (pion exclu)
563   * @warning Ne procède aucune vérification sur la validité du mouvement
564   * @tparam N Le nombre de ligne du plateau
565   * @tparam M Le nombre de colonnes du plateau
566   * @param[in] s Coordonnées de la source du mouvement

```

```

567      * @param[in] t Coordonnées de la cible du mouvement
568      */
569  CONSTEXPR void move_classic(Coord const &s, Coord const &t,
570                             std::optional<bool> val_mes);
571
572  /**
573   * @brief Mouvement split d'une pièce (pion exclu)
574   * @warning Ne procède aucune vérification sur la validité du mouvement
575   * @tparam N Le nombre de ligne du plateau
576   * @tparam M Le nombre de colonnes du plateau
577   * @param[in] s Coordonnées de la source du mouvement
578   * @param[in] t1 Coordonnées de la cible 1 (qui doit être vide)
579   * @param[in] t2 Coordonnées de la cible 2 (qui doit être vide)
580   */
581  CONSTEXPR void move_split(Coord const &s,
582                           Coord const &t1,
583                           Coord const &t2);
584
585  /**
586   * @brief Mouvement de merge de pièce (pion exclu)
587   *
588   * @warning Aucun test sur la validité du mouvement
589   * (cible vide, pièce identique sur les sources, ect)
590   *
591   * @tparam N Le nombre de lignes du plateau
592   * @tparam M Le nombre de colonnes du plateau
593   * @param[in] s1 Coordonnées de la source 1
594   * @param[in] s2 Coordonnées de la source 2
595   * @param[in] t Coordonnées de la cible
596   */

```

```

597     CONSTEXPR void move_merge(Coord const &s1,
598                               Coord const &s2,
599                               Coord const &t);
600
601     /**
602     * @brief Gère tout les mouvements d'un pion
603     * @warning Ne procède aucune vérification sur la validité du mouvement
604     * @tparam N Le nombre de lignes du plateau
605     * @tparam M Le nombre de colonnes du plateau
606     * @param[in] s Coordonnées de la source
607     * @param[in] t Coordonnées de la cible
608     * @param[in] val_mes Optionel peut determiner la mesure de
609     * la présence d'une pièce ou non
610     * @return true si le mouvement à etait effectué
611     * @return false sinon
612     */
613     CONSTEXPR bool move_pawn(
614         Coord const &s,
615         Coord const &t,
616         std::optional<bool> val_mes = std::nullopt) noexcept;
617
618     /**
619     * @brief Renvoie une position 1D d'une coordonnée 2D
620     *
621     * @param[in] ligne L'indice de ligne
622     * @param[in] colonne L'indice de colonne
623     * @return std::size_t La position en 1D
624     */
625     CONSTEXPR static std::size_t
626     offset(std::size_t ligne,

```



```

627         std::size_t colonne) noexcept;
628
629     /**
630     * @brief Initialise un plateau à l'aide des listes d'initialisations
631     *
632     * @param[in] board La liste d'initialisation en 2D
633     * @param[out] first_instance Le tableau de la première
634     * instance du plateau
635     * @param[out] piece_board Le plateau contenant les pièces
636     */
637     CONSTEXPR static void
638     initializer_list_to_2_array(
639         std::initializer_list<
640             std::initializer_list<
641                 Piece>> const &board,
642         std::array<bool, N * M> &first_instance,
643         std::array<Piece, N * M> &piece_board) noexcept;
644
645     /**
646     * @brief Construit les mailbox en fonction
647     * des dimensions du plateau
648     *
649     * @param[out] S_mailbox La petite mailbox de la taille du plateau
650     * @param[out] L_mailbox La grande mailbox comportant 4 ligne de plus
651     * et 2 colonne de plus
652     * @return CONSTEXPR
653     */
654     CONSTEXPR static void init_mailbox(std::array<int, N * M>
655                                         &S_mailbox,
656                                         std::array<int, (N + 4) * (M + 2)>

```

```

657                                     &L_mailbox) noexcept;
658
659  /**
660   * @brief fonction auxiliaire qui permet de modifier un plateau
661   * à l'aide d'un array de la forme du type de retour de la fonction
662   * qubitToArray, le deuxième éléments du tableau à une probabilité non nulle
663   * lors d'un mouvement split
664   *
665   * @tparam Q La taille du qubit
666   * @tparam N Le nombre de ligne du plateau
667   * @tparam M Le nombre de colonne du plateau
668   * @param[in] arrayQubit Type de retour de la fonction quibitToArray,
669   * représente la façon dont va être modifier m_board
670   * @param[in] position_board L'indice du plateau dans le tableau de
671   * toutes les instances du plateau
672   * @param[in] tab_positions Tableau des indices des cases modifiées,
673   * on utilise N*M+1 pour signifier que le qubit en question
674   * est un qubit auxiliaire qui ne modifie pas le board
675   */
676  template <std::size_t Q>
677  CONSTEXPR void modify(std::array<std::pair<std::array<bool, Q>,
678                                     std::complex<double>>,
679                       2> const &arrayQubit,
680                       std::size_t position_board,
681                       std::array<std::size_t, Q> const &tab_positions);
682
683  /**
684   * @brief Fonction qui permet d'effectuer un mouvement
685   * sur une instance du plateau
686   *

```

```

687      * @tparam N Le nombre de ligne du plateau
688      * @tparam M Le nombre de colonnes du plateau
689      * @tparam Q La taille du qubit
690      * @param[in] case_modif Permet d'initialiser le qubit
691      * @param[in] position L'instance du plateau modifiée
692      * @param[in] matrix La matrice du mouvement que l'on veut effectuer
693      * @param[in] tab_positions La position sur le plateau des variables
694      * utilisées dans le qubit, que l'on met à N*M+1 si les variables
695      * ne représentent pas des pièces
696      */
697      template <std::size_t Q>
698      CONSTEXPR void move_1_instance(std::array<bool, Q> const &case_modif,
699                                   std::size_t position,
700                                   CMatrix<_2POW(Q)> const &matrix,
701                                   std::array<std::size_t, Q> const
702                                   &tab_positions) noexcept;
703
704      /**
705       * @brief Mouvement du petit roque.
706       * @warning Aucun test sur la validité du mouvement
707       * @param[in] king Coordonnées du roi
708       * @param[in] rook Coordonnées de la tour
709       * @param[in] val_mes Optionel peut déterminer la mesure de
710       * la présence d'une pièce ou non
711       */
712      CONSTEXPR void king_side_castle(
713          Coord const &king,
714          Coord const &rook,
715          std::optional<bool> val_mes = std::nullopt);
716

```

```

717  /**
718   * @brief Fonction qui permet de mesurer la possibilité de faire le roque
719   *
720   * @param[in] king Coordonnées du roi
721   * @param[in] rook Coordonnées de la tour
722   * @param[in] val_mes Optionel peut déterminer la mesure de
723   * la présence d'une pièce ou non
724   * @return true si le roque est possible
725   * @return false sinon
726   */
727  CONSTEXPR bool mesure_castle(
728      Coord const &king,
729      Coord const &rook,
730      std::optional<bool> val_mes = std::nullopt);
731
732  /**
733   * @brief Mouvement du grand roque.
734   *
735   * @warning Ne procède aucune vérification sur la validité du mouvement
736   *
737   * @param[in] king Coordonnées du roi
738   * @param[in] rook Coordonnées de la tour
739   * @param[in] val_mes Optionel peut déterminer la mesure de
740   * la présence d'une pièce ou non
741   */
742  CONSTEXPR void queen_side_castle(
743      Coord const &king,
744      Coord const &rook,
745      std::optional<bool> val_mes = std::nullopt);
746

```

```

747  /**
748   * @brief Le tableau de toutes les instances possibles
749   * du plateau
750   */
751  std::vector<std::pair<std::array<bool, N * M>,
752                    std::complex<double>>>
753      m_board;
754
755  /**
756   * @brief Le plateau indiquant le type des pièces
757   */
758  std::array<Piece, N * M> m_piece_board;
759
760  /**
761   * @brief La petite mailbox
762   */
763  std::array<int, N * M> m_S_mailbox;
764
765  /**
766   * @brief La grande mailbox
767   */
768  std::array<int, (N + 4) * (M + 2)> m_L_mailbox;
769
770  /**
771   * @brief Color::WHITE si c'est aux blanc de jouer aux noirs sinon
772   */
773  Color m_color_current_player;
774
775  /**
776   * @brief Vrai si les noirs[0] / blancs[1] peuvent faire le petit roque

```

Retour au début

```
777     */
778     std::array<bool, 2> m_k_castle;
779
780     /**
781      * @brief Vrai si les noirs[0] / blancs[1] peuvent faire le grand roque
782      */
783     std::array<bool, 2> m_q_castle;
784
785     /**
786      * @brief Contient la case sur laquelle il est
787      * possible de faire une prise en passant qui
788      * est vide (la case occupé si le pion avait avancé
789      * d'une seule case)
790      */
791     std::optional<Coord> m_ep;
792 };
793
794 #include "Board.tpp"
795 #include "mesure.tpp"
796 #include "get_proba_move.tpp"
797 #include "move.tpp"
798
799
```

Board.hpp

Retour au début

```
1 // Lib standard
2 #include <array>
3 #include <initializer_list>
4 #include <algorithm>
5 #include <cassert>
6 #include <utility>
7 #include <functional>
8 #include <cmath>
9 #include <optional>
10 #include <stdexcept>
11
12 // Inclusion projet
13 #include <Qubit.hpp>
14 #include <Piece.hpp>
15 #include <CMatrice.hpp>
16 #include <Unitary.hpp>
17 #include <TypePiece.hpp>
18 #include <math_utility.hpp>
19 #include <Constexpr.hpp>
20 #include <Move.hpp>
21 #include <Random.hpp>
22 #include "Board.hpp"
23
24 template <std::size_t N, std::size_t M>
25 constexpr Board<N, M>::Board()
26     : m_board(),
```

```

27     m_piece_board(),
28     m_S_mailbox(),
29     m_L_mailbox(),
30     m_color_current_player(Color::WHITE),
31     m_k_castle({true, true}),
32     m_q_castle({true, true}),
33     m_ep()
34 {
35     m_board.push_back(std::pair<std::array<bool, 64>,
36                               std::complex<double>>{{}, 0});
37     init_mailbox(m_S_mailbox, m_L_mailbox);
38 };
39
40 template <std::size_t N, std::size_t M>
41 constexpr Board<N, M>::Board(std::initializer_list<
42                               std::initializer_list<
43                                   Piece>> const &board)
44 : m_board(),
45   m_piece_board(),
46   m_S_mailbox(),
47   m_L_mailbox(),
48   m_color_current_player(Color::WHITE),
49   m_k_castle({true, true}),
50   m_q_castle({true, true}),
51   m_ep()
52 {
53     init_mailbox(m_S_mailbox, m_L_mailbox);
54     m_board.push_back(std::pair<std::array<bool, N * M>,
55                               std::complex<double>>{{false}, 1.});
56     initializer_list_to_2_array(board, m_board[0].first, m_piece_board);

```



```

57 };
58
59 template <std::size_t N, std::size_t M>
60 CONSTEXPR std::size_t
61 Board<N, M>::offset(std::size_t ligne, std::size_t colonne) noexcept
62 {
63     return ligne * M + colonne;
64 };
65
66 template <std::size_t N, std::size_t M>
67 CONSTEXPR std::size_t Board<N, M>::numberLines() noexcept
68 {
69     return N;
70 };
71
72 template <std::size_t N, std::size_t M>
73 CONSTEXPR std::size_t Board<N, M>::numberColumns() noexcept
74 {
75     return M;
76 };
77
78 template <std::size_t N, std::size_t M>
79 CONSTEXPR void
80 Board<N, M>::initializer_list_to_2_array(
81     std::initializer_list<
82         std::initializer_list<
83             Piece>> const &board,
84     std::array<bool, N * M> &first_instance,
85     std::array<Piece, N * M> &piece_board) noexcept
86 {

```

```

87
88     auto it_tab{std::begin(first_instance)};
89     auto it_piece_dst{std::begin(piece_board)};
90     assert(std::size(board) <= N && "Value entry out of Board");
91     for (std::initializer_list<Piece> const &e : board)
92     {
93         auto const it_tab_begin_line{it_tab};
94         assert(std::size(e) <= M && "Value entry out of Board");
95         std::for_each(std::begin(e),
96                       std::end(e),
97                       [it_tab](Piece piece) mutable
98                       -> void
99                       {
100                             if (piece.get_type() != TypePiece::EMPTY)
101                             {
102                                 *it_tab = true;
103                             }
104                             else
105                             {
106                                 *it_tab = false;
107                             }
108                             it_tab++; });
109         std::move(std::begin(e), std::end(e), it_piece_dst);
110         it_tab = it_tab_begin_line + M;
111         it_piece_dst += M;
112     }
113 }
114
115 template <std::size_t N, std::size_t M>
116 constexpr void

```

```

117 Board<N, M>::init_mailbox(
118     std::array<int, N * M> &S_mailbox,
119     std::array<int, (N + 4) * (M + 2)> &L_mailbox) noexcept
120 {
121     std::fill(std::begin(L_mailbox),
122              std::begin(L_mailbox) + (2 * (M + 2) + 1), -1);
123
124     std::fill(std::end(L_mailbox) - (2 * (M + 2) + 1),
125              std::end(L_mailbox), -1);
126     auto offset_L_mailboard{
127         [](std::size_t n, std::size_t m)
128             -> std::size_t
129         {
130             return n * (M + 2) + m;
131         }
132     };
133     for (std::size_t i{0}; i < N; i++)
134     {
135         L_mailbox[offset_L_mailboard(i + 2, 0)] = -1;
136         L_mailbox[offset_L_mailboard(i + 2, M + 1)] = -1;
137         for (std::size_t j{0}; j < M; j++)
138         {
139             L_mailbox[offset_L_mailboard(i + 2, j + 1)] =
140                 static_cast<int>(Board<N, M>::offset(i, j));
141             S_mailbox[Board<N, M>::offset(i, j)] =
142                 static_cast<int>(offset_L_mailboard(i + 2, j + 1));
143         }
144     }
145
146     template <std::size_t N, std::size_t M>

```

```

147 CONSTEXPR double Board<N, M>::get_proba(Coord const &pos) const noexcept
148 {
149     double acc{0.};
150     for (auto const &e : m_board)
151     {
152         acc += pow(abs(e.second), 2.) * e.first[offset(pos.n, pos.m)];
153     }
154     return acc;
155 }
156
157 template <std::size_t N, std::size_t M>
158 CONSTEXPR std::forward_list<Coord>
159 Board<N, M>::get_list_normal_move(Coord const &pos) const
160 {
161     if ((*this)(pos.n, pos.m).get_type() != TypePiece::EMPTY)
162     {
163         return (*this)(pos.n, pos.m).get_list_normal_move(*this, pos);
164     }
165     return std::forward_list<Coord>{};
166 }
167
168 template <std::size_t N, std::size_t M>
169 CONSTEXPR std::forward_list<Coord>
170 Board<N, M>::get_list_split_move(Coord const &pos) const
171 {
172     if ((*this)(pos.n, pos.m).get_type() != TypePiece::EMPTY)
173     {
174         return (*this)(pos.n, pos.m).get_list_split_move(*this, pos);
175     }
176     return std::forward_list<Coord>{};

```

```

177 }
178
179 template <std::size_t N, std::size_t M>
180 CONSTEXPR Color Board<N, M>::get_current_player() const noexcept
181 {
182     return m_color_current_player;
183 }
184
185 template <std::size_t N, std::size_t M>
186 CONSTEXPR void Board<N, M>::change_player() noexcept
187 {
188     if (get_current_player() == Color::WHITE)
189     {
190         m_color_current_player = Color::BLACK;
191     }
192     else
193     {
194         m_color_current_player = Color::WHITE;
195     }
196 }
197
198 template <std::size_t N, std::size_t M>
199 template <std::size_t Q>
200 CONSTEXPR void Board<N, M>::modify(
201     std::array<
202         std::pair<
203             std::array<bool, Q>,
204             std::complex<double>>,
205             2> const &arrayQubit,
206     std::size_t position_board,

```

```

207     std::array<std::size_t, Q> const &tab_positions)
208
209 {
210     if (!complex_equal(arrayQubit[1].second, 0i))
211     {
212         std::pair<std::array<bool, N * M>, std::complex<double>> new_b{};
213         std::copy(std::begin(m_board[position_board].first),
214                 std::end(m_board[position_board].first),
215                 std::begin(new_b.first));
216         new_b.second = m_board[position_board].second;
217         for (std::size_t i{0}; i < Q; i++)
218         {
219             if (tab_positions[i] < N * M + 1)
220             { /*Ca nous permet de mettre des variables
221               dans les qubits qui ne sont pas prises
222               en compte lors de la modif du plateau */
223                 new_b.first[tab_positions[i]] = arrayQubit[1].first[i];
224             }
225         }
226         new_b.second *= arrayQubit[1].second;
227         m_board.push_back(new_b);
228     }
229     for (std::size_t i{0}; i < Q; i++)
230     {
231         if (tab_positions[i] < N * M + 1)
232         {
233             m_board[position_board].first[tab_positions[i]] =
234                 arrayQubit[0].first[i];
235         }
236     }

```

```

237     m_board[position_board].second *= arrayQubit[0].second;
238 }
239
240 template <std::size_t N, std::size_t M>
241 template <std::size_t Q>
242 constexpr void
243 Board<N, M>::move_1_instance(std::array<bool, Q> const &case_modif,
244                             std::size_t position,
245                             CMatrix<_2POW(Q)> const &matrix,
246                             std::array<std::size_t, Q> const
247                             &tab_positions) noexcept
248 {
249     Qubit<Q> q{case_modif};
250     auto x{qubitToArray(matrix * q)};
251     modify(std::move(x), position, tab_positions);
252 }
253
254 template <std::size_t N, std::size_t M>
255 constexpr Piece const &
256 Board<N, M>::operator()(std::size_t n, std::size_t m) const noexcept
257 {
258     return m_piece_board[offset(n, m)];
259 }
260
261 template <std::size_t N, std::size_t M>
262 void Board<N, M>::update_board() noexcept
263 {
264     std::size_t size_board = std::size(m_board);
265     for (std::size_t i{size_board}; i > 0; i--)
266     {

```

```

267     using namespace std::complex_literals;
268     if (complex_equal(m_board[i - 1].second, 0i))
269     {
270         m_board.erase(std::begin(m_board) + i - 1);
271     }
272     else
273     {
274         for (std::size_t j{i - 1}; j > 0; j--)
275         {
276             if (m_board[i - 1].first == m_board[j - 1].first)
277             {
278                 m_board[j - 1].second += m_board[i - 1].second;
279                 m_board.erase(std::begin(m_board) + i - 1);
280                 break;
281             }
282         }
283     }
284 }
285 }
286 template <std::size_t N, std::size_t M>
287 CONSTEXPR bool Board<N, M>::winning_position(Color c) const noexcept
288 {
289     for (std::size_t i{0}; i < N * M; i++)
290     {
291         if (m_piece_board[i].get_type() == TypePiece::KING &&
292             m_piece_board[i].get_color() != c)
293         {
294             return false;
295         }
296     }

```



```

297     return true;
298 }
299
300 template <std::size_t N, std::size_t M>
301 constexpr bool
302 Board<N, M>::move_is_legal(Move const &move) const
303 {
304     if (move.type == TypeMove::NORMAL)
305     {
306         auto list{get_list_normal_move(move.normal.src)};
307         return std::find(
308             std::begin(list),
309             std::end(list),
310             move.normal.arv) !=
311             std::end(list);
312     }
313     else if (move.type == TypeMove::SPLIT)
314     {
315         std::forward_list<Coord> list{get_list_split_move(move.split.src)};
316         std::array<bool, 2> found{};
317         std::array<Coord, 2> arv{{move.split.arv1, move.split.arv2}};
318         for (Coord const &e : list)
319         {
320             for (std::size_t i{0}; i < std::size(arv); i++)
321             {
322                 if (e == arv[i])
323                 {
324                     found[i] = true;
325                 }
326             }

```

```

327         if (std::all_of(
328             std::begin(found),
329             std::end(found),
330             [](bool v) -> bool
331             { return v; }))
332         {
333             break;
334         }
335     }
336 }
337 else if (move.type == TypeMove::MERGE)
338 {
339     std::forward_list<Coord> list1{
340         get_list_split_move(move.merge.src1)};
341     std::forward_list<Coord> list2{
342         get_list_split_move(move.merge.src2)};
343
344     return std::find(
345         std::begin(list1),
346         std::end(list1),
347         move.merge.arv) !=
348         std::end(list1) &&
349         std::find(
350             std::begin(list2),
351             std::end(list2),
352             move.merge.arv) !=
353             std::end(list2);
354     }
355 }
356

```

```

357 template <std::size_t N, std::size_t M>
358 void Board<N, M>::update_case(std::size_t pos) noexcept
359 {
360     std::size_t size_board{std::size(m_board)};
361     for (std::size_t i{0}; i < size_board; i++)
362     {
363         if (m_board[i].first[pos])
364         {
365             return;
366         }
367     }
368     m_piece_board[pos] = Piece();
369 }
370
371 template <std::size_t N, std::size_t M>
372 constexpr bool
373 Board<N, M>::check_if_use_move_promote(Coord const &pos) const noexcept
374 {
375     return (*this)(pos.n, pos.m).check_if_use_move_promote(*this, pos);
376 }
377
378 template <std::size_t N, std::size_t M>
379 constexpr std::forward_list<Move>
380 Board<N, M>::get_list_promote(Coord const &pos) const noexcept
381 {
382     return (*this)(pos.n, pos.m).get_list_promote(*this, pos);
383 }
384
385 template <std::size_t N, std::size_t M>
386 template <class UnitaryFunction>

```

```

387 CONSTEXPR void
388 Board<N, M>::all_move(
389     UnitaryFunction func,
390     std::optional<Color> color_player) const noexcept
391 {
392     std::unordered_map<TypePiece,
393         std::unordered_map<
394             Coord,
395             std::vector<Coord>,
396             Coord_hash>>
397         move_merge{};
398
399     for (std::size_t i{0}; i < numberLines(); i++)
400     {
401         for (std::size_t j{0}; j < numberColumns(); j++)
402         {
403             Piece const &piece{(*this)(i, j)};
404             double p_proba{get_proba(Coord(i, j))};
405             if (piece.get_type() != TypePiece::EMPTY &&
406                 piece.get_color() == color_player
407                     .value_or(get_current_player()))
408             {
409                 if (!check_if_use_move_promote(Coord(i, j)))
410                 {
411                     std::forward_list<Coord> move_normal{
412                         get_list_normal_move(Coord(i, j))};
413
414                     for (Coord const &c : move_normal)
415                     {
416                         if (func(Move_classic(Coord(i, j), c)))

```

```

417         {
418             return;
419         }
420     }
421
422     std::forward_list<Coord> move_split{
423         get_list_split_move(Coord(i, j))};
424
425     std::forward_list<Coord>::
426         const_iterator it_move{
427             std::cbegin(move_split)};
428     for (; it_move != std::cend(move_split);
429         it_move++)
430     {
431         for (std::forward_list<Coord>::
432             const_iterator it2{
433                 std::cbegin(move_split)};
434             it2 != cend(move_split); it2++)
435         {
436             if (it_move == it2)
437             {
438                 continue;
439             }
440             if (func(Move_split(Coord(i, j), *it_move, *it2)))
441             {
442                 return;
443             }
444         }
445     if (piece.get_type() != TypePiece::PAWN)
446     {

```

```

447     if (move_merge
448         .contains(piece.get_type()))
449     {
450         if (move_merge
451             .at(piece.get_type())
452             .contains(*it_move))
453         {
454             for (Coord const &c :
455                 move_merge
456                     .at(piece.get_type())
457                     .at(*it_move))
458             {
459                 if (p_proba < 1. ||
460                     get_proba(
461                         Coord(
462                             c.n,
463                             c.m)) < 1.)
464                 {
465                     if (func(Move_merge(
466                         Coord(i, j),
467                         c,
468                         *it_move)))
469                     {
470                         return;
471                     }
472                 }
473             }
474         }
475     else
476     {

```

```

477         move_merge
478             .at(piece.get_type())
479             .insert({*it_move,
480                     std::vector{
481                         Coord(i, j)}}});
482     }
483 }
484 else
485 {
486     move_merge
487         .insert(
488             std::make_pair(
489                 piece
490                     .get_type(),
491                 std::unordered_map<
492                     Coord,
493                     std::vector<Coord>,
494                     Coord_hash>{
495                         std::make_pair(
496                             *it_move,
497                             std::vector{
498                                 Coord(i, j)}}))));
499     }
500 }
501 }
502 }
503 else
504 {
505     std::forward_list<Move> list{
506

```

```

507         get_list_promote(Coord(i, j))});
508
509     for (Move const &move : list)
510     {
511         if (func(move))
512         {
513             return;
514         }
515     }
516 }
517 }
518 }
519 }
520 }
521
522 template <std::size_t N, std::size_t M>
523 void Board<N, M>::update_board_classic() noexcept
524 {
525     std::size_t size_board = std::size(m_board);
526     for (std::size_t i{size_board}; i > 0; i--)
527     {
528         using namespace std::complex_literals;
529         if (complex_equal(m_board[i - 1].second, 0i))
530         {
531             m_board.erase(std::begin(m_board) + i - 1);
532         }
533         else
534         {
535             for (std::size_t j{i - 1}; j > 0; j--)
536             {

```


Retour au début

```
537         if (m_board[i - 1].first == m_board[j - 1].first)
538         {
539             double inter = std::pow(std::abs(m_board[j - 1].second), 2)+
                    ↪ std::pow(std::abs(m_board[i - 1].second), 2);
540             m_board[j - 1].second =std::pow(inter, 1./2 );
541             m_board.erase(std::begin(m_board) + i - 1);
542             break;
543         }
544     }
545 }
546
547 /*std::size_t n_size_board = std::size(m_board);
548 double all_proba {0};
549 for (std::size_t i{0}; i<n_size_board; i++)
550 {
551     all_proba += std::pow(std::abs(m_board[i].second), 2);
552 }
553 all_proba = std::pow(all_proba, 1./2);
554 for (std::size_t i{0}; i<n_size_board; i++)
555 {
556     m_board[i].second /= all_proba;
557 }*/
558
```

Retour au début

```
1 // Lib standard
2 #include <cassert>
3 #include <utility>
4 #include <functional>
5 #include <cmath>
6 #include <optional>
7 #include <stdexcept>
8 #include <iostream>
9
10 // Inclusion projet
11 #include <Piece.hpp>
12 #include <TypePiece.hpp>
13 #include <Constexpr.hpp>
14 #include "Board.hpp"
15
16 template <std::size_t N, std::size_t M>
17 CONSTEXPR bool Board<N, M>::mesure(Coord const &p,
18                                     std::optional<bool> val_mes)
19 {
20
21     Piece p_actuelle = (*this)(p.n, p.m);
22     if (p_actuelle.get_type() == TypePiece::EMPTY)
23     {
24         return false;
25     }
26     else
```

```

27     {
28         bool mes;
29         if (val_mes == std::nullopt)
30         {
31             double x = rnd::randreal(0., 1.);
32             std::size_t indice_mes = 0;
33             double pow_coef{
34                 std::pow(
35                     std::abs(m_board[indice_mes].second), 2));
36             std::size_t size_board{std::size(m_board)};
37             while (x - pow_coef > 0 && indice_mes < size_board - 1)
38             {
39                 x -= pow_coef;
40                 indice_mes++;
41                 if (indice_mes >= size_board)
42                 {
43                     throw std::runtime_error("Indice mesuré de mesure trop grand");
44                 }
45                 pow_coef = std::pow(std::abs(m_board[indice_mes].second), 2);
46             }
47             mes = m_board[indice_mes].first[offset(p.n, p.m)];
48         }
49         else
50         {
51             mes = val_mes.value();
52         }
53         double proba_delete = 0;
54         // std::size_t nbr_elt_suppr{0};
55         for (std::size_t i{std::size(m_board)}; i > 0; i--)
56         {

```

```

57         if (m_board[i - 1].first[offset(p.n, p.m)] != mes) //?
58         {
59             proba_delete +=
60                 std::pow(std::abs(m_board[i - 1].second), 2);
61
62             m_board.erase(
63                 std::begin(m_board) + i - 1);
64         }
65     }
66     for (auto &e : m_board)
67     {
68         e.second /= std::sqrt(1. - proba_delete);
69     }
70     for (std::size_t i{0}; i < N * M; i++)
71     {
72         if (p_actuelle.get_type() != TypePiece::EMPTY)
73         {
74             update_case(i);
75         }
76     }
77     if (std::empty(m_board))
78     {
79         bool erreur = p_actuelle.get_type() == TypePiece::ROOK;
80         std::cout<<erreur<<std::endl;
81         std::cout<<mes<<std::endl;
82         throw std::runtime_error("La mesure fait nimp");
83     }
84     return mes;
85 }
86 }

```

```

87
88 template <std::size_t N, std::size_t M>
89 CONSTEXPR bool
90 Board<N, M>::mesure_capture_slide(
91     Coord const &s,
92     Coord const &t,
93     std::function<bool(Board<N, M> const &,
94                         Coord const &,
95                         Coord const &,
96                         std::size_t,
97                         std::optional<Coord>)>
98     check_path,
99     std::optional<bool> val_mes)
100 {
101     std::size_t position = offset(s.n, s.m);
102     Piece p_actuelle = (*this)(s.n, s.m);
103     if (p_actuelle.get_type() == TypePiece::EMPTY)
104     {
105         return false;
106     }
107     else
108     {
109         bool mes;
110         if (val_mes == std::nullopt)
111         {
112             double x = rnd::randreal(0., 1.);
113             std::size_t indice_mes = 0;
114             double pow_coef{
115                 std::pow(std::abs(m_board[0].second), 2)};
116             std::size_t size_board{std::size(m_board)};

```

```

117     while (x - pow_coef > 0 && indice_mes < size_board - 1)
118     {
119         x -= pow_coef;
120         indice_mes++;
121         if (indice_mes >= size_board)
122         {
123             throw std::runtime_error("Indice mesuré de mesure trop grand");
124         }
125         pow_coef = std::pow(std::abs(m_board[indice_mes].second), 2);
126
127         // indice_suppr++,
128     }
129     mes = m_board[indice_mes].first[position] &&
130         check_path(*this, s, t, indice_mes, std::nullopt);
131 }
132 else
133 {
134     mes = val_mes.value();
135 }
136 double proba_delete = 0;
137 // std::size_t nbr_elt_suppr{0};
138 for (std::size_t i{std::size(m_board)}; i > 0; i--)
139 {
140     if ((m_board[i - 1].first[position] &&
141         check_path(*this, s, t, i - 1, std::nullopt)) != mes)
142     {
143         proba_delete +=
144             std::pow(std::abs(m_board[i - 1].second), 2);
145
146         m_board.erase(

```

```

147         std::begin(m_board) + i - 1);
148     }
149 }
150 for (auto &e : m_board)
151 {
152     e.second /= std::sqrt(1. - proba_delete);
153 }
154 for (std::size_t i{0}; i < N * M; i++)
155 {
156     if (p_actuelle.get_type() != TypePiece::EMPTY)
157     {
158         update_case(i);
159     }
160 }
161 if (std::empty(m_board))
162 {
163     throw std::runtime_error("La mesure_capture_slide fait nimp");
164 }
165 return mes;
166 }
167 }
168
169 template <std::size_t N, std::size_t M>
170 constexpr bool
171 Board<N, M>::mesure_castle(
172     Coord const &king,
173     Coord const &rook,
174     std::optional<bool> val_mes)
175 {
176     std::size_t position_king = offset(king.n, king.m);

```

```

177     std::size_t position_rook = offset(rook.n, rook.m);
178
179     bool mes;
180     if (val_mes == std::nullopt)
181     {
182         double x = rnd::randreal(0., 1.);
183         std::size_t indice_mes = 0;
184         std::size_t size_board{std::size(m_board)};
185         double pow_coef{
186             std::pow(std::abs(m_board[0].second), 2)};
187
188         while (x - pow_coef > 0 && indice_mes < size_board - 1)
189         {
190             x -= pow_coef;
191             indice_mes++;
192             if (indice_mes >= size_board)
193             {
194                 throw std::runtime_error("Indice mesuré de mesure trop grand");
195             }
196             pow_coef = std::pow(std::abs(m_board[indice_mes].second), 2);
197         }
198         mes = m_board[indice_mes].first[position_king] &&
199             m_board[indice_mes].first[position_rook] &&
200             check_path_straight_1_instance(*this, king, rook, indice_mes, std::nullopt);
201     }
202     else
203     {
204         mes = val_mes.value();
205     }
206     double proba_delete = 0;

```


Retour au début

```
207 // std::size_t nbr_elt_suppr{0};
208 for (std::size_t i{std::size(m_board)}; i > 0; i--)
209 {
210     if ((m_board[i - 1].first[position_king] &&
211         m_board[i - 1].first[position_rook] &&
212         check_path_straight_1_instance(*this, king, rook, i - 1, std::nullopt)) != mes)
213     {
214         proba_delete +=
215             std::pow(std::abs(m_board[i - 1].second), 2);
216
217         m_board.erase(
218             std::begin(m_board) + i - 1);
219     }
220 }
221 for (auto &e : m_board)
222 {
223     e.second /= std::sqrt(1. - proba_delete);
224 }
225 for (std::size_t i{0}; i < N * M; i++)
226 {
227     if (m_piece_board[i].get_type() != TypePiece::EMPTY)
228     {
229         update_case(i);
230     }
231 }
232 return mes;
233
```

Retour au début

```
1 // Lib standard
2 #include <array>
3 #include <algorithm>
4 #include <cassert>
5 #include <utility>
6 #include <functional>
7 #include <cmath>
8 #include <optional>
9 #include <stdexcept>
10
11 // Inclusion projet
12 #include <Piece.hpp>
13 #include <CMatrix.hpp>
14 #include <Unitary.hpp>
15 #include <TypePiece.hpp>
16 #include <math_utility.hpp>
17 #include <Constexpr.hpp>
18 #include <Move.hpp>
19 #include "Board.hpp"
20
21 template <std::size_t N, std::size_t M>
22 constexpr void Board<N, M>::king_side_castle(Coord const &k,
23                                               Coord const &r,
24                                               std::optional<bool> val_mes)
25 {
26     std::size_t king = offset(k.n, k.m);
```

```

27     std::size_t new_king = offset(k.n, k.m + 2);
28     std::size_t rook = offset(r.n, r.m);
29     std::size_t new_rook = offset(r.n, r.m - 2);
30     if (mesure_castle(k, r, val_mes))
31     {
32         for (std::size_t i{0}; i < std::size(m_board); i++)
33         {
34             move_1_instance(
35                 std::array<bool, 2>{true, false}, i,
36                 MATRIX_ISWAP,
37                 std::array<std::size_t, 2>{king, new_king});
38             move_1_instance(
39                 std::array<bool, 2>{true, false}, i,
40                 MATRIX_ISWAP,
41                 std::array<std::size_t, 2>{rook, new_rook});
42         }
43         m_piece_board[new_king] = std::move(m_piece_board[king]);
44         m_piece_board[new_rook] = std::move(m_piece_board[rook]);
45         m_piece_board[king] = Piece();
46         m_piece_board[rook] = Piece();
47     }
48 }
49 template <std::size_t N, std::size_t M>
50 CONSTEXPR void Board<N, M>::queen_side_castle(Coord const &k,
51                                                  Coord const &r,
52                                                  std::optional<bool> val_mes)
53 {
54     std::size_t king = offset(k.n, k.m);
55     std::size_t new_king = offset(k.n, k.m - 2);
56     std::size_t rook = offset(r.n, r.m);

```

```

57     std::size_t new_rook = offset(r.n, r.m + 3);
58     if (mesure_castle(k, r, val_mes))
59     {
60         for (std::size_t i{0}; i < std::size(m_board); i++)
61         {
62             move_1_instance(
63                 std::array<bool, 2>{true, false}, i,
64                 MATRIX_ISWAP,
65                 std::array<std::size_t, 2>{king, new_king});
66             move_1_instance(
67                 std::array<bool, 2>{true, false}, i,
68                 MATRIX_ISWAP,
69                 std::array<std::size_t, 2>{rook, new_rook});
70         }
71         m_piece_board[new_king] = std::move(m_piece_board[king]);
72         m_piece_board[new_rook] = std::move(m_piece_board[rook]);
73         m_piece_board[king] = Piece();
74         m_piece_board[rook] = Piece();
75     }
76 }
77 template <std::size_t N, std::size_t M>
78 constexpr void
79 Board<N, M>::move_classic_jump(Coord const &s, Coord const &t,
80                                std::optional<bool> val_mes)
81 {
82     std::size_t source = offset(s.n, s.m);
83     std::size_t target = offset(t.n, t.m);
84
85     if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
86         m_piece_board[target].get_type() == m_piece_board[source].get_type() &&

```

```

87     m_piece_board[target].get_color() == m_piece_board[source].get_color()))
88 {
89     std::size_t const size_board{std::size(m_board)};
90     for (std::size_t i{0}; i < size_board; i++)
91     {
92         move_1_instance(
93             std::array<bool, 2>{m_board[i].first[source], false}, i,
94             MATRIX_ISWAP,
95             std::array<std::size_t, 2>{source, target});
96     }
97     m_piece_board[target] = std::move(m_piece_board[source]);
98     m_piece_board[source] = Piece();
99 }
100 else
101 {
102     if (m_piece_board[source].same_color(m_piece_board[target]))
103     {
104         std::optional<bool> negation ;
105         if(val_mes.has_value())
106         {
107             negation = !val_mes.value();
108         }
109         if (!measure(t, negation))
110         {
111             for (std::size_t i{0}; i < std::size(m_board); i++)
112             {
113                 move_1_instance(
114                     std::array<bool, 2>{m_board[i].first[source], false},
115                     i, MATRIX_ISWAP,
116                     std::array<std::size_t, 2>{source, target});

```

```

117         }
118         m_piece_board[target] = std::move(m_piece_board[source]);
119         m_piece_board[source] = Piece();
120     }
121 }
122 else
123 {
124     if (mesure(s, val_mes))
125     {
126         for (std::size_t i{0}; i < std::size(m_board); i++)
127         {
128             CONSTEXPR CMatrix<8> A{
129                 MATRIX_JUMP
130                 .tensoriel_product(
131                     CMatrix<2>::identity()) *
132                     CMatrix<2>::identity()
133                 .tensoriel_product(MATRIX_JUMP)};
134
135             move_1_instance(
136                 std::array<bool, 3>{m_board[i].first[source],
137                                     m_board[i].first[target],
138                                     false},
139                 i,
140                 A,
141                 std::array<std::size_t, 3>{source,
142                                             target,
143                                             N * M + 1});
144         }
145         m_piece_board[target] = std::move(m_piece_board[source]);
146         m_piece_board[source] = Piece();

```

```

147         }
148     }
149 }
150 }
151
152 template <std::size_t N, std::size_t M>
153 CONSTEXPR bool
154 Board<N, M>::move_pawn_one_step(Coord const &s, Coord const &t,
155                                 std::optional<bool> val_mes)
156 {
157     std::size_t source = offset(s.n, s.m);
158     std::size_t target = offset(t.n, t.m);
159
160     if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
161         (m_piece_board[target].get_type() == m_piece_board[source].get_type() &&
162          m_piece_board[target].get_color() == m_piece_board[source].get_color()))
163     {
164         std::size_t const size_board{std::size(m_board)};
165         for (std::size_t i{0}; i < size_board; i++)
166         {
167             move_1_instance(
168                 std::array<bool, 2>{m_board[i].first[source], false},
169                 i, MATRIX_ISWAP,
170                 std::array<std::size_t, 2>{source, target});
171         }
172         m_piece_board[target] = std::move(m_piece_board[source]);
173         m_piece_board[source] = Piece();
174         return true;
175     }
176     else

```

```

177     {
178         std::optional<bool> negation ;
179         if(val_mes.has_value())
180         {
181             negation = !val_mes.value();
182         }
183         if (!mesure(t, negation))
184         {
185             for (std::size_t i{0}; i < std::size(m_board); i++)
186             {
187                 move_1_instance(
188                     std::array<bool, 2>{m_board[i].first[source], false},
189                     i, MATRIX_ISWAP,
190                     std::array<std::size_t, 2>{source, target});
191             }
192             m_piece_board[target] = std::move(m_piece_board[source]);
193             m_piece_board[source] = Piece();
194             return true;
195         }
196     }
197     return false;
198 }
199
200 template <std::size_t N, std::size_t M>
201 constexpr bool
202 Board<N, M>::move_pawn_two_step(Coord const &s, Coord const &t,
203                                 std::optional<bool> val_mes)
204 {
205     {
206         std::size_t source = offset(s.n, s.m);

```



```

207     std::size_t target = offset(t.n, t.m);
208     if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
209         (m_piece_board[target].get_type() == m_piece_board[source].get_type() &&
210          m_piece_board[target].get_color() == m_piece_board[source].get_color()))
211     {
212         std::size_t const size_board{std::size(m_board)};
213         for (std::size_t i{0}; i < size_board; i++)
214         {
215             move_1_instance(
216                 std::array<bool, 3>{!check_path_straight_1_instance(
217                                         *this, s, t, i, std::nullopt),
218                                         false,
219                                         m_board[i].first[source]},
220                 i, MATRIX_SLIDE,
221                 std::array<std::size_t, 3>{N * M + 1, target, source});
222         }
223         m_piece_board[target] = m_piece_board[source];
224         update_case(source);
225         return true;
226     }
227     else
228     {
229         std::optional<bool> negation ;
230         if (val_mes.has_value())
231         {
232             negation = !val_mes.value();
233         }
234         if (!mesure(t, negation))
235         {
236             for (std::size_t i{0}; i < std::size(m_board); i++)

```

```

237     {
238         move_1_instance(
239             std::array<bool, 3>{!check_path_straight_1_instance(
240                                     *this, s, t, i, std::nullopt),
241                                     false,
242                                     m_board[i].first[source]},
243             i, MATRIX_SLIDE,
244             std::array<std::size_t, 3>{N * M + 1, target, source});
245     }
246     m_piece_board[target] = std::move(m_piece_board[source]);
247     update_case(source);
248     return true;
249 }
250 }
251 return false;
252 }
253
254 template <std::size_t N, std::size_t M>
255 constexpr bool Board<N, M>::capture_pawn(
256     Coord const &s,
257     Coord const &t,
258     std::optional<bool> val_mes)
259 {
260     std::size_t source = offset(s.n, s.m);
261     std::size_t target = offset(t.n, t.m);
262     if (measure(s, val_mes))
263     {
264         for (std::size_t i{0}; i < std::size(m_board); i++)
265         {
266             if (m_board[i].first[target])

```

```

267     {
268         CONSTEXPR CMatrix<8> A{
269             MATRIX_JUMP
270             .tensoriel_product(
271                 CMatrix<2>::identity()) *
272                 CMatrix<2>::identity()
273             .tensoriel_product(MATRIX_JUMP)};
274
275         move_1_instance(
276             std::array<bool, 3>{
277                 m_board[i].first[source],
278                 m_board[i].first[target],
279                 false},
280             i, A,
281             std::array<std::size_t, 3>{source, target, N * M + 1});
282     }
283 }
284 m_piece_board[target] = m_piece_board[source];
285 update_case(source);
286 return true;
287 }
288 return false;
289 }
290
291 template <std::size_t N, std::size_t M>
292 CONSTEXPR bool
293 Board<N, M>::move_enpassant(Coord const &s, Coord const &t, Coord const &ep,
294                             std::optional<bool> val_mes)
295 {
296     std::size_t source = offset(s.n, s.m);

```

```

297     std::size_t target = offset(t.n, t.m);
298     std::size_t enpassant = offset(ep.n, ep.m);
299
300     if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
301         (m_piece_board[target].get_type() == m_piece_board[source].get_type() &&
302          m_piece_board[target].get_color() == m_piece_board[source].get_color()))
303     {
304
305         std::size_t const size_board{std::size(m_board)};
306         for (std::size_t i{0}; i < size_board; i++)
307         {
308             if (m_board[i].first[enpassant])
309             {
310                 // permet d'enlever le pion pris en passant
311                 move_1_instance(
312                     std::array<bool, 2>{m_board[i].first[enpassant], false},
313                     i, MATRIX_ISWAP,
314                     std::array<std::size_t, 2>{enpassant, N * M + 1});
315                 move_1_instance(
316                     std::array<bool, 2>{m_board[i].first[source], false},
317                     i, MATRIX_ISWAP,
318                     std::array<std::size_t, 2>{source, target});
319             }
320         }
321         m_piece_board[target] = std::move(m_piece_board[source]);
322         m_piece_board[source] = Piece();
323         m_piece_board[enpassant] = Piece();
324         return true;
325     }
326     else

```

```

327     {
328         if (m_piece_board[source].same_color(m_piece_board[target]))
329         {
330             std::optional<bool> negation ;
331             if(val_mes.has_value())
332             {
333                 negation = !val_mes.value();
334             }
335             if (!mesure(t, negation))
336             {
337
338                 for (std::size_t i{0}; i < std::size(m_board); i++)
339                 {
340                     if (m_board[i].first[enpassant])
341                     {
342                         // permet d'enlever le pion pris en passant
343                         move_1_instance(
344                             std::array<bool, 2>{
345                                 m_board[i].first[enpassant],
346                                 false},
347                             i, MATRIX_ISWAP,
348                             std::array<std::size_t, 2>{enpassant, N * M + 1});
349
350                         move_1_instance(
351                             std::array<bool, 2>{
352                                 m_board[i].first[source],
353                                 false},
354                             i, MATRIX_ISWAP,
355                             std::array<std::size_t, 2>{source, target});
356                     }

```

```

357     }
358     m_piece_board[target] = m_piece_board[source];
359     m_piece_board[source] = Piece();
360     return true;
361 }
362 return false;
363 }
364 else
365 {
366     if (mesure(s, val_mes))
367     {
368         for (std::size_t i{0}; i < std::size(m_board); i++)
369         {
370             if (m_board[i].first[enpassant] ||
371                 m_board[i].first[target])
372             {
373                 // permet d'enlever le pion pris en passant
374                 move_1_instance(
375                     std::array<bool, 2>{
376                         m_board[i].first[enpassant], false},
377                     i, MATRIX_ISWAP,
378                     std::array<std::size_t, 2>{
379                         enpassant, N * M + 1});
380                 // ces deux instructions représentent un mouvement
381                 // de capture jump
382                 move_1_instance(
383                     std::array<bool, 2>{
384                         m_board[i].first[target], false},
385                     i, MATRIX_ISWAP,
386                     std::array<std::size_t, 2>{

```

```

387         target, N * M + 1});
388
389         move_1_instance(
390             std::array<bool, 2>{
391                 m_board[i].first[source], false},
392             i, MATRIX_ISWAP,
393             std::array<std::size_t, 2>{
394                 source, target});
395     }
396 }
397 m_piece_board[target] = std::move(m_piece_board[source]);
398 m_piece_board[source] = Piece();
399 m_piece_board[enpassant] = Piece();
400 return true;
401 }
402 }
403 }
404 return false;
405 }
406
407 template <std::size_t N, std::size_t M>
408 constexpr void
409 Board<N, M>::move_split_jump(Coord const &s, Coord const &t1, Coord const &t2)
410 {
411     std::size_t const source = offset(s.n, s.m);
412     std::size_t const target1 = offset(t1.n, t1.m);
413     std::size_t const target2 = offset(t2.n, t2.m);
414     std::size_t const size_board{std::size(m_board)};
415     for (std::size_t i{0}; i < size_board; i++)
416     {

```

```

417     move_1_instance(
418         std::array<bool, 3>{
419             m_board[i].first[target2],
420             m_board[i].first[target1],
421             m_board[i].first[source]},
422         i, MATRIX_SPLIT,
423         std::array<std::size_t, 3>{target2, target1, source});
424     }
425     m_piece_board[target1] = m_piece_board[source];
426
427     if (m_piece_board[target1].get_type() == TypePiece::EMPTY &&
428         m_piece_board[target2].get_type() == TypePiece::EMPTY)
429     {
430         m_piece_board[target2] = std::move(m_piece_board[source]);
431         m_piece_board[source] = Piece();
432         update_case(target1);
433         update_case(target2);
434     }
435     else
436     {
437         m_piece_board[target2] = m_piece_board[source];
438         update_case(source);
439         update_case(target1);
440         update_case(target2);
441         update_board();
442     }
443 }
444
445 template <std::size_t N, std::size_t M>
446 constexpr void

```



```

447 Board<N, M>::move_merge_jump(Coord const &s1, Coord const &s2, Coord const &t)
448 {
449     std::size_t source1 = offset(s1.n, s1.m);
450     std::size_t source2 = offset(s2.n, s2.m);
451     std::size_t target = offset(t.n, t.m);
452     std::size_t const size_board{std::size(m_board)};
453     for (std::size_t i{0}; i < size_board; i++)
454     {
455         move_1_instance(
456             std::array<bool, 3>{
457                 m_board[i].first[source1],
458                 m_board[i].first[source2],
459                 m_board[i].first[target]},
460             i, MATRIX_MERGE,
461             std::array<std::size_t, 3>{source1, source2, target});
462     }
463     m_piece_board[target] = m_piece_board[source1];
464     update_board();
465     update_case(target);
466     update_case(source1);
467     update_case(source2);
468 }
469
470 template <std::size_t N, std::size_t M>
471 CONSTEXPR void
472 Board<N, M>::move_classic_slide(
473     Coord const &s,
474     Coord const &t,
475     std::function<
476         bool(

```

```

477         Board<N, M> const &,
478         Coord const &,
479         Coord const &,
480         std::size_t,
481         std::optional<Coord>>>
482     check_path,
483     std::optional<bool> val_mes)
484 {
485     std::size_t source = offset(s.n, s.m);
486     std::size_t target = offset(t.n, t.m);
487     if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
488         (m_piece_board[target].get_type() == m_piece_board[source].get_type() &&
489          m_piece_board[target].get_color() == m_piece_board[source].get_color()))
490     {
491         std::size_t const size_board{std::size(m_board)};
492         for (std::size_t i{0}; i < size_board; i++)
493         {
494             move_1_instance(
495                 std::array<bool, 3>{
496                     !check_path(
497                         *this, s, t, i, std::nullopt),
498                     false,
499                     m_board[i].first[source]},
500                 i, MATRIX_SLIDE,
501                 std::array<std::size_t, 3>{N * M + 1, target, source});
502         }
503         m_piece_board[target] = m_piece_board[source];
504         update_case(source);
505     }
506     else

```

```

507     {
508         if (m_piece_board[source].same_color(m_piece_board[target]))
509         {
510             std::optional<bool> negation ;
511             if(val_mes.has_value())
512             {
513                 negation = !val_mes.value();
514             }
515             if (!measure(t, negation))
516             {
517                 for (std::size_t i{0}; i < std::size(m_board); i++)
518                 {
519                     move_1_instance(
520                         std::array<bool, 3>{
521                             !check_path(*this, s, t, i, std::nullopt),
522                             false,
523                             m_board[i].first[source]},
524                         i, MATRIX_SLIDE,
525                         std::array<std::size_t, 3>{N * M + 1, target, source});
526                 }
527                 m_piece_board[target] = std::move(m_piece_board[source]);
528                 m_piece_board[source] = Piece();
529             }
530         }
531     else
532     {
533         if (measure_capture_slide(s, t, check_path, val_mes))
534         {
535             for (std::size_t i{0}; i < std::size(m_board); i++)
536             {

```

```

537         auto const A{
538             MATRIX_ISWAP.tensoriel_product(
539                 CMatrix<2>::identity()) *
540                 CMatrix<2>::identity()
541                 .tensoriel_product(MATRIX_ISWAP)};
542
543         move_1_instance(
544             std::array<bool, 3>{
545                 m_board[i].first[source],
546                 m_board[i].first[target], false},
547             i, A,
548             std::array<std::size_t, 3>{source, target, N * M + 1});
549     }
550     m_piece_board[target] = std::move(m_piece_board[source]);
551     m_piece_board[source] = Piece();
552 }
553 }
554 }
555 }
556
557 template <std::size_t N, std::size_t M>
558 CONSTEXPR void
559 Board<N, M>::move_split_slide(
560     Coord const &s,
561     Coord const &t1,
562     Coord const &t2,
563     std::function<
564         bool(
565             Board<N, M> const &,
566             Coord const &,

```

```

567         Coord const &,
568         std::size_t,
569         std::optional<Coord>>>
570     check_path)
571 {
572     std::size_t source = offset(s.n, s.m);
573     std::size_t target1 = offset(t1.n, t1.m);
574     std::size_t target2 = offset(t2.n, t2.m);
575     std::size_t const size_board{std::size(m_board)};
576     for (std::size_t i{0}; i < size_board; i++)
577     {
578         move_1_instance(
579             std::array<bool, 5>{
580                 !check_path(*this, s, t2, i, t1),
581                 !check_path(*this, s, t1, i, t2),
582                 m_board[i].first[target2],
583                 m_board[i].first[target1],
584                 m_board[i].first[source]},
585             i, MATRIX_SPLIT_SLIDE,
586             std::array<std::size_t, 5>{
587                 N * M + 1,
588                 N * M + 1,
589                 target2,
590                 target1,
591                 source});
592         /*La matrice split slide est créée pour
593         être utilisé sans appliquer de porte cnot a
594         un chemin, nos fonctions check_path renvoie
595         un résultat où l'on a appliqué la porte cnot */
596     }

```

```

597     if (m_piece_board[target1].get_type() == TypePiece::EMPTY &&
598         m_piece_board[target2].get_type() == TypePiece::EMPTY)
599     {
600         m_piece_board[target1] = m_piece_board[source];
601         m_piece_board[target2] = m_piece_board[source];
602         update_case(source);
603         update_case(target1);
604         update_case(target2);
605     }
606     else
607     {
608         m_piece_board[target1] = m_piece_board[source];
609         m_piece_board[target2] = m_piece_board[source];
610         update_case(source);
611         update_case(target1);
612         update_case(target2);
613         update_board();
614     }
615 }
616
617 template <std::size_t N, std::size_t M>
618 CONSTEXPR void
619 Board<N, M>::move_merge_slide(
620     Coord const &s1,
621     Coord const &s2,
622     Coord const &t,
623     std::function<
624         bool(
625             Board<N, M> const &,
626             Coord const &,

```

```

627         Coord const &,
628         std::size_t,
629         std::optional<Coord>>>
630     check_path)
631 {
632     std::size_t const source1 = offset(s1.n, s1.m);
633     std::size_t const source2 = offset(s2.n, s2.m);
634     std::size_t const target = offset(t.n, t.m);
635     std::size_t const size_board{std::size(m_board)};
636     for (std::size_t i{0}; i < size_board; i++)
637     {
638         move_1_instance(
639             std::array<bool, 5>{
640                 !check_path(*this, s2, t, i, s1),
641                 !check_path(*this, s1, t, i, s2),
642                 m_board[i].first[source1],
643                 m_board[i].first[source2],
644                 m_board[i].first[target]},
645             i, MATRIX_MERGE_SLIDE,
646             std::array<std::size_t, 5>{
647                 N * M + 1,
648                 N * M + 1,
649                 source1,
650                 source2,
651                 target});
652         /* La matrice merge slide est créée pour
653         être utilisé sans appliquer de porte cnot au chemin,
654         nos fonctions check_path renvoie un résultat où l'on
655         a appliqué la porte cnot */
656     }

```

```

657     m_piece_board[target] = m_piece_board[source1];
658     update_board();
659     update_case(target);
660     update_case(source1);
661     update_case(source2);
662 }
663
664 template <std::size_t N, std::size_t M>
665 CONSTEXPR bool Board<N, M>::move_pawn(
666     Coord const &s,
667     Coord const &t,
668     std::optional<bool> val_mes) noexcept
669 {
670     bool move_exec{false};
671     auto abs_diff{[](std::size_t x, std::size_t y) -> std::size_t
672         {
673             return (x >= y) ? x - y : y - x;
674         }};
675     std::size_t diff_line{abs_diff(s.n, t.n)};
676     if (diff_line == 2)
677     {
678         move_exec = move_pawn_two_step(s, t, val_mes);
679         m_ep = Coord((s.n + t.n) / 2, s.m);
680     }
681     else if (diff_line == 1)
682     {
683         std::size_t diff_col{abs_diff(s.m, t.m)};
684         if (diff_col == 1)
685         {
686

```



```

687         if (m_ep != std::nullopt && m_ep == t)
688         {
689             int step{
690                 ((*this)(s.n, s.m).get_color() ==
691                     Color::WHITE)
692                     ? -1
693                     : 1};
694             Coord ep;
695             ep.m = m_ep->m;
696             ep.n = m_ep->n - step;
697             move_exec = move_enpassant(s, t, ep, val_mes);
698         }
699         else
700         {
701             move_exec = capture_pawn(s, t, val_mes);
702         }
703     }
704     else
705     {
706         move_exec = move_pawn_one_step(s, t, val_mes);
707     }
708     m_ep = std::nullopt;
709 }
710 return move_exec;
711 }
712
713 template <std::size_t N, std::size_t M>
714 constexpr void
715 Board<N, M>::move_promotion(
716     Move const &move,

```

```

717     std::optional<bool> val_mes)
718 {
719     TypePiece p{move.promote.piece};
720     Coord s{move.promote.src};
721     Coord t{move.promote.arv};
722
723     if (p == TypePiece::QUEEN ||
724         p == TypePiece::KNIGHT ||
725         p == TypePiece::BISHOP ||
726         p == TypePiece::ROOK)
727     {
728         if (move_pawn(s, t, val_mes))
729         {
730             Piece piece{p, m_piece_board[offset(t.n, t.m)].get_color()};
731             m_piece_board[offset(t.n, t.m)] = std::move(piece);
732         }
733     }
734     else
735     {
736         throw std::runtime_error("Piece non valide pour faire une promotion");
737     }
738 }
739
740 template <std::size_t N, std::size_t M>
741 constexpr void
742 Board<N, M>::move_classic(
743     Coord const &s,
744     Coord const &t,
745     std::optional<bool> val_mes)
746 {

```

```

747     TypePiece piece{(*this)(s.n, s.m).get_type()};
748     switch (piece)
749     {
750     case TypePiece::KING:
751         if constexpr (N == 8 && M == 8)
752         {
753             auto abs_diff{[](std::size_t x, std::size_t y) -> std::size_t
754                 {
755                     return (x >= y) ? x - y : y - x;
756                 }};
757             std::size_t diff_column{abs_diff(s.m, t.m)};
758             if (diff_column == 2)
759             {
760                 if (s.m > t.m)
761                 {
762                     queen_side_castle(s, Coord(s.n, t.m - 2), val_mes);
763                 }
764                 else
765                 {
766                     king_side_castle(s, Coord(s.n, t.m + 1), val_mes);
767                 }
768             }
769             else
770             {
771                 move_classic_jump(s, t, val_mes);
772             }
773         }
774     else
775     {
776         move_classic_jump(s, t, val_mes);

```

```
777     }
778     break;
779     case TypePiece::KNIGHT:
780         move_classic_jump(s, t, val_mes);
781         break;
782     case TypePiece::BISHOP:
783         move_classic_slide(
784             s, t,
785             &check_path_diagonal_1_instance<N, M>,
786             val_mes);
787         break;
788     case TypePiece::ROOK:
789         move_classic_slide(
790             s, t,
791             &check_path_straight_1_instance<N, M>,
792             val_mes);
793         break;
794     case TypePiece::QUEEN:
795         move_classic_slide(
796             s, t,
797             &check_path_queen_1_instance<N, M>,
798             val_mes);
799         break;
800     case TypePiece::PAWN:
801         move_pawn(s, t, val_mes);
802         return;
803     case TypePiece::EMPTY:
804     default:
805         break;
806 }
```

```

807     m_ep = std::nullopt;
808 }
809
810 template <std::size_t N, std::size_t M>
811 constexpr void Board<N, M>::move_split(Coord const &s,
812                                         Coord const &t1,
813                                         Coord const &t2)
814 {
815     TypePiece piece{(*this)(s.n, s.m).get_type()};
816     switch (piece)
817     {
818     case TypePiece::KING:
819     case TypePiece::KNIGHT:
820         move_split_jump(s, t1, t2);
821         break;
822     case TypePiece::BISHOP:
823         move_split_slide(s, t1, t2, &check_path_diagonal_1_instance<N, M>);
824         break;
825     case TypePiece::ROOK:
826         move_split_slide(s, t1, t2, &check_path_straight_1_instance<N, M>);
827         break;
828     case TypePiece::QUEEN:
829         move_split_slide(s, t1, t2, &check_path_queen_1_instance<N, M>);
830         break;
831     case TypePiece::PAWN:
832     case TypePiece::EMPTY:
833     default:
834         break;
835     }
836     m_ep = std::nullopt;

```

```

837 }
838
839 template <std::size_t N, std::size_t M>
840 constexpr void Board<N, M>::move_merge(Coord const &s1,
841                                         Coord const &s2,
842                                         Coord const &t)
843 {
844     TypePiece piece1{(*this)(s1.n, s1.m).get_type()};
845     TypePiece piece2{(*this)(s2.n, s2.m).get_type()};
846     if (piece1 != piece2)
847     {
848         return;
849     }
850     switch (piece1)
851     {
852     case TypePiece::KING:
853     case TypePiece::KNIGHT:
854         move_merge_jump(s1, s2, t);
855         break;
856     case TypePiece::BISHOP:
857         move_merge_slide(s1, s2, t, &check_path_diagonal_1_instance<N, M>);
858         break;
859     case TypePiece::ROOK:
860         move_merge_slide(s1, s2, t, &check_path_straight_1_instance<N, M>);
861         break;
862     case TypePiece::QUEEN:
863         move_merge_slide(s1, s2, t, &check_path_queen_1_instance<N, M>);
864         break;
865     case TypePiece::PAWN:
866     case TypePiece::EMPTY:

```

```

867     default:
868         break;
869     }
870     m_ep = std::nullopt;
871 }
872
873 template <std::size_t N, std::size_t M>
874 constexpr void Board<N, M>::move(Move const &movement, std::optional<bool> val_mes)
875 {
876     if (std::empty(m_board))
877     {
878         throw std::runtime_error("Le plateau est vide impossible de réaliser l'opération move");
879     }
880     switch (movement.type)
881     {
882     case TypeMove::NORMAL:
883         move_classic(movement.normal.src, movement.normal.arv, val_mes);
884         break;
885     case TypeMove::SPLIT:
886         move_split(movement.split.src,
887                     movement.split.arv1,
888                     movement.split.arv2);
889         break;
890     case TypeMove::MERGE:
891         move_merge(movement.merge.src1,
892                     movement.merge.src2,
893                     movement.merge.arv);
894         break;
895     case TypeMove::PROMOTE:
896         move_promotion(movement, val_mes);

```

Retour au début

```
897         break;
898     default:
899         break;
900 }
901 update_board_classic();
902 //update_board();
903
```


get-proba-move.tpp

Retour au début

```
1 // Lib standard
2 #include <cassert>
3 #include <utility>
4 #include <functional>
5 #include <cmath>
6 #include <optional>
7 #include <stdexcept>
8
9 // Inclusion projet
10 #include <Piece.hpp>
11 #include <TypePiece.hpp>
12 #include <Constexpr.hpp>
13 #include "Board.hpp"
14 #include <math_utility.hpp>
15
16 template <std::size_t N, std::size_t M>
17 constexpr double
18 Board<N, M>::get_proba_mesure(Coord const &p) const noexcept
19 {
20     double res{0};
21     std::size_t size_board = std::size(m_board);
22     for (std::size_t i{0}; i < size_board; i++)
23     {
24         if (m_board[i].first[offset(p.n, p.m)])
25         {
26             res += std::pow(std::abs(m_board[i].second), 2);
```

```

27     }
28 }
29 return res;
30 }
31
32 template <std::size_t N, std::size_t M>
33 CONSTEXPR double
34 Board<N, M>::get_proba_mesure_capture_slide(
35     Coord const &s,
36     Coord const &t,
37     std::function<bool(Board<N, M> const &,
38         Coord const &,
39         Coord const &,
40         std::size_t,
41         std::optional<Coord>)>
42     check_path) const noexcept
43 {
44     double res{0};
45     std::size_t position = offset(s.n, s.m);
46     std::size_t size_board = std::size(m_board);
47     for (std::size_t i{0}; i < size_board; i++)
48     {
49         if (m_board[i].first[position] &&
50             check_path(*this, s, t, i, std::nullopt))
51         {
52             res += std::pow(std::abs(m_board[i].second), 2);
53         }
54     }
55     return res;
56 }

```

```

57
58 template <std::size_t N, std::size_t M>
59 CONSTEXPR double
60 Board<N, M>::get_proba_mesure_castle(
61     Coord const &king,
62     Coord const &rook) const noexcept
63 {
64     double res{0};
65     std::size_t size_board = std::size(m_board);
66     std::size_t position_king = offset(king.n, king.m);
67     std::size_t position_rook = offset(rook.n, rook.m);
68     for (std::size_t i{0}; i < size_board; i++)
69     {
70         if (m_board[i].first[position_king] &&
71             m_board[i].first[position_rook] &&
72             check_path_straight_1_instance(*this, king, rook, i, std::nullopt))
73         {
74             res += std::pow(std::abs(m_board[i].second), 2);
75         }
76     }
77     return res;
78 }
79
80 template <std::size_t N, std::size_t M>
81 CONSTEXPR double Board<N, M>::get_proba_move(Move const &move) const noexcept
82 {
83     Coord s;
84     Coord t;
85     switch (move.type)
86     {

```

```

87     case TypeMove::NORMAL:
88         s = move.normal.src;
89         t = move.normal.arv;
90         break;
91     case TypeMove::PROMOTE:
92         s = move.promote.src;
93         t = move.promote.arv;
94         break;
95     case TypeMove::MERGE:
96         return 1.;
97     case TypeMove::SPLIT:
98         return 1.;
99     default:
100         return 1.;
101 }
102 TypePiece piece{(*this)(s.n, s.m).get_type()};
103 TypePiece piece_target{(*this)(t.n, t.m).get_type()};
104 if((piece == piece_target) && (*this)(s.n, s.m).same_color((*this)(t.n, t.m)))
105 {
106     return 1.;
107 }
108 switch (piece)
109 {
110 case TypePiece::KING:
111     if constexpr (N == 8 && M == 8)
112     {
113         auto abs_diff{[](std::size_t x, std::size_t y) -> std::size_t
114             {
115                 return (x >= y) ? x - y : y - x;
116             }};

```

```

117     std::size_t diff_colum{abs_diff(s.m, t.m)};
118     if (diff_colum == 2)
119     {
120         if (s.m > t.m)
121         {
122             return get_proba_mesure_castle(s, Coord(s.n, t.m - 2));
123         }
124         else
125         {
126             return get_proba_mesure_castle(s, Coord(s.n, t.m + 1));
127         }
128     }
129     else
130     {
131         if (piece_target != TypePiece::EMPTY)
132         {
133             if ((*this)(s.n, s.m).same_color((*this)(t.n, t.m)))
134             {
135                 return 1. - get_proba_mesure(t);
136             }
137             else
138             {
139                 return get_proba_mesure(s);
140             }
141         }
142         else
143         {
144             return 1.;
145         }
146     }

```

```
147     }
148     else
149     {
150         if (piece_target != TypePiece::EMPTY)
151         {
152             if ((*this)(s.n, s.m).same_color((*this)(t.n, t.m)))
153             {
154                 return 1. - get_proba_mesure(t);
155             }
156             else
157             {
158                 return get_proba_mesure(s);
159             }
160         }
161         else
162         {
163             return 1.;
164         }
165     }
166     break;
167 case TypePiece::KNIGHT:
168     if (piece_target != TypePiece::EMPTY)
169     {
170         if ((*this)(s.n, s.m).same_color((*this)(t.n, t.m)))
171         {
172             return 1. - get_proba_mesure(t);
173         }
174         else
175         {
176             return get_proba_mesure(s);
```

```

177         }
178     }
179     else
180     {
181         return 1.;
182     }
183
184     break;
185 case TypePiece::BISHOP:
186     if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY)
187     {
188         return 1.;
189     }
190     else
191     {
192         if (!(*this)(t.n, t.m).same_color((*this)(s.n, s.m)))
193         {
194             return get_proba_mesure_capture_slide(
195                 s, t,
196                 &check_path_diagonal_1_instance<N, M>);
197         }
198         else
199         {
200             return 1. - get_proba_mesure(t);
201         }
202     }
203     break;
204 case TypePiece::ROOK:
205     if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY)
206     {

```

```

207         return 1.;
208     }
209     else
210     {
211         if (!(*this)(t.n, t.m).same_color((*this)(s.n, s.m)))
212         {
213             return get_proba_mesure_capture_slide(
214                 s, t,
215                 &check_path_straight_1_instance<N, M>);
216         }
217         else
218         {
219             return 1. - get_proba_mesure(t);
220         }
221     }
222     break;
223 case TypePiece::QUEEN:
224     if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY)
225     {
226         return 1.;
227     }
228     else
229     {
230         if (!(*this)(t.n, t.m).same_color((*this)(s.n, s.m)))
231         {
232             return get_proba_mesure_capture_slide(
233                 s, t,
234                 &check_path_queen_1_instance<N, M>);
235         }
236         else

```


Retour au début

```
237         {
238             return 1. - get_proba_mesure(t);
239         }
240     }
241     break;
242     case TypePiece::PAWN:
243         if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY ||
244             ((*this)(t.n, t.m).get_type() == (*this)(s.n, s.m).get_type() &&
245              (*this)(t.n, t.m).get_color() == (*this)(s.n, s.m).get_color()))
246         {
247             return 1.;
248         }
249         else
250         {
251             return 1. - get_proba_mesure(t);
252         }
253     case TypePiece::EMPTY:
254     default:
255         break;
256     }
257     return 1.;
258 }
```

Piece.hpp

Retour au début

```
1  #ifndef PIECE_HPP
2  #define PIECE_HPP
3
4  #include <Color.hpp>
5  #include <Coord.hpp>
6  #include <concepts>
7  #include <vector>
8  #include <forward_list>
9  #include <constexpr.hpp>
10 #include <Move.hpp>
11 #include "TypePiece.hpp"
12
13 template <std::size_t N, std::size_t M>
14 class Board;
15
16 /**
17  * @brief Class conservant le type et la couleur d'une pièce.
18  * Elle permet aussi de recuperer ses mouvement possible
19  */
20 class Piece
21 {
22 public:
23     constexpr inline Piece() noexcept;
24
25     /**
26      * @brief Construit une piece à l'aide de son type et de sa couleur
```

```

27      *
28      * @param[in] piece Le type de la piece
29      * @param[in] color La couleur de la piece
30      */
31      constexpr inline Piece(TypePiece piece, Color color) noexcept;
32      constexpr inline Piece(Piece const &) = default;
33      constexpr inline Piece &operator=(Piece const &) = default;
34
35      /**
36       * @brief Deplacement d'un objet piece vers un autre objet piece
37       */
38      constexpr inline Piece(Piece &&) = default;
39      constexpr inline Piece &operator=(Piece &&) = default;
40      constexpr inline ~Piece() = default;
41
42      /**
43       * @brief Renvoie le type de la piece
44       *
45       * @return Une énumération TypePiece
46       */
47      constexpr inline TypePiece get_type() const noexcept;
48
49      /**
50       * @brief Renvoie la couleur d'un piece
51       *
52       * @return Color La couleur de la piece
53       */
54      constexpr inline Color get_color() const noexcept;
55
56      /**

```

```

57      * @brief Récupère la liste des mouvements classiques pour une piece
58      *
59      * @tparam N Le nombre de ligne du plateau
60      * @tparam M Le nombre de colonne du plateau
61      * @param[in] board Le plateau
62      * @param[in] pos la position de la piece sur le plateau
63      * @warning Le resultat n'est pas garanti si la position ne concorde pas
64      * avec la position de la piece sur le plateau
65      * @return std::forward_list<Coord> La liste de coordonnées des positions
66      * d'arrivées de chaque mouvement possible
67      */
68      template <std::size_t N, std::size_t M>
69      CONSTEXPR std::forward_list<Coord>
70      get_list_normal_move(Board<N, M> const &board,
71                          Coord const &pos) const noexcept;
72
73      /**
74       * @brief Récupère la liste des mouvements split pour une piece
75       *
76       * @tparam N Le nombre de ligne du plateau
77       * @tparam M Le nombre de colonne du plateau
78       * @param[in] board Le plateau
79       * @param[in] pos la position de la piece sur le plateau
80       * @warning Le resultat n'est pas garanti si la position ne concorde pas
81       * avec la position de la piece sur le plateau
82       * @warning Ne renvoie qu'une liste de coordonnées uniques car
83       * l'ensemble des mouvements possibles est toutes les couples
84       * de coordonnées de la liste
85       * @return std::forward_list<Coord> La liste de coordonnées
86       * des positions d'arrivées possible sachant que chaque élément

```

```

87      * représente une seule des deux cases d'arrivées d'un split move
88      */
89      template <std::size_t N, std::size_t M>
90      CONSTEXPR std::forward_list<Coord>
91      get_list_split_move(Board<N, M> const &board,
92                          Coord const &pos) const noexcept;
93
94      /**
95       * @brief Teste si la piece est blanche
96       *
97       * @return bool true si la piece est blanche, false sinon
98       */
99      constexpr inline bool is_white() const noexcept;
100
101      /**
102       * @brief Teste si la piece est noire
103       *
104       * @return bool true si la piece est noire, false sinon
105       */
106      constexpr inline bool is_black() const noexcept;
107
108      /**
109       * @brief Teste si l'autre piece est de la meme couleur
110       *
111       * @param[in] other L'autre pièce à comparer
112       *
113       * @return bool true si la piece est de la meme couleur, false sinon
114       */
115      constexpr inline bool same_color(Piece const &other) const noexcept;
116

```

```

117  /**
118   * @brief Teste si les mouvement de la piece sont si la
119   * pièce est un pion un mouvement de promotion
120   *
121   * @note peut etre utiliser sans verifier si la pièce est un pion
122   *
123   * @param[in] board Le plateau
124   * @param[in] pos La position de la pièce
125   * @return true si le mouvement possible est un mouvement
126   * de promotion
127   * @return false sinon
128   */
129  template <std::size_t N, std::size_t M>
130  CONSTEXPR bool
131  check_if_use_move_promote(Board<N, M> const &board,
132                           Coord const &pos) const noexcept;
133
134  /**
135   * @brief Renvoie la liste de toutes les promotions
136   *
137   * @warning Ne verifie pas la validité du mouvement,
138   * de la position ou du type de la pièce
139   *
140   * @param[in] board Le plateau
141   * @param[in] pos La position du pion
142   * @return La liste de tout les mouvements de promotion
143   */
144  template <std::size_t N, std::size_t M>
145  CONSTEXPR std::forward_list<Move>
146  get_list_promote(Board<N, M> const &board,

```

```

147         Coord const &pos) const noexcept;
148
149 private:
150     /**
151      * @brief Énumération utilisé dans le template des fonctions
152      * qui récupère les listes de mouvement pour modifier leurs
153      * comportement lors de la prise de pièce.
154      */
155     enum class Move_Mode
156     {
157         /**
158          * @brief Dans le cas du mouvement classique on
159          * autorise la prise de pièce.
160          */
161         NORMAL,
162
163         /**
164          * @brief Dans le cas du split on interdit la
165          * prise de pièce.
166          */
167         SPLIT
168     };
169
170     /**
171      * @brief renvoie la valeur absolue d'une soustraction sur des types size_t
172      *
173      * @param[in] x Variable de type size_t
174      * @param[in] y Variable de type size_t
175      * @return std::size_t Retourne l'opération |x - y|
176      */

```

```

177     constexpr inline static std::size_t
178     abs_subtracte(std::size_t x, std::size_t y) noexcept;
179
180     /**
181      * @brief renvoie la distance entre deux coordonnées
182      * à l'aide de la norme 2
183      *
184      * @param[in] x Première coordonnée
185      * @param[in] y Seconde coordonnée
186      * @return double le résultat de la norme 1
187      */
188     constexpr inline static double
189     norm(Coord const &x, Coord const &y) noexcept;
190
191     /**
192      * @brief Récupère la liste de mouvement d'une pièce pour
193      * les déplacements indiqué de manière récursive
194      * c'est à dire tout les mouvements infini (ex : diagonale du fou)
195      *
196      * @tparam MOVE Le type de mouvement utiliser entre normal
197      * et split (split ne prend pas les mouvements de capture de pièce)
198      * @tparam N Le nombre de ligne du plateau
199      * @tparam M Le nombre de colonne du plateau
200      * @tparam SIZE La taille de la liste des mouvements possible
201      * @param[in] board Le plateau
202      * @param[in] pos La position de la pièce
203      * @param[in] list_move La liste de tout les mouvements possible
204      * @return std::forward_list<Coord> La liste de coordonnées
205      * des positions d'arrivées possible
206      */

```



```

207     template <Move_Mode MOVE, std::size_t N, std::size_t M, std::size_t SIZE>
208     CONSTEXPR std::forward_list<Coord>
209     get_list_move_rec(Board<N, M> const &board, Coord const &pos,
210                     std::array<int, SIZE> const &list_move) const noexcept;
211
212     /**
213     * @brief Récupère la liste de mouvement du roi
214     *
215     * @warning Ne récupère les mouvements du roque que pour un tableau
216     * de taille 8 x 8
217     *
218     * @warning ne verifie pas si c'est bien un roi
219     *
220     * @tparam MOVE Le type de mouvement utiliser entre normal
221     * et split (split ne prend pas les mouvements de capture de pièce)
222     * @tparam N Le nombre de ligne du plateau
223     * @tparam M Le nombre de colonne du plateau
224     * @param[in] board Le plateau
225     * @param[in] pos La position du roi
226     * @return std::forward_list<Coord> La liste de coordonnées
227     * des positions d'arrivées possible pour le roi
228     */
229     template <Move_Mode MOVE, std::size_t N, std::size_t M>
230     CONSTEXPR std::forward_list<Coord>
231     get_list_move_king(Board<N, M> const &board,
232                      Coord const &pos) const noexcept;
233
234     /**
235     * @brief Récupère la liste de mouvement du cavalier
236     *

```

```

237      * @warning ne verifie pas si c'est bien un cavalier
238      *
239      * @tparam MOVE Le type de mouvement utiliser entre normal
240      * et split (split ne prend pas les mouvements de capture de pièce)
241      * @tparam N Le nombre de ligne du plateau
242      * @tparam M Le nombre de colonne du plateau
243      * @param[in] board Le plateau
244      * @param[in] pos La position du cavalier
245      * @return std::forward_list<Coord> La liste de coordonnées
246      * des positions d'arrivées possible pour le cavalier
247      */
248      template <Move_Mode MOVE, std::size_t N, std::size_t M>
249      CONSTEXPR std::forward_list<Coord>
250      get_list_move_knight(Board<N, M> const &board,
251                          Coord const &pos) const noexcept;
252
253      /**
254       * @brief Récupère la liste de mouvement du fou
255       *
256       * @warning ne verifie pas si c'est bien un fou
257       *
258       * @tparam MOVE Le type de mouvement utiliser entre normal
259       * et split (split ne prend pas les mouvements de capture de pièce)
260       * @tparam N Le nombre de ligne du plateau
261       * @tparam M Le nombre de colonne du plateau
262       * @param[in] board Le plateau
263       * @param[in] pos La position du fou
264       * @return std::forward_list<Coord> La liste de coordonnées
265       * des positions d'arrivées possible pour le fou
266       */

```

```

267     template <Move_Mode MOVE, std::size_t N, std::size_t M>
268     CONSTEXPR std::forward_list<Coord>
269     get_list_move_bishop(Board<N, M> const &board,
270                          Coord const &pos) const noexcept;
271
272     /**
273      * @brief Récupère la liste de mouvement de la tour
274      *
275      * @warning ne verifie pas si c'est bien une tour
276      *
277      * @tparam MOVE Le type de mouvement utiliser entre normal
278      * et split (split ne prend pas les mouvements de capture de pièce)
279      * @tparam N Le nombre de ligne du plateau
280      * @tparam M Le nombre de colonne du plateau
281      * @param[in] board Le plateau
282      * @param[in] pos La position de la tour
283      * @return std::forward_list<Coord> La liste de coordonnées
284      * des positions d'arrivées possible pour la tour
285      */
286     template <Move_Mode MOVE, std::size_t N, std::size_t M>
287     CONSTEXPR std::forward_list<Coord>
288     get_list_move_rook(Board<N, M> const &board,
289                        Coord const &pos) const noexcept;
290
291     /**
292      * @brief Récupère la liste de mouvement de la dame
293      *
294      * @warning ne verifie pas si c'est bien une dame
295      *
296      * @tparam MOVE Le type de mouvement utiliser entre normal

```

```

297     * et split (split ne prend pas les mouvements de capture de pièce)
298     * @tparam N Le nombre de ligne du plateau
299     * @tparam M Le nombre de colonne du plateau
300     * @param[in] board Le plateau
301     * @param[in] pos La position de la dame
302     * @return std::forward_list<Coord> La liste de coordonnées
303     * des positions d'arrivées possible pour la dame
304     */
305     template <Move_Mode MOVE, std::size_t N, std::size_t M>
306     CONSTEXPR std::forward_list<Coord>
307     get_list_move_queen(Board<N, M> const &board,
308                        Coord const &pos) const noexcept;
309
310     /**
311     * @brief Récupère la liste de mouvement du pion
312     *
313     * @warning ne verifie pas si c'est bien un pion
314     *
315     * @tparam N Le nombre de ligne du plateau
316     * @tparam M Le nombre de colonne du plateau
317     * @param[in] board Le plateau
318     * @param[in] pos La position du pion
319     * @return std::forward_list<Coord> La liste de coordonnées
320     * des positions d'arrivées possible pour le pion
321     */
322     template <std::size_t N, std::size_t M>
323     CONSTEXPR std::forward_list<Coord>
324     get_list_move_pawn(Board<N, M> const &board,
325                       Coord const &pos) const noexcept;
326

```

```

327  /**
328   * @brief Récupère la liste de mouvement pour
329   * n'importe qu'elle pièce
330   *
331   * @tparam MOVE Le type de mouvement utiliser entre normal
332   * et split (split ne prend pas les mouvements de capture de pièce)
333   * @tparam N Le nombre de ligne du plateau
334   * @tparam M Le nombre de colonne du plateau
335   * @param[in] board Le plateau
336   * @param[in] pos La position de la pièce
337   * @param[in] list_move La liste de tout les mouvements possible
338   * @return std::forward_list<Coord> La liste de coordonnées
339   * des positions d'arrivées possible
340   */
341  template <Move_Mode MOVE, std::size_t N, std::size_t M>
342  CONSTEXPR std::forward_list<Coord>
343  get_list_move(Board<N, M> const &board,
344               Coord const &pos) const noexcept;
345
346  /**
347   * @brief La couleur de la pièce (WHITE/BLACK)
348   */
349  Color m_color;
350
351  /**
352   * @brief Le type de la piece
353   * ( KING / QUEEN / ROOK / BISHOP / KNIGHT / PAWN )
354   */
355  TypePiece m_type;
356  };

```

```
357
358 #include "Piece.tpp"
359 #include "get_list_move.tpp"
360
361 /**
362  * @brief L'objet représentant le roi blanc
363  */
364 constexpr Piece W_KING{TypePiece::KING, Color::WHITE};
365
366 /**
367  * @brief L'objet représentant le roi noir
368  */
369 constexpr Piece B_KING{TypePiece::KING, Color::BLACK};
370
371 /**
372  * @brief L'objet représentant la renne blanche
373  */
374 constexpr Piece W_QUEEN{TypePiece::QUEEN, Color::WHITE};
375
376 /**
377  * @brief L'objet représentant la renne noire
378  */
379 constexpr Piece B_QUEEN{TypePiece::QUEEN, Color::BLACK};
380
381 /**
382  * @brief L'objet représentant le fou blanc
383  */
384 constexpr Piece W_BISHOP{TypePiece::BISHOP, Color::WHITE};
385
386 /**
```

```
387  * @brief L'objet représentant le fou noir
388  */
389  constexpr Piece B_BISHOP{TypePiece::BISHOP, Color::BLACK};
390
391  /**
392   * @brief L'objet représentant la tour blanche
393   */
394  constexpr Piece W_ROOK{TypePiece::ROOK, Color::WHITE};
395
396  /**
397   * @brief L'objet représentant la tour noire
398   */
399  constexpr Piece B_ROOK{TypePiece::ROOK, Color::BLACK};
400
401  /**
402   * @brief L'objet représentant le cavalier blanc
403   */
404  constexpr Piece W_KNIGHT{TypePiece::KNIGHT, Color::WHITE};
405
406  /**
407   * @brief L'objet représentant le cavalier noir
408   */
409  constexpr Piece B_KNIGHT{TypePiece::KNIGHT, Color::BLACK};
410
411  /**
412   * @brief L'objet représentant le pion blanc
413   */
414  constexpr Piece W_PAWN{TypePiece::PAWN, Color::WHITE};
415
416  /**
```

Retour au début

```
417  * @brief L'objet représentant le pion noir
418  */
419  constexpr Piece B_PAWN{TypePiece::PAWN, Color::BLACK};
420
421
```


Piece.hpp

Retour au début

```
1  #include "Piece.hpp"
2  #include <Coord.hpp>
3  #include <check_path.hpp>
4  #include <cmath>
5  #include <forward_list>
6  #include <observer_ptr.hpp>
7  #include <math_utility.hpp>
8  #include <Constexpr.hpp>
9  #include <Move.hpp>
10
11 #define POW2(x) ((x) * (x))
12
13 constexpr inline Piece::Piece() noexcept
14     : m_color(Color::BLACK),
15       m_type(TypePiece::EMPTY)
16 {
17 }
18
19 constexpr inline Piece::Piece(TypePiece piece, Color color) noexcept
20     : m_color(color),
21       m_type(piece)
22 {
23 }
24
25 constexpr inline bool Piece::is_white() const noexcept
26 {
```

```
27     return m_color == Color::WHITE;
28 }
29
30 constexpr inline bool Piece::is_black() const noexcept
31 {
32     return m_color == Color::BLACK;
33 }
34
35 constexpr inline bool
36 Piece::same_color(Piece const &other) const noexcept
37 {
38     return m_color == other.m_color;
39 }
40
41 constexpr inline TypePiece Piece::get_type() const noexcept
42 {
43     return m_type;
44 }
45
46 constexpr inline Color Piece::get_color() const noexcept
47 {
48     return m_color;
49 }
50
51 constexpr inline std::size_t
52 Piece::abs_substracte(std::size_t x, std::size_t y) noexcept
53 {
54     if (x >= y)
55     {
56         return x - y;
```

```

57     }
58     return y - x;
59 }
60
61 constexpr inline double
62 Piece::norm(Coord const &x, Coord const &y) noexcept
63 {
64     return sqrt(
65         static_cast<double>(POW2(abs_substracte(x.n, y.n)) +
66                             POW2(abs_substracte(x.m, y.m))));
67 }
68
69 template <std::size_t N, std::size_t M>
70 CONSTEXPR bool
71 Piece::check_if_use_move_promote(Board<N, M> const &board,
72                                   Coord const &pos) const noexcept
73 {
74     if constexpr (N >= 2)
75     {
76         if (board(pos.n, pos.m).get_type() == TypePiece::PAWN)
77         {
78             auto sign_color{
79                 [](Color color) -> int
80                 {
81                     return (color == Color::WHITE) ? -1 : 1;
82                 }
83             };
84             std::size_t required_line{(get_color() == Color::WHITE) ? 1 : N - 2};
85             if (pos.n == required_line)
86             {
87                 return true;
88             }
89         }
90     }
91 }

```

Retour au début

```
87         }  
88     }  
89 }  
90 return false;  
91
```

get-list-move.hpp

Retour au début

```
1  #include "Piece.hpp"
2  #include <Coord.hpp>
3  #include <check_path.hpp>
4  #include <cmath>
5  #include <forward_list>
6  #include <observer_ptr.hpp>
7  #include <math_utility.hpp>
8  #include <Constexpr.hpp>
9  #include <Move.hpp>
10
11 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
12 constexpr std::forward_list<Coord>
13 Piece::get_list_move_king(Board<N, M> const &board,
14                          Coord const &pos) const noexcept
15 {
16     constexpr int P{M + 2};
17     std::array<int, 8> const
18         move_king{{-P - 1, -P, -P + 1, -1, 1, P - 1, P, P + 1}};
19     std::size_t current_pos{board.offset(pos.n, pos.m)};
20     int posBox{board.m_S_mailbox[current_pos]};
21     std::forward_list<Coord> list_move{};
22     for (int const e : move_king)
23     {
24         int arv{board.m_L_mailbox[posBox + e]};
25         if (arv >= 0)
26         {
```

```

27         std::size_t n{arv / M}, m{arv % M};
28         Piece const &arvPiece{board(n, m)};
29         bool eval {false};
30         if CONSTEXPR (MOVE == Move_Mode::NORMAL)
31         {
32             eval = arvPiece.get_type() == TypePiece::EMPTY ||
33                 !same_color(arvPiece) ||
34                 board.get_proba(Coord(n, m)) < 1. - EPSILON;
35         }
36         #if !defined(KING_NO_SPLIT)
37             else
38             {
39                 eval = arvPiece.get_type() == TypePiece::EMPTY;
40             }
41         #endif
42         if (eval)
43         {
44             list_move.push_front(Coord(n, m));
45         }
46     }
47 }
48 if CONSTEXPR (N == 8 && M == 8 && MOVE == Move_Mode::NORMAL)
49 {
50     if (board.m_q_castle[static_cast<int>(get_color())] &&
51         check_path_straight(board, pos, Coord(pos.n, pos.m - 4)))
52     {
53         list_move.push_front(Coord(pos.n, pos.m - 2));
54     }
55     if (board.m_k_castle[static_cast<int>(get_color())] &&
56         check_path_straight(board, pos, Coord(pos.n, pos.m + 3)))

```

```

57     {
58         list_move.push_front(Coord(pos.n, pos.m + 2));
59     }
60 }
61 return list_move;
62 }
63
64 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
65 CONSTEXPR std::forward_list<Coord>
66 Piece::get_list_move_knight(Board<N, M> const &board,
67                             Coord const &pos) const noexcept
68 {
69     CONSTEXPR int P{M + 2};
70     std::array<int, 8> const
71         move_knight{{-2 * P - 1, -P - 2, -2 * P + 1, -P + 2,
72                     P - 2, 2 * P - 1, 2 * P + 1, P + 2}};
73     std::size_t current_pos{board.offset(pos.n, pos.m)};
74     int posBox{board.m_S_mailbox[current_pos]};
75     std::forward_list<Coord> list_move{};
76     for (int const e : move_knight)
77     {
78         int arv{board.m_L_mailbox[posBox + e]};
79         if (arv >= 0)
80         {
81             std::size_t n{arv / M}, m{arv % M};
82             Piece const &arvPiece{board(n, m)};
83             bool eval;
84             if CONSTEXPR (MOVE == Move_Mode::NORMAL)
85             {
86                 eval = arvPiece.get_type() == TypePiece::EMPTY ||

```

```

87             !same_color(arvPiece) ||
88             board.get_proba(Coord(n, m)) < 1. - EPSILON;
89         }
90     else
91     {
92         eval = arvPiece.get_type() == TypePiece::EMPTY;
93     }
94     if (eval)
95     {
96         list_move.push_front(Coord(n, m));
97     }
98     }
99 }
100 return list_move;
101 }
102
103 template <Piece::Move_Mode MOVE,
104           std::size_t N,
105           std::size_t M,
106           std::size_t SIZE>
107 CONSTEXPR std::forward_list<Coord>
108 Piece::get_list_move_rec(
109     Board<N, M> const &board,
110     Coord const &pos,
111     std::array<int, SIZE> const &list_move) const noexcept
112 {
113     std::size_t current_pos{board.offset(pos.n, pos.m)};
114     int const posBox{board.m_S_mailbox[current_pos]};
115     std::forward_list<Coord> move{};
116     for (int const e : list_move)

```



```

117 {
118     int arv{board.m_L_mailbox[posBox + e]};
119     int posBox_tmp{posBox};
120     while (arv >= 0)
121     {
122         std::size_t n{arv / M}, m{arv % M};
123         Piece const &arvPiece{board(n, m)};
124         if CONSTEXPR (MOVE == Move_Mode::NORMAL)
125         {
126             if (arvPiece.get_type() == TypePiece::EMPTY ||
127                 board.get_proba(Coord(n, m)) < 1. - EPSILON)
128             {
129                 move.push_front(Coord(n, m));
130             }
131             else if (!same_color(arvPiece))
132             {
133                 move.push_front(Coord(n, m));
134                 break;
135             }
136             else
137             {
138                 break;
139             }
140         }
141         else
142         {
143             if (arvPiece.get_type() == TypePiece::EMPTY ||
144                 (arvPiece.get_type() == board(pos.n, pos.m).get_type() &&
145                  arvPiece.get_color() == board(pos.n, pos.m).get_color()))
146             {

```

```

147         move.push_front(Coord(n, m));
148     }
149     else
150     {
151         break;
152     }
153 }
154 posBox_tmp = board.m_S_mailbox[arv];
155 arv = board.m_L_mailbox[posBox_tmp + e];
156 }
157 }
158 return move;
159 }
160
161 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
162 CONSTEXPR std::forward_list<Coord>
163 Piece::get_list_move_bishop(Board<N, M> const &board,
164                             Coord const &pos) const noexcept
165 {
166     CONSTEXPR int P{M + 2};
167     std::array<int, 4> const move_bishop{{-P - 1, -P + 1, P - 1, P + 1}};
168     return get_list_move_rec<MOVE>(board, pos, move_bishop);
169 }
170
171 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
172 CONSTEXPR std::forward_list<Coord>
173 Piece::get_list_move_rook(Board<N, M> const &board,
174                           Coord const &pos) const noexcept
175 {
176     CONSTEXPR int P{M + 2};

```

```

177     std::array<int, 4> const move_rook{{-P, 1, P, -1}};
178     return get_list_move_rec<MOVE>(board, pos, move_rook);
179 }
180
181 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
182 CONSTEXPR std::forward_list<Coord>
183 Piece::get_list_move_queen(Board<N, M> const &board,
184                           Coord const &pos) const noexcept
185 {
186     CONSTEXPR int P{M + 2};
187     std::array<int, 8> const
188         move_queen{{-P - 1, -P, -P + 1, -1, 1, P - 1, P, P + 1}};
189     return get_list_move_rec<MOVE>(board, pos, move_queen);
190 }
191
192 template <std::size_t N, std::size_t M>
193 CONSTEXPR std::forward_list<Coord>
194 Piece::get_list_move_pawn(Board<N, M> const &board,
195                           Coord const &pos) const noexcept
196 {
197     CONSTEXPR int P{M + 2};
198     auto sign_color{
199         [](Color color)
200         {
201             return (color == Color::WHITE) ? -1 : 1;
202         }
203     };
204     int sign{sign_color(get_color())};
205     std::array<int, 3> move_pawn{sign * P, sign * P - 1, sign * P + 1};
206     std::forward_list<Coord> list_move{};
207     std::size_t current_pos{board.offset(pos.n, pos.m)};

```

```

207     int posBox{board.m_S_mailbox[current_pos]};
208     int arv{board.m_L_mailbox[posBox + move_pawn[0]]};
209     if (arv >= 0)
210     {
211         std::size_t n{arv / N}, m{arv % N};
212         if (board(n, m).get_type() == TypePiece::EMPTY ||
213             board.get_proba(Coord(n, m)) < 1. - EPSILON)
214         {
215             list_move.push_front(Coord(n, m));
216             if (((pos.n == 1 && get_color() == Color::BLACK) ||
217                 (pos.n == N - 2 && get_color() == Color::WHITE)) &&
218                 (board(n + sign, m).get_type() == TypePiece::EMPTY ||
219                     board.get_proba(Coord(n + sign, m)) < 1. - EPSILON))
220             {
221                 list_move.push_front(Coord(n + sign, m));
222             }
223         }
224     }
225     for (std::size_t i{1}; i < 3; i++)
226     {
227         arv = board.m_L_mailbox[posBox + move_pawn[i]];
228         if (arv >= 0)
229         {
230             std::size_t n{arv / N}, m{arv % N};
231             if ((board(n, m).get_type() != TypePiece::EMPTY &&
232                 !same_color(board(n, m))) ||
233                 (board.m_ep != std::nullopt &&
234                     board.m_ep->n == pos.n &&
235                     board.m_ep->m == pos.m))
236             {

```

```

237         list_move.push_front(Coord(n, m));
238     }
239 }
240 }
241 return list_move;
242 }
243
244 template <std::size_t N, std::size_t M>
245 CONSTEXPR std::forward_list<Move>
246 Piece::get_list_promote(Board<N, M> const &board, Coord const &pos) const noexcept
247 {
248     std::forward_list<Coord> list_move{get_list_move_pawn(board, pos)};
249     std::forward_list<Move> list_promote;
250     std::array<TypePiece, 4> promote_choice{TypePiece::QUEEN,
251                                             TypePiece::ROOK,
252                                             TypePiece::BISHOP,
253                                             TypePiece::KNIGHT};
254     for (Coord const &e : list_move)
255     {
256         for (TypePiece p : promote_choice)
257         {
258             list_promote.push_front(Move_promote(pos, e, p));
259         }
260     }
261     return list_promote;
262 }
263
264 template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
265 CONSTEXPR std::forward_list<Coord>
266 Piece::get_list_move(Board<N, M> const &board, Coord const &pos) const noexcept

```

```

267 {
268     switch (get_type())
269     {
270     case TypePiece::PAWN:
271         if CONSTEXPR (MOVE == Move_Mode::NORMAL)
272         {
273             return get_list_move_pawn(board, pos);
274         }
275         else
276         {
277             return std::forward_list<Coord>{};
278         }
279     case TypePiece::KNIGHT:
280         return get_list_move_knight<MOVE>(board, pos);
281     case TypePiece::BISHOP:
282         return get_list_move_bishop<MOVE>(board, pos);
283     case TypePiece::ROOK:
284         return get_list_move_rook<MOVE>(board, pos);
285     case TypePiece::QUEEN:
286         return get_list_move_queen<MOVE>(board, pos);
287     case TypePiece::KING:
288         return get_list_move_king<MOVE>(board, pos);
289     case TypePiece::EMPTY:
290     default:
291         return std::forward_list<Coord>{};
292     }
293 }
294
295 template <std::size_t N, std::size_t M>
296 CONSTEXPR std::forward_list<Coord>

```

Retour au début

```
297 Piece::get_list_normal_move(Board<N, M> const &board,
298                               Coord const &pos) const noexcept
299 {
300     return get_list_move<Move_Mode::NORMAL>(board, pos);
301 }
302
303 template <std::size_t N, std::size_t M>
304 CONSTEXPR std::forward_list<Coord>
305 Piece::get_list_split_move(Board<N, M> const &board,
306                             Coord const &pos) const noexcept
307 {
308     return get_list_move<Move_Mode::SPLIT>(board, pos);
309 }
```

TypePiece.hpp

Retour au début

```
1  #ifndef TYPE_PIECE_HPP
2  #define TYPE_PIECE_HPP
3
4  /**
5   * @brief Enumère toutes les sortes de pièces
6   */
7  enum class TypePiece
8  {
9      EMPTY,
10     PAWN,
11     KNIGHT,
12     BISHOP,
13     ROOK,
14     QUEEN,
15     KING
16 };
17
18
```


Coord.hpp

Retour au début

```
1  #ifndef COORD_HPP
2  #define COORD_HPP
3
4  #include <csddef>
5
6  /**
7   * @brief La structure qui représente une coordonnée
8   * sur le plateau
9   */
10 struct Coord
11 {
12     Coord() = default;
13     Coord(std::size_t i, std::size_t j) : n(i), m(j) {};
14     Coord(Coord const&) = default;
15     Coord& operator=(Coord const&) = default;
16     Coord(Coord &&) = default;
17     Coord& operator=(Coord &&) = default;
18
19
20     /**
21      * @brief l'indice sur les lignes
22      */
23     std::size_t n;
24
25     /**
26      * @brief l'indice sur les colonnes
```

Retour au début

```
27     */
28     std::size_t m;
29 };
30
31
32 struct Coord_hash
33 {
34     std::size_t operator () (Coord const &coord) const;
35 };
36
37 bool operator==(Coord const &lhs, Coord const &rhs);
38
39
```

Coord.cpp

Retour au début

```
1  #include "Coord.hpp"
2  #include <functional>
3
4  std::size_t Coord_hash::operator () (Coord const &coord) const
5  {
6      std::size_t h1 = std::hash<std::size_t>()(coord.n);
7      std::size_t h2 = std::hash<std::size_t>()(coord.m);
8
9      return h1 ^ h2;
10 }
11
12
13 bool operator==(Coord const &lhs, Coord const &rhs)
14 {
15     return lhs.n == rhs.n && lhs.m == rhs.m;
16 }
```

Move.hpp

Retour au début

```
1  #ifndef MOVE_HPP
2  #define MOVE_HPP
3
4  #include <Coord.hpp>
5  #include <TypePiece.hpp>
6
7  /**
8   * @brief Énumération représentant tout les types de mouvements
9   */
10 enum class TypeMove
11 {
12     /**
13      * @brief Représente le mouvement classic
14      */
15     NORMAL,
16
17     /**
18      * @brief Représente le mouvement de split de deux pièces
19      */
20     SPLIT,
21
22     /**
23      * @brief Représente le mouvement de fusion de deux pièces
24      */
25     MERGE,
26 }
```

```

27  /**
28   * @brief Représente le mouvement de promotion du pion
29   *
30   */
31  PROMOTE
32  };
33
34  /**
35   * @brief Stoque tout les mouvements possibles
36   */
37  struct Move
38  {
39      Move() = default;
40      Move(Move const &) = default;
41      Move &operator=(Move const &) = default;
42      Move(Move &&) = default;
43      Move &operator=(Move &&) = default;
44      /**
45       * @brief Indique quel est le mouvement stocké par l'union
46       */
47      TypeMove type;
48
49      /**
50       * @brief Stoque au choix l'un des trois mouvements possible
51       */
52      union
53      {
54          /**
55           * @brief Stoque les coordonnées pour un mouvement classic
56           */

```

```
57  struct
58  {
59      /**
60       * @brief La coordonnée de depart
61       */
62      Coord src;
63
64      /**
65       * @brief La coordonnées d'arrivée
66       */
67      Coord arv;
68  } normal;
69
70  /**
71   * @brief stoque l'ensemble des coordonnées pour un
72   * mouvement splité
73   */
74  struct
75  {
76      /**
77       * @brief La coordonnée de depart
78       */
79      Coord src;
80
81      /**
82       * @brief Une coordonnée d'arrivée
83       */
84      Coord arv1;
85
86      /**
```

```
87      * @brief Une seconde coordonnée d'arrivée
88      *
89      */
90      Coord arv2;
91  } split;
92
93  /**
94   * @brief stocke l'ensemble des coordonnées pour un
95   * mouvement de fusion
96   */
97  struct
98  {
99      /**
100     * @brief La coordonnée de la première pièce à fusionner
101     */
102     Coord src1;
103
104     /**
105     * @brief La coordonnée de la seconde pièce à fusionner
106     */
107     Coord src2;
108
109     /**
110     * @brief La coordonnée de la position d'arrivée
111     */
112     Coord arv;
113  } merge;
114
115  /**
116   * @brief stocke l'ensemble des coordonnées et la pièce choisi
```

```

117     * pour un mouvement de promotion
118     */
119     struct
120     {
121         /**
122          * @brief La case de depart
123          */
124         Coord src;
125
126         /**
127          * @brief La case d'arrivée
128          */
129         Coord arv;
130
131         /**
132          * @brief La piece choisi pour le move
133          */
134         TypePiece piece;
135     } promote;
136 };
137
138 bool operator==(Move const &m) const noexcept;
139 };
140
141 constexpr inline Move Move_classic(Coord const &src, Coord const &arv)
142 {
143     Move move;
144     move.type = TypeMove::NORMAL;
145     move.normal.src = src;
146     move.normal.arv = arv;

```



```
147     return move;
148 }
149
150 constexpr inline Move Move_split(Coord const &src, Coord const &arv1, Coord const &arv2)
151 {
152     Move move;
153     move.type = TypeMove::SPLIT;
154     move.split.src = src;
155     move.split.arv1 = arv1;
156     move.split.arv2 = arv2;
157     return move;
158 }
159
160 constexpr inline Move Move_merge(Coord const &src1, Coord const &src2, Coord const &arv)
161 {
162     Move move;
163     move.type = TypeMove::MERGE;
164     move.merge.src1 = src1;
165     move.merge.src2 = src2;
166     move.merge.arv = arv;
167     return move;
168 }
169
170 constexpr inline Move Move_promote(Coord const &src, Coord const &arv, TypePiece promotion)
171 {
172     Move move;
173     move.type = TypeMove::PROMOTE;
174     move.promote.src = src;
175     move.promote.arv = arv;
176     move.promote.piece = promotion;
```

Retour au début

```
177     return move;
178 }
179
180
```

Move.cpp

Retour au début

```
1  #include <Coord.hpp>
2  #include <TypePiece.hpp>
3  #include "Move.hpp"
4
5  bool Move::operator==(Move const &m) const noexcept
6  {
7      if (type == m.type)
8      {
9          switch (type)
10         {
11             case TypeMove::NORMAL:
12                 return normal.src == m.normal.src &&
13                    normal.arv == m.normal.arv;
14             case TypeMove::SPLIT:
15                 return split.src == m.split.src &&
16                    split.arv1 == m.split.arv1 &&
17                    split.arv2 == m.split.arv2;
18             case TypeMove::MERGE:
19                 return merge.src1 == m.merge.src1 &&
20                    merge.src2 == m.merge.src2 &&
21                    merge.arv == m.merge.arv;
22             case TypeMove::PROMOTE:
23                 return promote.src == m.promote.src &&
24                    promote.arv == m.promote.arv &&
25                    promote.piece == m.promote.piece;
26             default:
```

Retour au début

```
27     return false;
28   }
29 }
30 return false;
31
```

Qubit.hpp

Retour au début

```
1  #ifndef QUBIT_HPP
2  #define QUBIT_HPP
3
4  #include <csddef>
5  #include <complex>
6  #include <array>
7  #include <CMatrix.hpp>
8  #include <Constexpr.hpp>
9
10 /**
11  * @brief Représente un qubit
12  *
13  * @tparam N La taille du Qubit
14  */
15 template <std::size_t N>
16 class Qubit final
17 {
18 public:
19     // Constructeur
20     CONSTEXPR Qubit() = default;
21
22     /**
23     * @brief Construit un nouveau qubit à l'aide d'un tableau de booléen
24     *
25     * @param data le tableau de n booléen utilisé pour
26     * construire un n-qubit ex : |10>
```

```

27      */
28      CONSTEXPR Qubit(std::array<bool, N> const &data);
29      CONSTEXPR Qubit(std::array<std::complex<double>, _2POW(N)> &&init_list);
30
31      // Copie
32      CONSTEXPR Qubit(Qubit const &) = delete;
33      CONSTEXPR Qubit &operator=(Qubit const &) = delete;
34
35      // Mouvement
36      CONSTEXPR Qubit(Qubit &&) = delete;
37      CONSTEXPR Qubit &operator=(Qubit &&) = delete;
38
39      // Destructeur
40      CONSTEXPR ~Qubit() = default;
41
42      template <std::size_t M>
43      friend CONSTEXPR Qubit<M> operator*(
44          CMatrix<_2POW(M)> const &lhs,
45          Qubit<M> const &rhs);
46
47      template <std::size_t M>
48      friend std::ostream &operator<<(
49          std::ostream &out,
50          Qubit<M> const &qubit);
51
52      template <std::size_t M>
53      friend CONSTEXPR std::array<
54          std::pair<
55              std::array<bool, M>,
56              std::complex<double>>>,

```

```

57         2>
58         qubitToArray(Qubit<M> const &qubit);
59
60     private:
61         std::array<std::complex<double>, _2POW(N)> m_data;
62     };
63
64     /**
65     * @brief Transforme un qubit dans sa représentation sous forme de
66     * vecteur en sa représentation classique.
67     * Si le qubit contient deux cases non nuls, c'est une combinaison
68     * linéaires de deux qubits que l'on peut écrire avec la forme classique,
69     * c'est-à-dire avec des "0" et des "1".
70     * @warning Cette fonction ne permet de faire la transformation que
71     * pour au plus deux cases non nuls dans le qubits
72     * @tparam N La taille du qubit
73     * @param[in] qubit Le qubit à transformer
74     * @return Dans chaque cases du tableau, on a le qubits
75     * sous sa représentation classique que l'on modélise par
76     * un tableau de booléen, et un complexe qui est la scalaire.
77     */
78     template <std::size_t N>
79     constexpr std::array<std::pair<std::array<bool, N>, std::complex<double>>, 2>
80     qubitToArray(Qubit<N> const &qubit);
81
82     /**
83     * @brief Opérateur du produit entre un qubit et une matrice
84     * @warning le produit n'est pas associatif
85     * @tparam M La taille du qubit, la matrice est de taille  $2^M$ 
86     * @param[in] lhs La matrice carré complexe de taille  $2^M$ 

```

Retour au début

```
87 * @param[in] rhs Le M-qubit
88 * @return Qubit<M> renvoie le qubit resultant du produit
89 */
90 template <std::size_t N>
91 CONSTEXPR Qubit<N> operator*(
92     CMatrix<_2POW(N)> const &lhs,
93     Qubit<N> const &rhs);
94
95 /**
96  * @brief Fonction d'affichage des qubit sous forme de
97  * liste de nombre complexe
98  *
99  * @tparam M La taille du qubit
100  * @param[out] out Le flux de destination
101  * @param[in] qubit Le qubit à afficher
102  * @return std::ostream& le flux de destination
103  */
104 template <std::size_t N>
105 std::ostream &operator<<(std::ostream &out, Qubit<N> const &qubit);
106
107 #include "Qubit.hpp"
108
109
```


Qubit.hpp

Retour au début

```
1  #include "Qubit.hpp"
2  #include <Matrix.hpp>
3  #include <algorithm>
4  #include <numeric>
5  #include <math_utility.hpp>
6  #include <Constexpr.hpp>
7  #include <Coord.hpp>
8
9  template <std::size_t N>
10 CONSTEXPR Qubit<N>::Qubit(
11     std::array<std::complex<double>, _2POW(N)> &&init_list)
12     : m_data(std::move(init_list))
13 {
14 }
15
16 template <std::size_t N>
17 CONSTEXPR Qubit<N>::Qubit(std::array<bool, N> const &data) : m_data()
18 {
19
20     auto sum{[] (std::array<bool, N> const &tab) -> int
21         {
22             int acc{0};
23             for (std::size_t i{0}; i < std::size(tab); i++)
24             {
25                 acc += _2POW(i) * tab[std::size(tab) - i - 1];
26             }
27         }
28     }
```

```

27         return acc;
28     });
29     m_data[sum(data)] = 1;
30 }
31
32 template <std::size_t N>
33 constexpr Qubit<N> operator*(
34     CMatrix<_2POW(N)> const &lhs,
35     Qubit<N> const &rhs)
36 {
37     std::array<std::complex<double>, _2POW(N)> tab{};
38     for (std::size_t i{0}; i < lhs.numberLines(); i++)
39     {
40         for (std::size_t j{0}; j < lhs.numberColumns(); j++)
41         {
42             tab[i] += lhs(i, j) * rhs.m_data[j];
43         }
44     }
45
46     return Qubit<N>(std::move(tab));
47 }
48
49 template <std::size_t N>
50 std::ostream &operator<<(
51     std::ostream &out,
52     Qubit<N> const &qubit)
53 {
54     for (std::size_t i{0}; i < _2POW(N); i++)
55     {
56         out << qubit.m_data[i].real() << "+" << qubit.m_data[i].imag() << "i ";

```

```
57     }
58     return out;
59 }
60
61 template <std::size_t N>
62 constexpr std::array<
63     std::pair<std::array<bool, N>,
64         std::complex<double>>>,
65     2>
66 qubitToArray(Qubit<N> const &qubit)
67 {
68     std::array<
69         std::pair<std::array<bool, N>,
70             std::complex<double>>>,
71         2>
72     tab{};
73     bool b{true};
74     std::size_t compteur{N};
75     std::size_t pow = _2POW(N);
76     for (std::size_t i{0}; i < pow; i++)
77     {
78         using namespace std::complex_literals;
79         if (complex_equal(qubit.m_data[i], 0i))
80         {
81             if (i == pow - 1)
82             {
83                 return tab;
84             }
85             else
86             {
```

```
87     if (b)
88     {
89         while (tab[1].first[compteur - 1])
90         {
91             tab[0].first[compteur - 1] = false;
92             tab[1].first[compteur - 1] = false;
93             compteur--;
94         }
95         tab[0].first[compteur - 1] = true;
96         tab[1].first[compteur - 1] = true;
97         compteur = N;
98     }
99     else
100    {
101        while (tab[1].first[compteur - 1])
102        {
103            tab[1].first[compteur - 1] = false;
104            compteur--;
105        }
106        tab[1].first[compteur - 1] = true;
107        compteur = N;
108    }
109 }
110 }
111 else
112 {
113     if (b)
114     {
115         tab[0].second = qubit.m_data[i];
116         b = false;
```

Retour au début

```
117         if (i != pow - 1)
118         {
119             while (tab[1].first[compteur - 1])
120             {
121                 tab[1].first[compteur - 1] = false;
122                 compteur--;
123             }
124             tab[1].first[compteur - 1] = true;
125             compteur = N;
126         }
127     }
128     else
129     {
130         tab[1].second = qubit.m_data[i];
131         return tab;
132     }
133 }
134 }
135 return tab;
136
```

Random.hpp

Retour au début

```
1  #ifndef RANDOM_HPP
2  #define RANDOM_HPP
3
4  namespace rnd
5  {
6      /**
7       * @brief Renvoie un entier dans l'intervale [a, b]
8       *
9       * @param a, b Entier pour spécifier la plage
10      * @return int Un entier entre a et b inclu
11      */
12      int randint(int a, int b);
13
14      /**
15       * @brief Renvoie un réel dans l'intervale [a, b)
16       *
17       * @param a, b Réel pour spécifier la plage
18       * @return double Un réel entre a et b inclu
19       */
20      double randreal(double a, double b);
21  }
22
23
24
```

Random.cpp

Retour au début

```
1  #include <random>
2  #include "Random.hpp"
3
4  namespace
5  {
6      std::random_device rd;
7  }
8
9  namespace rnd
10 {
11     int randint(int a, int b)
12     {
13         static std::uniform_int_distribution<> distrib(a, b);
14         return distrib(rd);
15     }
16
17     double randreal(double a, double b)
18     {
19         static std::uniform_real_distribution<> distrib(a, b);
20         return distrib(rd);
21     }
22 }
```

math-utility.hpp

Retour au début

```
1  #ifndef MATH_UTILITY_HPP
2  #define MATH_UTILITY_HPP
3
4  #define EPSILON 10e-6
5
6  #include <complex>
7  #include <Consteexpr.hpp>
8
9  namespace
10 {
11     CONSTEXPR bool double_equal(double x, double y);
12     CONSTEXPR bool complex_equal(
13         std::complex<double> const &z1,
14         std::complex<double> const &z2);
15 }
16
17 #include "math_utility.hpp"
18
```


math-utility.hpp

Retour au début

```
1  #include <cmath>
2  #include <complex>
3  #include <Constexpr.hpp>
4  #include "math_utility.hpp"
5
6  namespace
7  {
8      CONSTEXPR bool double_equal(double x, double y)
9      {
10         return std::abs(x - y) <= EPSILON;
11     }
12
13     CONSTEXPR bool complex_equal(
14         std::complex<double> const &z1,
15         std::complex<double> const &z2)
16     {
17         return double_equal(z1.real(), z2.real()) &&
18             double_equal(z1.imag(), z2.imag());
19     }
20 }
```

check-path.hpp

Retour au début

```
1  #ifndef CHECK_PATH_HPP
2  #define CHECK_PATH_HPP
3
4  #include <Coord.hpp>
5  #include <Board.hpp>
6  #include <Constepr.hpp>
7
8  template <std::size_t N, std::size_t M>
9  CONSTEXPR static bool
10 check_path_straight(
11     Board<N, M> const &board,
12     Coord const &dpt,
13     Coord const &arv) noexcept;
14
15 template <std::size_t N, std::size_t M>
16 CONSTEXPR static bool
17 check_path_diagonal(
18     Board<N, M> const &board,
19     Coord const &dpt,
20     Coord const &arv) noexcept;
21
22 /**
23  * @brief Vérifie si il y a une pièce entre deux cases sur une instance
24  * du plateau pour un mouvement orthogonale
25  *
26  * @tparam N Le nombre de ligne du plateau
```

```

27 * @tparam M Le nombre de colonnes du plateau
28 * @param[in] board Le plateau
29 * @param[in] dpt Les coordonnées de la case de départ
30 * @param[in] arv Les coordonnées de la case d'arrivé
31 * @param[in] position L'instance du plateau
32 * @return true si le mouvement est possible
33 * @return false si le mouvement est bloqué,
34 * on si ce n'est pas un mouvement orthogonal
35 */
36 template <std::size_t N, std::size_t M>
37 CONSTEXPR bool
38 check_path_straight_1_instance(
39     Board<N, M> const &board,
40     Coord const &dpt,
41     Coord const &arv,
42     std::size_t position,
43     std::optional<Coord> position_other_piece_merge);
44
45 /**
46  * @brief Vérifie si il y a une pièce entre deux cases sur une
47  * instance du plateau pour un mouvement diagonale.
48  *
49  * @tparam N Le nombre de ligne du plateau
50  * @tparam M Le nombre de colonnes du plateau
51  * @param[in] board Le plateau
52  * @param[in] dpt Les coordonnées de la case de départ
53  * @param[in] arv Les coordonnées de la case d'arrivé
54  * @param[in] position L'instance du plateau
55  * @return true si le mouvement est possible
56  * @return false si le mouvement est bloqué, on si ce n'est pas

```

```

57  * un mouvement diagonale.
58  */
59  template <std::size_t N, std::size_t M>
60  CONSTEXPR bool check_path_diagonal_1_instance(
61      Board<N, M> const &board,
62      Coord const &dpt,
63      Coord const &arv,
64      std::size_t position,
65      std::optional<Coord> position_other_piece_merge);
66
67  /**
68   * @brief Vérifie si il y a une pièce entre deux cases
69   * sur une instance du plateau pour un mouvement orthogonale ou diagonale.
70   *
71   * @tparam N Le nombre de ligne du plateau
72   * @tparam M Le nombre de colonnes du plateau
73   * @param[in] board Le plateau
74   * @param[in] dpt Les coordonnées de la case de départ
75   * @param[in] arv Les coordonnées de la case d'arrivée
76   * @param[in] position L'instance du plateau
77   * @return true si le mouvement est possible
78   * @return false si le mouvement est bloqué, on si ce n'est pas
79   * un mouvement orthogonal ou diagonale.
80   */
81  template <std::size_t N, std::size_t M>
82  CONSTEXPR bool check_path_queen_1_instance(
83      Board<N, M> const &board,
84      Coord const &dpt,
85      Coord const &arv,
86      std::size_t position,

```

Retour au début

```
87     std::optional<Coord> position_other_piece_merge);  
88  
89     #include "check_path.tpp"  
90
```

check-path.hpp

Retour au début

```
1  #include <algorithm>
2  #include <Board.hpp>
3  #include <Coord.hpp>
4  #include <Constexpr.hpp>
5  #include "check_path.hpp"
6
7  template <std::size_t N, std::size_t M>
8  CONSTEXPR bool
9  check_path_straight(
10     Board<N, M> const &board,
11     Coord const &dpt,
12     Coord const &arv) noexcept
13 {
14     if (dpt.n == arv.n)
15     {
16         for (std::size_t i{std::min(dpt.m, arv.m) + 1};
17              i < std::max(dpt.m, arv.m);
18              i++)
19         {
20             if (board(dpt.n, i).get_type() != TypePiece::EMPTY ||
21                 board.get_proba(Coord(dpt.n, i)) < 1.)
22             {
23                 return false;
24             }
25         }
26         return true;
27     }
```

```

27     }
28     else if (dpt.m == arv.m)
29     {
30         for (std::size_t i{std::min(dpt.n, arv.n) + 1};
31             i < std::max(dpt.n, arv.n);
32             i++)
33         {
34             if (board(i, dpt.m).get_type() != TypePiece::EMPTY ||
35                 board.get_proba(Coord(i, dpt.m)) < 1.)
36             {
37                 return false;
38             }
39         }
40         return true;
41     }
42     return false;
43 }
44
45 template <std::size_t N, std::size_t M>
46 constexpr bool
47 check_path_diagonal(
48     Board<N, M> const &board,
49     Coord const &dpt,
50     Coord const &arv) noexcept
51 {
52     std::size_t const max_lines{std::max(dpt.n, arv.n)};
53     std::size_t const min_lines{std::min(dpt.n, arv.n)};
54     std::size_t const max_columns{std::max(dpt.m, arv.m)};
55     std::size_t const min_columns{std::min(dpt.m, arv.m)};
56

```

```

57     std::size_t const dist{max_lines - min_lines};
58
59     if (dist == max_columns - min_columns)
60     {
61         for (std::size_t i{1}; i < dist; i++)
62         {
63             if (board(min_lines + i, min_columns + i).get_type() != TypePiece::EMPTY)
64             {
65                 return false;
66             }
67         }
68         return true;
69     }
70     return false;
71 }
72
73 template <std::size_t N, std::size_t M>
74 constexpr bool
75 check_path_straight_1_instance(
76     Board<N, M> const &board,
77     Coord const &dpt,
78     Coord const &arv,
79     std::size_t position,
80     std::optional<Coord> position_other_piece_merge)
81 {
82     if (dpt.n == arv.n)
83     {
84         for (std::size_t i{std::min(dpt.m, arv.m) + 1};
85              i < std::max(dpt.m, arv.m);
86              i++)

```



```

87     {
88         if (board.m_board[position].first[board.offset(dpt.n, i)])
89         {
90             if (position_other_piece_merge.has_value())
91             {
92                 if (!(board.offset(position_other_piece_merge.value().n,
93                                     position_other_piece_merge.value().m) == board.offset(dpt.n,
94                                                 ↪ i)))
95                 {
96                     return false;
97                 }
98             }
99             else
100             {
101                 return false;
102             }
103         }
104         return true;
105     }
106 }
107 else if (dpt.m == arv.m)
108 {
109     for (std::size_t i{std::min(dpt.n, arv.n) + 1};
110          i < std::max(dpt.n, arv.n);
111          i++)
112     {
113         if (board.m_board[position].first[board.offset(i, dpt.m)])
114         {
115             if (position_other_piece_merge.has_value())
116             {

```

```

117         if (!(board.offset(position_other_piece_merge.value().n,
118                             position_other_piece_merge.value().m) == board.offset(i,
119                                         ↪ dpt.m)))
120         {
121             return false;
122         }
123         else
124         {
125             return false;
126         }
127     }
128 }
129 return true;
130 }
131 return false;
132 }
133
134 template <std::size_t N, std::size_t M>
135 constexpr bool
136 check_path_diagonal_1_instance(
137     Board<N, M> const &board,
138     Coord const &dpt,
139     Coord const &arv,
140     std::size_t position,
141     std::optional<Coord> position_other_piece_merge)
142 {
143     std::size_t const max_lines{std::max(dpt.n, arv.n)};
144     std::size_t const min_lines{std::min(dpt.n, arv.n)};
145     std::size_t const max_columns{std::max(dpt.m, arv.m)};
146     std::size_t const min_columns{std::min(dpt.m, arv.m)};

```

```

147
148     std::size_t const dist{max_lines - min_lines};
149
150     if (dist == max_columns - min_columns)
151     {
152         for (std::size_t i{1}; i < dist; i++)
153         {
154             if (board.m_board[position]
155                 .first[board.offset(min_lines + i, min_columns + i)])
156             {
157                 if (position_other_piece_merge.has_value())
158                 {
159                     if (!(board.offset(position_other_piece_merge.value().n,
160                                         position_other_piece_merge.value().m) ==
161                         board.offset(min_lines + i,
162                                       min_columns + i)))
163                     {
164                         return false;
165                     }
166                 }
167                 else
168                 {
169                     return false;
170                 }
171             }
172         }
173         return true;
174     }
175     return false;
176 }

```

Retour au début

```
177
178     template <std::size_t N, std::size_t M>
179     CONSTEXPR bool check_path_queen_1_instance(
180         Board<N, M> const &board,
181         Coord const &dpt,
182         Coord const &arv,
183         std::size_t position,
184         std::optional<Coord> position_other_piece_merge)
185     {
186         return check_path_straight_1_instance<N, M>(board, dpt, arv, position,
187             ↪ position_other_piece_merge) ||
188             check_path_diagonal_1_instance<N, M>(board, dpt, arv, position,
189             ↪ position_other_piece_merge);
```

Unitary.hpp

[Retour au début](#)

```
1  #ifndef UNITARY_HPP
2  #define UNITARY_HPP
3  #include <CMatrix.hpp>
4  #include <numbers>
5  #include <complex>
6  #include <Constexpr.hpp>
7
8  using namespace std::complex_literals;
9  using namespace std::numbers;
10
11 constexpr inline
12 CMatrix<4> MATRIX_ISWAP {
13     {
14         {1, 0, 0, 0},
15         {0, 0, 1i, 0},
16         {0, 1i, 0, 0},
17         {0, 0, 0, 1}
18     }
19 };
20
21
22 constexpr inline
23 CMatrix<4> MATRIX_SQRT_ISWAP {
24     {
25         {1, 0, 0, 0},
26         {0, 1./sqrt2, 1i/sqrt2, 0},
```

```

27         {0, 1i/sqrt2, 1./sqrt2, 0},
28         {0,          0,          0, 1}
29     }
30 };
31
32 constexpr inline
33 CMatrix<4> MATRIX_JUMP { MATRIX_ISWAP };
34
35 constexpr inline
36 CMatrix<8> MATRIX_SLIDE {
37     {
38         {1, 0, 0, 0, 0, 0, 0, 0},
39         {0, 0, 1i, 0, 0, 0, 0, 0 },
40         {0, 1i, 0, 0, 0, 0, 0, 0},
41         {0, 0, 0, 1, 0, 0, 0, 0},
42         {0, 0, 0, 0, 1, 0, 0, 0 },
43         {0, 0, 0, 0, 0, 1, 0, 0},
44         {0, 0, 0, 0, 0, 0, 1, 0},
45         {0, 0, 0, 0, 0, 0, 0, 1}
46     }
47 };
48
49 constexpr inline
50 CMatrix<8> MATRIX_ISWAP_8 {
51     {
52         {1, 0, 0, 0, 0, 0, 0, 0},
53         {0, 0, 0, 0, 1i, 0, 0, 0 },
54         {0, 0, 1, 0, 0, 0, 0, 0},
55         {0, 0, 0, 0, 0, 0, 1i, 0},
56         {0, 1i, 0, 0, 0, 0, 0, 0 },// erreur dans le papier?

```

```

57         {0, 0, 0, 0, 0, 1, 0, 0},
58         {0, 0, 0, 1i, 0, 0, 0, 0},
59         {0, 0, 0, 0, 0, 0, 0, 1}
60     }
61 };
62
63 constexpr inline
64 CMatrix<8> MATRIX_SQRT_ISWAP_8 {
65     {
66         {1,      0,      0, 0, 0,      0,      0, 0},
67         {0, 1./sqrt2, 1i/sqrt2, 0, 0,      0,      0, 0},
68         {0, 1i/sqrt2, 1./sqrt2, 0, 0,      0,      0, 0},
69         {0,      0,      0, 1, 0,      0,      0, 0},
70         {0,      0,      0, 0, 1,      0,      0, 0},
71         {0,      0,      0, 0, 0, 1./sqrt2, 1i/sqrt2, 0},
72         {0,      0,      0, 0, 0, 1i/sqrt2, 1./sqrt2, 0},
73         {0,      0,      0, 0, 0,      0,      0, 1}
74     }
75 };
76
77 constexpr inline
78 CMatrix<8> MATRIX_SPLIT {
79     {
80         {1,      0,      0, 0, 0,      0,      0, 0},
81         {0,      0,      0, 0, 1i,      0,      0, 0},
82         {0, 1i/sqrt2, 1./sqrt2, 0, 0,      0,      0, 0},
83         {0,      0,      0, 0, 0, -1./sqrt2, 1i/sqrt2, 0},
84         {0, 1i/sqrt2, -1./sqrt2, 0, 0,      0,      0, 0},
85         {0,      0,      0, 0, 0, 1i/sqrt2, -1./sqrt2, 0},
86         {0,      0,      0, 1i, 0,      0,      0, 0},

```

```

87         {0,          0,          0,  0,  0,          0,          0,  1}
88     }
89 };
90
91 constexpr inline
92 CMatrix<8> MATRIX_MERGE {
93     {
94         {1,  0,          0,          0,          0,          0,          0,  0,  0},
95         {0,  0, -1i/sqrt2,          0, -1i/sqrt2,          0,          0,  0,  0},
96         {0,  0,  1/sqrt2,          0,  -1/sqrt2,          0,          0,  0,  0},
97         {0,  0,          0,          0,          0,          0,          0, -1i,  0},
98         {0, -1i,          0,          0,          0,          0,          0,  0,  0},
99         {0,  0,          0,  -1/sqrt2,          0, -1i/sqrt2,          0,  0,  0},
100        {0,  0,          0, -1i/sqrt2,          0,  -1/sqrt2,          0,  0,  0},
101        {0,  0,          0,          0,          0,          0,          0,  0,  1}
102    }
103 };
104
105 constexpr inline
106 CMatrix<32> MATRIX_SPLIT_SLIDE {
107     CMatrix<16>{
108         MATRIX_SPLIT, CMatrix<8>{},
109         CMatrix<8>{}, MATRIX_ISWAP_8
110     },
111     CMatrix<16>{
112         CMatrix<16>{},
113         CMatrix<16>{
114             CMatrix<2>::identity()
115             .tensoriel_product(MATRIX_ISWAP), CMatrix<8>{},
116             CMatrix<8>{}, CMatrix<8>::identity()
117         }
118     }
119 };

```


Retour au début

```
117
118 CONSTEXPR inline
119 CMatrix<32> MATRIX_MERGE_SLIDE {
120     CMatrix<16>{
121         MATRIX_MERGE, CMatrix<8>{},
122         CMatrix<8>{}, conj(CMatrix<2>::identity().tensoriel_product(MATRIX_ISWAP).transposed())
123     },
124     CMatrix<16>{},
125     CMatrix<16>{},
126     CMatrix<16>{
127         conj(MATRIX_ISWAP_8.transposed()), CMatrix<8>{},
128         CMatrix<8>{}, CMatrix<8>::identity()
129     }
130 };
131
132
133
134
```

CMatrix.hpp

Retour au début

```
1  #ifndef CMATRIX_HPP
2  #define CMATRIX_HPP
3  #include <Matrix.hpp>
4  #include <complex>
5  #include <Constexpr.hpp>
6  #include <cstdint>
7
8  /**
9   * @brief Objet représentant les matrices carrées complexe
10   * de dimension N
11   *
12   * @tparam N La dimension de la matrice
13   */
14 template <std::size_t N>
15 using CMatrix = Matrix<std::complex<double>, N>;
16
17 namespace
18 {
19     template <std::size_t N>
20     CONSTEXPR CMatrix<N> conj(CMatrix<N> const & matrix)
21     {
22         CMatrix<N> matrix_conj{};
23         for(std::size_t i = 0; i<N; i++)
24         {
25             for(std::size_t j = 0; j<N; j++)
26             {
```

Retour au début

```
27         matrix_conj(i, j) = std::conj(matrix(i, j));
28     }
29 }
30     return matrix_conj;
31 }
32 }
33
34 /**
35  * @brief Réalise l'opération  $2^n$ 
36  * @warning Est valable uniquement sur les types entiers
37  */
38 #define _2POW(n) (1 << (n))
39
40
```

Complex-printer.hpp

Retour au début

```
1  #ifndef COMPLEX_PRINTER_HPP
2  #define COMPLEX_PRINTER_HPP
3
4  #include <complex>
5  #include <concepts>
6  #include <string>
7
8  namespace std
9  {
10     /**
11      * @brief convertit un nombre complexe en chaîne de caractère
12      */
13     using namespace std::literals;
14     template <std::floating_point T>
15     std::string to_string(std::complex<T> value)
16     {
17         return std::string{std::to_string(std::real(value))
18             + ((std::imag(value) >= 0) ? '+' : '\0')
19             + std::to_string(std::imag(value)) + 'i'};
20     }
21
22     /**
23      * @brief Implémente l'opérateur de flux sortant
24      * sur les nombres complexes
25      *
26      * @tparam T le type de représentation floatante du complexe
```

Retour au début

```
27     * @param os Le flux de sorti
28     * @param value Le nombre complexe a transférer
29     * @return std::ostream& retourne le flux passé en paramètre
30     */
31     template <std::floating_point T>
32     std::ostream& operator<<(std::ostream& os, std::complex<T> value)
33     {
34         os << std::real(value) << ((std::imag(value) >= 0) ? '+' : '\0' ) << std::imag(value) << 'i';
35         return os;
36     }
37 }
38
39
```

Constexpr.hpp

Retour au début

```
1  #ifndef CONSTEXPR_HPP
2  #define CONSTEXPR_HPP
3
4  /**
5   * @brief Utilisé pour utiliser ou non constexpr
6   */
7  #define CONSTEXPR constexpr
8
9
```

ConsoleInterface.hpp

[Retour au début](#)

```
1  #ifndef CONSOLE_INTERFACE_HPP
2  #define CONSOLE_INTERFACE_HPP
3
4  #include <ostream>
5  #include <Board.hpp>
6
7  template <std::size_t N, std::size_t M>
8  std::ostream& operator<<(std::ostream& os, Board<N, M> const& board);
9
10 #include "ConsoleInterface.hpp"
11
```

ConsoleInterface.hpp

Retour au début

```
1  #include <iostream>
2  #include <Board.hpp>
3
4  const char *getUnicodeChar(TypePiece piece, Color color)
5  {
6      using enum TypePiece;
7      if (color == Color::WHITE)
8      {
9          switch (piece)
10         {
11             case KING:
12                 return "K";
13             case QUEEN:
14                 return "Q";
15             case ROOK:
16                 return "R";
17             case BISHOP:
18                 return "B";
19             case KNIGHT:
20                 return "N";
21             case PAWN:
22                 return "P";
23             case EMPTY:
24                 default:
25                     return " ";
26         }
```



```
27     }
28     else
29     {
30         switch (piece)
31         {
32             case KING:
33                 return "k";
34             case QUEEN:
35                 return "q";
36             case ROOK:
37                 return "r";
38             case BISHOP:
39                 return "b";
40             case KNIGHT:
41                 return "n";
42             case PAWN:
43                 return "p";
44             case EMPTY:
45             default:
46                 return " ";
47         }
48     }
49 }
50
51 template <std::size_t N, std::size_t M>
52 std::ostream &operator<<(std::ostream &os, Board<N, M> const &board)
53 {
54     for (std::size_t i{0}; i < M; i++)
55     {
56         os << "|-----";
```

```

57 }
58 os << "\\n";
59 for (std::size_t i{0}; i < N; i++)
60 {
61     for (std::size_t j{0}; j < M; j++)
62     {
63         TypePiece piece{board(i, j).get_type()};
64         Color color{board(i, j).get_color()};
65         os << "| " << getUnicodeChar(piece, color) << " ";
66     }
67     os << "\\n";
68     for (std::size_t j{0}; j < M; j++)
69     {
70         if (board(i, j).get_type() != TypePiece::EMPTY)
71         {
72             double proba{board.get_proba(Coord(i, j))};
73             int p{static_cast<int>(std::round(100. * proba))};
74             if (p != 100)
75             {
76                 os << "| ";
77                 if (p < 10){
78                     os << '0';
79                 }
80                 os << p << "% ";
81             }
82             else
83             {
84                 os << "| ";
85             }
86         }

```

[Retour au début](#)

```
87     else
88     {
89         os << " |      ";
90     }
91 }
92 os << "\n";
93 for (std::size_t j{0}; j < M; j++)
94 {
95     os << " |-----";
96 }
97 os << "\n";
98 }
99 (void)board;
100 return os;
101
```

final-solver.hpp

Retour au début

```
1  #ifndef FINAL_SOLVER_HPP
2  #define FINAL_SOLVER_HPP
3  #include <Color.hpp>
4  #include <Constexpr.hpp>
5  #include <memory>
6  #include <stack>
7
8  struct Node
9  {
10     Node(Move const &y, std::stack<Node> &&o = std::stack<Node>{});
11     Move m;
12     std::stack<Node> other_way;
13 };
14
15 Node::Node(Move const &y, std::stack<Node> &&o) : m(y), other_way(o)
16 {
17 }
18
19 namespace Final
20 {
21     template <std::size_t N, std::size_t M>
22     CONSTEXPR bool brut_force_classic_chess(Board<N, M> const &board, std::size_t profondeur, Color
        ↪ c);
23
24     template <std::size_t N, std::size_t M>
25     CONSTEXPR bool brut_force_quantum_chess(Board<N, M> const &board, std::size_t profondeur, Color
        ↪ c);
26
```

Retour au début

```
27     template <std::size_t N, std::size_t M>
28     CONSTEXPR std::pair<double, std::stack<Node>> res_pos(Board<N, M> &board, std::size_t
    ↪ profondeur);
29     template<std::size_t N>
30     CONSTEXPR void print_stack(std::stack<Node> &stack);
31 };
32
33 /*struct C_hash{
34     std::size_t operator () (std::pair<TypePiece, Coord> const &c) const;
35 };
36 bool operator==(std::pair<TypePiece, Coord> const &lhs, std::pair<TypePiece, Coord> const &rhs);
37 */
38
39 #include "final_solver.tpp"
40
41
```

final-solver.tpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Move.hpp>
4  #include <iostream>
5  #include <Color.hpp>
6  #include <unordered_map>
7  #include <functional>
8  #include <stack>
9
10 /*std::size_t C_hash::operator()(std::pair<TypePiece, Coord> const &c) const
11 {
12     std::size_t h1 = std::hash<std::size_t>()(c.second.n);
13     std::size_t h2 = std::hash<std::size_t>()(c.second.m);
14
15     return h1 ^ h2;
16 }
17
18 bool operator==(std::pair<TypePiece, Coord> const &lhs, std::pair<TypePiece, Coord> const &rhs)
19 {
20     return lhs.second.n == rhs.second.n && lhs.second.m == rhs.second.m && lhs.first == rhs.first;
21 }*/
22 template <std::size_t N, std::size_t M>
23 constexpr bool Final::brut_force_classic_chess(Board<N, M> &board, std::size_t profondeur, Color c)
24 {
25     if (board.winning_position(c))
26     {
```

```

27         return true;
28     }
29     else
30     {
31         Color col = Color::WHITE;
32         if (c == Color::WHITE)
33         {
34             col = Color::BLACK;
35         }
36         if (board.winning_position(col))
37         {
38             return false;
39         }
40     else
41     {
42         if (profondeur == 0)
43         {
44             // std::cout<<"profondeur pas assez élevé"<<std::endl;
45             return false;
46         }
47     else
48     {
49
50         if (board.get_current_player() == c)
51         {
52             for (std::size_t i{0}; i < N; i++)
53             {
54                 for (std::size_t j{0}; j < M; j++)
55                 {
56                     if (board(i, j).get_type() != TypePiece::EMPTY && board(i, j).get_color()
↪ == board.get_current_player())

```

```

57         {
58             std::forward_list<Coord> l = board.get_list_normal_move(Coord(i, j));
59             for (auto const &e : l)
60             {
61                 Move m = Move_classic(Coord(i, j), e);
62                 Board<N, M> b = board;
63                 b.change_player();
64                 b.move(m);
65                 if (brut_force_classic_chess(b, profondeur - 1, c))
66                 {
67                     return true;
68                 }
69             }
70         }
71     }
72 }
73 return false;
74 }
75 else
76 {
77     for (std::size_t i{0}; i < N; i++)
78     {
79         for (std::size_t j{0}; j < M; j++)
80         {
81             if (board(i, j).get_type() != TypePiece::EMPTY && board(i, j).get_color()
82                 ↪ == board.get_current_player())
83             {
84                 std::forward_list<Coord> l = board.get_list_normal_move(Coord(i, j));
85                 for (auto const &e : l)
86                 {
87                     Move m = Move_classic(Coord(i, j), e);

```



```

87         Board<N, M> b = board;
88         b.change_player();
89         b.move(m);
90         if (!brut_force_classic_chess(b, profondeur - 1, c))
91         {
92             return false;
93         }
94     }
95 }
96 }
97 }
98     return true;
99 }
100 }
101 }
102 }
103     std::cout << "erreur" << std::endl;
104     return false;
105 }
106
107 template <std::size_t N, std::size_t M>
108 constexpr bool Final::brut_force_quantum_chess(Board<N, M> &board, std::size_t profondeur, Color c)
109 {
110     bool found_win_pos{false};
111     if (board.winning_position(c))
112     {
113         return true;
114     }
115     else
116     {

```

```
117     if (profondeur == 0)
118     {
119         return false;
120     }
121     else
122     {
123         board.all_move(
124             [&board,
125              profondeur,
126              c,
127              &found_win_pos](Move const &m) mutable -> bool
128         {
129             Board<N, M> board_cpy = board;
130             board_cpy.move(m);
131             board_cpy.change_player();
132             if (c == board.get_current_player())
133             {
134                 if (brut_force_quantum_chess(board_cpy, profondeur - 1, c))
135                 {
136                     found_win_pos = true;
137                     return true;
138                 }
139             }
140             else
141             {
142                 if (!brut_force_quantum_chess(board_cpy, profondeur - 1, c))
143                 {
144                     return true;
145                 }
146                 found_win_pos = true;
```

```
147         }
148         return false;
149     });
150     }
151     }
152     return found_win_pos;
153 }
154
155 template <std::size_t N, std::size_t M>
156 double evaluer(const Board<N, M> &board, Color c)
157 {
158
159     if (board.winning_position(c))
160     {
161         return 1.;
162     }
163     else
164     {
165         if (board.winning_position(opponent_color(c)))
166         {
167             return -1.;
168         }
169         else
170         {
171             return 0.;
172         }
173     }
174 }
175
176 template<std::size_t N>
177 void print_move(std::ostream &output, Move m)
```

```

177 {
178     std::string data1{{static_cast<char>('a' + m.normal.src.m), static_cast<char>('0' + N -
    ↪ m.normal.src.n)}};
179     std::string data2{{static_cast<char>('a' + m.normal.arv.m), static_cast<char>('0' + N -
    ↪ m.normal.arv.n)}};
180     if (m.type == TypeMove::NORMAL)
181     {
182         output << "N " << data1 << data2 ;
183     }
184     else if (m.type == TypeMove::PROMOTE)
185     {
186         output << "P " << data1 << data2 << " :: " <<
    ↪ std::to_string(static_cast<int>(m.promote.piece)) ;
187     }
188     else
189     {
190         std::string data3{{static_cast<char>('a' + m.split.arv2.m), static_cast<char>('0' + N -
    ↪ m.split.arv2.n)}};
191         if (m.type == TypeMove::SPLIT)
192         {
193             output << "S " << data1 << '(' << data2 << data3 << ')';
194         }
195         else
196         {
197             output << "M " << '(' << data1 << data2 << ') ' << data3 ;
198         }
199     }
200 }
201 // Fonction récursive pour l'élagage alpha-bêta
202 template <std::size_t N, std::size_t M>
203 CONSTEXPR double alphaBeta(Board<N, M> const &board, std::size_t profondeur, double alpha, double
    ↪ beta, bool estMax, Color c, std::stack<Node> &stack)
204 {
205     if (profondeur == 0 || board.winning_position(c) || board.winning_position(opponent_color(c)))
206     {

```

```

207     Move m = Move_classic(Coord(0, 0), Coord(0, 0));
208     stack.push(m); // Cette information n'a pas d'intérêt dans la pile mais évite les erreurs de
    ↪ segmentation lorsqu'on dépile
209     return evaluer(board, c);
210 }
211 else
212 {
213     Move meilleur_move;
214     if (estMax)
215     {
216         double meilleurScore = {-2.};
217         board.all_move(
218             [&board,
219              profondeur,
220              c,
221              &beta,
222              &alpha,
223              &meilleur_move,
224              &meilleurScore,
225              &stack](Move const &m) mutable -> bool
226
227         {
228             double res;
229             bool one_move{true};
230             double proba_move{board.get_proba_move(m)};
231             if (double_equal(proba_move, 1.))
232             {
233                 Board<N, M> board_cpy = board;
234                 board_cpy.change_player();
235                 board_cpy.move(m, true);
236                 double eval{evaluer(board_cpy, c)};

```

```

237         if (double_equal(eval, 1.))
238         {
239             meilleur_move = m;
240             meilleurScore = 1.;
241             return true;
242         }
243     else
244     {
245         if (double_equal(eval, -1.))
246         {
247             if (-1. > meilleurScore)
248             {
249                 meilleurScore = -1.;
250                 meilleur_move = m;
251             }
252         }
253     else
254     {
255         std::stack<Node> new_stack{};
256         res = alphaBeta(board_cpy, profondeur - 1, alpha, beta, false, c,
↪ new_stack);
257         if (res > meilleurScore)
258         {
259             meilleurScore = res;
260             meilleur_move = m;
261             stack = std::move(new_stack);
262         }
263     }
264 }
265 }
266 else

```

```

267 {
268     if (double_equal(proba_move, 0.))
269     {
270         Board<N, M> board_cpy = board;
271         board_cpy.change_player();
272         board_cpy.move(m, false);
273         double eval{evaluer(board_cpy, c)};
274         if (double_equal(eval, 1.))
275         {
276             meilleur_move = m;
277             meilleurScore = 1.;
278             return true;
279         }
280     else
281     {
282         if (double_equal(eval, -1.))
283         {
284             if (-1. > meilleurScore)
285             {
286                 meilleurScore = -1.;
287                 meilleur_move = m;
288             }
289         }
290     else
291     {
292         std::stack<Node> new_stack{};
293         res = alphaBeta(board_cpy, profondeur - 1, alpha, beta, false, c,
↪ new_stack);
294         if (res > meilleurScore)
295         {
296             meilleurScore = res;

```

```

297             meilleur_move = m;
298             stack = std::move(new_stack);
299         }
300     }
301 }
302 }
303 else
304 {
305     one_move = false;
306     Board<N, M> board_cpy1 = board;
307     board_cpy1.change_player();
308     board_cpy1.move(m, true);
309     Board<N, M> board_cpy2 = board;
310     board_cpy2.change_player();
311     board_cpy2.move(m, false);
312     double eval1{evaluer(board_cpy1, c)};
313     std::stack<Node> new_stack1{};
314     std::stack<Node> new_stack2{};
315     if (double_equal(eval1, 0.))
316     {
317         eval1 = alphaBeta(board_cpy1, profondeur - 1, -2., 2., false, c,
318             ↪ new_stack1);
319     }
320     double eval2{evaluer(board_cpy2, c)};
321     if (double_equal(eval2, 0.))
322     {
323         eval2 = alphaBeta(board_cpy2, profondeur - 1, -2., 2., false, c,
324             ↪ new_stack2);
325     }
326     res = eval1 * proba_move + eval2 * (1 - proba_move);
327     if (res > meilleurScore)
328     {

```



```

327         meilleurScore = res;
328         meilleur_move = m;
329         stack = std::move(new_stack1);
330         if (stack.empty())
331         {
332             stack = std::move(new_stack2);
333         }
334         else
335         {
336             Node inter = stack.top();
337             stack.pop();
338             inter.other_way = std::move(new_stack2);
339         }
340     }
341 }
342 }
343 if (one_move)
344 {
345     alpha = std::max(alpha, meilleurScore);
346     if (beta <= alpha)
347     {
348         return true; // Élagage bêta
349     }
350 }
351 return false;
352 });
353 stack.push(Node(meilleur_move));
354 return meilleurScore;
355 }
356 else

```

```

357     {
358         double meilleurScore{2.};
359         board.all_move(
360             [&board,
361              profondeur,
362              c,
363              &alpha,
364              &meilleurScore,
365              &meilleur_move,
366              &beta,
367              &stack](Move const &m) mutable -> bool
368
369             {
370                 double res;
371                 bool one_move{true};
372                 double proba_move{board.get_proba_move(m)};
373                 if (double_equal(proba_move, 1.))
374                 {
375                     Board<N, M> board_cpy = board;
376                     board_cpy.change_player();
377                     board_cpy.move(m, true);
378                     double eval{evaluer(board_cpy, c)};
379                     if (double_equal(eval, -1.))
380                     {
381                         meilleur_move = m;
382                         meilleurScore = -1.;
383                         return true;
384                     }
385                     else
386                     {

```

```

387         if (double_equal(eval, 1.))
388         {
389             if (1. < meilleurScore)
390             {
391                 meilleurScore = 1.;
392                 meilleur_move = m;
393             }
394         }
395     else
396     {
397         std::stack<Node> new_stack{};
398         res = alphaBeta(board_cpy, profondeur - 1, alpha, beta, true, c,
399             ↪ new_stack);
400         if (res < meilleurScore)
401         {
402             meilleurScore = res;
403             meilleur_move = m;
404             stack = std::move(new_stack);
405         }
406     }
407 }
408 else
409 {
410     if (double_equal(proba_move, 0.))
411     {
412         Board<N, M> board_cpy = board;
413         board_cpy.change_player();
414         board_cpy.move(m, false);
415         double eval{evaluer(board_cpy, c)};
416         if (double_equal(eval, -1.))

```

```

417     {
418         meilleur_move = m;
419         meilleurScore = -1.;
420         return true;
421     }
422     else
423     {
424         if (double_equal(eval, 1.))
425         {
426             if (1. < meilleurScore)
427             {
428                 meilleurScore = 1.;
429                 meilleur_move = m;
430             }
431         }
432         else
433         {
434             std::stack<Node> new_stack{};
435             res = alphaBeta(board_cpy, profondeur - 1, alpha, beta, true, c,
436 ↪ new_stack);
437             if (res < meilleurScore)
438             {
439                 meilleurScore = res;
440                 meilleur_move = m;
441             }
442         }
443     }
444     else
445     {
446         one_move = false;

```

```

447 Board<N, M> board_cpy1 = board;
448 board_cpy1.change_player();
449 board_cpy1.move(m, true);
450 Board<N, M> board_cpy2 = board;
451 board_cpy2.change_player();
452 board_cpy2.move(m, false);
453 double eval1{evaluer(board_cpy1, c)};
454 std::stack<Node> new_stack1{};
455 std::stack<Node> new_stack2{};
456 if (double_equal(eval1, 0.))
457 {
458     eval1 = alphaBeta(board_cpy1, profondeur - 1, -2., 2., true, c,
459 ↪ new_stack1);
459 }
460 double eval2{evaluer(board_cpy2, c)};
461 if (double_equal(eval2, 0.))
462 {
463     eval2 = alphaBeta(board_cpy2, profondeur - 1, -2., 2., true, c,
464 ↪ new_stack2);
465 }
466 res = eval1 * proba_move + eval2 * (1 - proba_move);
467 if (res < meilleurScore)
468 {
469     meilleurScore = res;
470     meilleur_move = m;
471
472     stack = std::move(new_stack1);
473     if (stack.empty())
474     {
475         stack = std::move(new_stack2);
476     }
477     else

```

Retour au début

```
477         {
478             Node inter = stack.top();
479             stack.pop();
480             inter.other_way = std::move(new_stack2);
481             stack.push(inter);
482         }
483     }
484 }
485
486 if (one_move)
487 {
488     beta = std::min(beta, meilleurScore);
489     if (beta <= alpha)
490     {
491         return true; // Élagage bêta
492     }
493 }
494 return false;
495 });
496 stack.push(Node(meilleur_move));
497 return meilleurScore;
498 }
499 }
500 }
501
502 template <std::size_t N, std::size_t M>
503 CONSTEXPR std::pair<double, std::stack<Node>> Final::res_pos(Board<N, M> &board, std::size_t
↳ profondeur)
504 {
505     Color c{board.get_current_player()};
506     std::stack<Node> stack{};
```

```

507     return std::make_pair(alphaBeta(board, profondeur, -2., 2., true, c, stack), stack);
508 }
509 template <std::size_t N>
510 constexpr void Final::print_stack( std::stack<Node> & stack)
511 {
512     std::vector<std::stack<Node>> tab{};
513     tab.push_back(stack);
514     std::size_t compteur{0};
515     while (compteur < std::size(tab))
516     {
517         compteur = 0;
518         for (auto &e : tab)
519         {
520             if (e.empty())
521             {
522                 compteur++;
523                 std::cout << "      ";
524             }
525             else
526             {
527                 Node elt = e.top();
528                 e.pop();
529                 if(!elt.other_way.empty())
530                 {
531                     tab.push_back(elt.other_way);
532                 }
533                 print_move<N>(std::cout, elt.m);
534             }
535         }
536         std::cout<<std::endl;

```

Retour au début

537 }
538

Matrix.hpp

Retour au début

```
1  #ifndef MATRIX_HPP
2  #define MATRIX_HPP
3
4  #include <array>
5  #include <initializer_list>
6  #include <concepts>
7  #include <type_traits>
8  #include <ostream>
9
10 template <typename T>
11 concept arithmetic = std::is_arithmetic_v<T>;
12
13 template <typename T>
14 concept MathObj = requires(T lhs, T rhs) {
15     lhs + rhs == rhs + lhs;
16     lhs - rhs;
17     lhs *rhs;
18 };
19
20 template <MathObj T, std::size_t LINES, std::size_t COLUMNS = LINES>
21 class Matrix final
22 {
23     static_assert(LINES > 0 || COLUMNS > 0,
24         "The matrix cannot have a zero size");
25
26 public:
```

```

27  Matrix() = default;
28  constexpr Matrix(T const initialValue);
29  constexpr Matrix(std::initializer_list<std::initializer_list<T>> const &matrix);
30
31  using SubBloc = Matrix<T, LINES / 2, COLUMNS / 2>;
32  constexpr Matrix(SubBloc const &a, SubBloc const &b, SubBloc const &c, SubBloc const &d);
33
34  constexpr ~Matrix() noexcept = default;
35
36  Matrix(Matrix const &matrix) = default;
37  Matrix &operator=(Matrix const &matrix) = default;
38
39  Matrix(Matrix &&matrix) noexcept = default;
40  Matrix &operator=(Matrix &&matrix) = default;
41
42  constexpr std::size_t numberLines() const noexcept;
43  constexpr std::size_t numberColumns() const noexcept;
44
45  constexpr T const &operator()(std::size_t const lines,
46                                std::size_t const columns) const noexcept;
47  constexpr T &operator()(std::size_t const lines, std::size_t const columns);
48
49  constexpr Matrix<T, LINES, COLUMNS>
50  operator+=(Matrix<T, LINES, COLUMNS> const &matrix) noexcept;
51
52  template <typename U, std::size_t N, std::size_t M>
53  constexpr friend Matrix<U, N, M>
54  operator+(Matrix<U, N, M> matrix_left, Matrix<U, N, M> const &matrix_right) noexcept;
55
56  constexpr Matrix<T, LINES, COLUMNS>

```

```

57 operator==(Matrix<T, LINES, COLUMNS> const &matrix) noexcept;
58
59 template <MathObj U, std::size_t N, std::size_t M>
60 constexpr friend Matrix<U, N, M>
61 operator-(Matrix<U, N, M> matrix_left, Matrix<U, N, M> const &matrix_right) noexcept;
62
63 constexpr Matrix<T, LINES, COLUMNS> &operator*=(int const multiplier) noexcept;
64
65 template <MathObj U, std::size_t N, std::size_t M, arithmetic V>
66 constexpr friend Matrix<U, N, M> operator*(Matrix<U, N, M> matrix,
67      V const multiplier) noexcept;
68
69 template <MathObj U, std::size_t N, std::size_t M, arithmetic V>
70 constexpr friend Matrix<U, N, M> operator*(V const multiplier,
71      Matrix<U, N, M> matrix) noexcept;
72
73 template <MathObj U, std::size_t LINES_M1, std::size_t SHARED,
74      std::size_t COLUMNS_M2>
75 constexpr friend Matrix<U, LINES_M1, COLUMNS_M2>
76 operator*(Matrix<U, LINES_M1, SHARED> const &matrix_left,
77      Matrix<U, SHARED, LINES_M1> const &matrix_right) noexcept;
78
79 template <MathObj U, std::size_t N, std::size_t M>
80 friend std::ostream &operator<<(std::ostream &out,
81      Matrix<U, N, M> const &matrix);
82
83 constexpr Matrix<T, COLUMNS, LINES> transposed() const noexcept;
84
85 constexpr Matrix<T, 1, COLUMNS>
86 line(std::size_t const indexLine) const noexcept;

```

Retour au début

```
87
88 constexpr Matrix<T, LINES, 1>
89 column(std::size_t const indexColumn) const noexcept;
90
91 constexpr static Matrix<T, LINES, COLUMNS> identity() noexcept;
92
93 template <std::size_t N, std::size_t M>
94 constexpr Matrix<T, LINES*N, COLUMNS*M> tensoriel_product(Matrix<T, N, M> const& rhs) const
95 ↪ noexcept;
96
97 private:
98     constexpr std::size_t offset(std::size_t const lines,
99                                   std::size_t const columns) const noexcept;
100
101     constexpr std::array<std::size_t, COLUMNS> strSizeMax() const;
102     constexpr std::size_t strSize(T const value) const;
103
104     constexpr std::array<T, COLUMNS * LINES>
105     array_matrix_with_initializer_list(std::initializer_list<std::initializer_list<T>> const &matrix);
106
107     std::array<T, LINES * COLUMNS> m_matrix{0};
108 };
109
110 #include "Matrix.tpp"
111
```

Matrix.hpp

Retour au début

```
1  #include <cassert>
2  #include <algorithm>
3  #include <string>
4  #include <ostream>
5  #include <iomanip>
6  #include <cmath>
7  #include "Matrix.hpp"
8
9  template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
10 constexpr Matrix<T, LINES, COLUMNS>::Matrix(T const initialValue)
11 {
12     std::fill(std::begin(m_matrix), std::end(m_matrix), initialValue);
13 }
14
15 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
16 constexpr Matrix<T, LINES, COLUMNS>::Matrix(std::initializer_list<std::initializer_list<T>> const &
    ↪ matrix) :
17 m_matrix()
18 {
19     m_matrix = array_matrix_with_initializer_list(matrix);
20 }
21
22 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
23 constexpr Matrix<T, LINES, COLUMNS>::Matrix(SubBloc const& a, SubBloc const& b, SubBloc const& c,
    ↪ SubBloc const& d)
24 : m_matrix()
25 {
26     static_assert( (LINES % 2 == 0 || COLUMNS % 2 == 0)
```

```

27         && "Le constructeur par bloc n'est valable que pour des matrices de taille pair" );
28     constexpr std::size_t N { LINES/2 };
29     constexpr std::size_t M { COLUMNS/2 };
30
31     for (std::size_t i {0}; i < N; i++)
32     {
33         for (std::size_t j {0}; j < M; j++)
34         {
35             m_matrix[offset(i, j)] = a(i, j);
36             m_matrix[offset(i, j + M)] = b(i, j);
37             m_matrix[offset(i + N, j)] = c(i, j);
38             m_matrix[offset(i + N, j + M)] = d(i, j);
39         }
40     }
41 }
42
43 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
44 constexpr std::size_t
45 Matrix<T, LINES, COLUMNS>::numberLines() const noexcept
46 {
47     return LINES;
48 }
49
50 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
51 constexpr std::size_t
52 Matrix<T, LINES, COLUMNS>::numberColumns() const noexcept
53 {
54     return COLUMNS;
55 }
56

```

```

57 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
58 constexpr std::size_t
59 Matrix<T, LINES, COLUMNS>::offset(std::size_t const lines,
60                                     std::size_t const columns) const noexcept
61 {
62     assert(lines < numberLines() && "Column requested invalid.");
63     assert(columns < numberColumns() && "Line requested invalid.");
64     return lines * numberColumns() + columns;
65 }
66
67 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
68 constexpr T const &
69 Matrix<T, LINES, COLUMNS>::operator()(std::size_t const lines,
70                                         std::size_t const columns) const noexcept
71 {
72     return m_matrix[offset(lines, columns)];
73 }
74
75 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
76 constexpr T &
77 Matrix<T, LINES, COLUMNS>::operator()(std::size_t const lines,
78                                         std::size_t const columns)
79 {
80     return m_matrix[offset(lines, columns)];
81 }
82
83 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
84 constexpr Matrix<T, LINES, COLUMNS>
85 Matrix<T, LINES, COLUMNS>::operator+=(Matrix<T, LINES, COLUMNS> const &matrix) noexcept
86 {

```

```

87   for (std::size_t i{0}; i < numberLines(); i++)
88   {
89       for (std::size_t j{0}; j < numberColumns(); j++)
90       {
91           (*this)(i, j) += matrix(i, j);
92       }
93   }
94   return *this;
95 }
96
97 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
98 constexpr Matrix<T, LINES, COLUMNS>
99 operator+(Matrix<T, LINES, COLUMNS> matrix_left,
100          Matrix<T, LINES, COLUMNS> const &matrix_right) noexcept
101 {
102     return matrix_left += matrix_right;
103 }
104
105 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
106 constexpr Matrix<T, LINES, COLUMNS>
107 Matrix<T, LINES, COLUMNS>::operator-(Matrix<T, LINES, COLUMNS> const &matrix) noexcept
108 {
109     assert(matrix.numberLines() == numberLines() && "The subtraction of two matrixes, requires that
110     ↪ they be of the "
111             "same size");
112     assert(matrix.numberColumns() == numberColumns() && "The subtraction of two matrixes, requires that
113     ↪ they be of the "
114             "same size");
115     for (std::size_t i{0}; i < numberLines(); i++)
116     {
117         for (std::size_t j{0}; j < numberColumns(); j++)

```



```

117     {
118         (*this)(i, j) -= matrix(i, j);
119     }
120 }
121 return *this;
122 }
123
124 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
125 constexpr Matrix<T, LINES, COLUMNS>
126 operator-(Matrix<T, LINES, COLUMNS> matrix_left,
127           Matrix<T, LINES, COLUMNS> const &matrix_right) noexcept
128 {
129     return matrix_left -= matrix_right;
130 }
131
132 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
133 constexpr Matrix<T, LINES, COLUMNS> &
134 Matrix<T, LINES, COLUMNS>::operator*=(int const multiplicier) noexcept
135 {
136     for (std::size_t i{0}; i < numberLines(); i++)
137     {
138         for (std::size_t j{0}; j < numberColumns(); j++)
139         {
140             (*this)(i, j) *= multiplicier;
141         }
142     }
143     return *this;
144 }
145
146 template <MathObj T, std::size_t LINES, std::size_t COLUMNS, arithmetic V>

```

```

147 constexpr Matrix<T, LINES, COLUMNS>
148 operator*(Matrix<T, LINES, COLUMNS> matrix, V const multiplicier) noexcept
149 {
150     return matrix *= multiplicier;
151 }
152
153 template <MathObj T, std::size_t LINES, std::size_t COLUMNS, arithmetic V>
154 constexpr Matrix<T, LINES, COLUMNS>
155 operator*(V const multiplicier, Matrix<T, LINES, COLUMNS> matrix) noexcept
156 {
157     return matrix * multiplicier;
158 }
159
160 template <MathObj T, std::size_t LINES_M1, std::size_t SHARED,
161         std::size_t COLUMNS_M2>
162 constexpr Matrix<T, LINES_M1, COLUMNS_M2>
163 operator*(Matrix<T, LINES_M1, SHARED> const &matrix_left,
164         Matrix<T, SHARED, COLUMNS_M2> const &matrix_right) noexcept
165 {
166
167     Matrix<T, LINES_M1, COLUMNS_M2> destination{};
168     for (std::size_t i{0}; i < matrix_left.numberLines(); i++)
169     {
170         for (std::size_t j{0}; j < matrix_right.numberColumns(); j++)
171         {
172             T somme = 0;
173             for (std::size_t k{0}; k < matrix_left.numberColumns(); k++)
174             {
175                 somme += matrix_left(i, k) * matrix_right(k, j);
176             }

```

```

177     destination(i, j) = somme;
178 }
179 }
180 return destination;
181 }
182
183 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
184 constexpr std::size_t Matrix<T, LINES, COLUMNS>::strSize(T const value) const
185 {
186     std::size_t size{1};
187     if constexpr (std::is_integral<T>())
188     {
189         if (value != 0)
190         {
191             size = static_cast<std::size_t>(std::log10(value)) + 1;
192         }
193     }
194     else
195     {
196         std::string str{std::to_string(value)};
197         size = std::size(str);
198     }
199     return size;
200 }
201
202 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
203 constexpr std::array<std::size_t, COLUMNS> Matrix<T, LINES, COLUMNS>::strSizeMax() const
204 {
205     std::array<std::size_t, COLUMNS> maxSize {};
206     for (std::size_t i {0}; i < LINES; i++)

```

```

207 {
208     for (std::size_t j {0}; j < COLUMNS; j++)
209     {
210         std::size_t size = strSize(m_matrix[offset(i, j)]);
211         if (size > maxSize[j])
212         {
213             maxSize[j] = size;
214         }
215     }
216 }
217 return maxSize;
218 }
219
220 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
221 std::ostream &
222 operator<<(std::ostream &out, Matrix<T, LINES, COLUMNS> const &matrix)
223 {
224     out << std::setfill(' ');
225     if constexpr (LINES == 1)
226     {
227         out << "(";
228         std::for_each(std::begin(matrix.m_matrix), std::end(matrix.m_matrix),
229             [&out](T const& v) -> void
230             {
231                 out << v << ' ';
232             });
233         out << ')' << std::endl;
234         return out;
235     }
236     else

```

```
237 {
238     out << "/";
239 }
240 std::array<std::size_t, COLUMNS> maxSize {matrix.strSizeMax()};
241 for (std::size_t i{0}; i < matrix.numberLines(); i++)
242 {
243     if (i == matrix.numberLines() -1)
244     {
245         out << '\\';
246     }
247     else if (i > 0)
248     {
249         out << '|';
250     }
251     for (std::size_t j{0}; j < matrix.numberColumns(); j++)
252     {
253         std::size_t const size = maxSize[j] - matrix.strSize(matrix(i, j));
254         if (size > 0)
255         {
256             out << std::setw(static_cast<int>(size)+1);
257         }
258         out << ' ' << matrix(i, j);
259     }
260     if (i == 0)
261     {
262         out << " \\\" << std::endl;
263     }
264     else if (i < matrix.numberLines() -1)
265     {
266         out << " |\" << std::endl;
```

```
267     }
268 }
269 if constexpr (LINES == 1)
270 {
271     out << ")" << std::endl;
272 }
273 else
274 {
275     out << " / " << std::endl;
276 }
277 return out;
278 }
279
280 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
281 constexpr Matrix<T, COLUMNS, LINES>
282 Matrix<T, LINES, COLUMNS>::transposed() const noexcept
283 {
284     Matrix<T, COLUMNS, LINES> trans{};
285     for (std::size_t i{0}; i < numberLines(); i++)
286     {
287         for (std::size_t j{0}; j < numberColumns(); j++)
288         {
289             trans(j, i) = (*this)(i, j);
290         }
291     }
292     return trans;
293 }
294
295 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
296 constexpr Matrix<T, 1, COLUMNS>
```

```

297 Matrix<T, LINES, COLUMNS>::line(std::size_t const indexLine) const noexcept
298 {
299     assert(indexLine < numberLines() && "index outside the matrix");
300     Matrix<T, 1, COLUMNS> cpyLine{};
301     for (std::size_t i{0}; i < numberColumns(); i++)
302     {
303         cpyLine(0, i) = (*this)(indexLine, i);
304     }
305     return cpyLine;
306 }
307
308 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
309 constexpr Matrix<T, LINES, 1>
310 Matrix<T, LINES, COLUMNS>::column(
311     std::size_t const indexColumn) const noexcept
312 {
313     assert(indexColumn < numberColumns() && "index outside the matrix");
314     Matrix<T, LINES, 1> cpyLine{};
315     for (std::size_t i{0}; i < numberLines(); i++)
316     {
317         cpyLine(i, 0) = (*this)(i, indexColumn);
318     }
319     return cpyLine;
320 }
321
322 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
323 constexpr std::array<T, COLUMNS*LINES>
324 Matrix<T, LINES, COLUMNS>::array_matrix_with_initializer_list(
325     std::initializer_list<std::initializer_list<T>> const & matrix
326 )

```

```

327 {
328     std::array<T, COLUMNS*LINES> array_matrix = {};
329     auto it_mat_dst { std::begin(array_matrix) };
330     for (std::initializer_list<T> const & e : matrix)
331     {
332         assert(std::size(e) <= COLUMNS && "Value entry out of matrix");
333         std::move(std::begin(e), std::end(e), it_mat_dst);
334         it_mat_dst += COLUMNS;
335     }
336     return array_matrix;
337 }
338
339 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
340 constexpr Matrix<T, LINES, COLUMNS> Matrix<T, LINES, COLUMNS>::identity() noexcept
341 {
342     static_assert(LINES == COLUMNS && "not identity matrix in non square matrix");
343     Matrix<T, LINES, COLUMNS> Id {};
344     for (std::size_t i { 0 }; i < LINES; i++)
345     {
346         Id(i, i) = static_cast<T>(1);
347     }
348     return Id;
349 }
350
351 template <MathObj T, std::size_t LINES, std::size_t COLUMNS>
352 template <std::size_t N, std::size_t M>
353 constexpr Matrix<T, LINES*N, COLUMNS*M> Matrix<T, LINES, COLUMNS>::tensoriel_product(Matrix<T, N, M>
    ↪ const& rhs) const noexcept
354 {
355     Matrix<T, LINES*N, COLUMNS*M> res {};
356     for (std::size_t i {0}; i < LINES; i++)

```


Retour au début

```
357 {
358     for (std::size_t j {0}; j < COLUMNS; j++)
359     {
360         for (std::size_t k {0}; k < N; k++)
361         {
362             for (std::size_t l {0}; l < M; l++)
363             {
364                 res(k + i*N, l + j * M) = m_matrix[offset(i, j)] * rhs(k, l);
365             }
366         }
367     }
368 }
369 return res;
370
```

test-move-promotion.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_promotion(int argc, char **argv)
9  {
10     Board<3> board{
11         {W_ROOK, Piece(), Piece()},
12         {B_PAWN, Piece(), Piece()},
13         {Piece(), Piece(), Piece()}};
14     Move m = Move_promote(Coord(1, 0), Coord(2, 0), TypePiece::BISHOP);
15     board.move_promotion(m);
16     return !(board(2, 0).get_type() == TypePiece::BISHOP &&
17             double_equal(board.get_proba(Coord(1, 0)), 0.) &&
18             board(1, 0).get_type() == TypePiece::EMPTY);
19 }
```

test-move-enpassant-with-capture.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_enpassant_with_capture(int argc, char **argv)
9  {
10     Board<5> board{
11         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
12         {B_PAWN, Piece(), Piece(), Piece(), Piece()},
13         {Piece(), Piece(), Piece(), Piece(), Piece()},
14         {Piece(), W_PAWN, Piece(), Piece(), Piece()},
15         {Piece(), Piece(), Piece(), W_BISHOP, Piece()}};
16     Move m = Move_split(Coord(4, 3), Coord(3, 2), Coord(2, 1));
17     Move m1 = Move_classic(Coord(3, 1), Coord(1, 1));
18     Move m2 = Move_classic(Coord(1, 0), Coord(2, 1));
19     board.move(m);
20     board.move(m1);
21     board.move(m2);
22     bool equal = double_equal(board.get_proba(Coord(3, 2)), 0.5);
23     return !(board(2, 1).get_type() == TypePiece::PAWN &&
24             equal &&
25             board(1, 1).get_type() == TypePiece::EMPTY &&
26             board(1, 0).get_type() == TypePiece::EMPTY);
27 }
```

test-move-pawn-two-step.cpp

Retour au début

```
1  #include <math_utility.hpp>
2  #include <Board.hpp>
3  #include <Piece.hpp>
4  #include <Coord.hpp>
5  #include <TypePiece.hpp>
6  #include <Move.hpp>
7
8  int test_move_pawn_two_step(int argc, char **argv)
9  {
10     Board<5> board{
11         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
12         {B_PAWN, Piece(), Piece(), Piece(), Piece()},
13         {Piece(), Piece(), Piece(), Piece(), Piece()},
14         {Piece(), Piece(), Piece(), Piece(), Piece()},
15         {Piece(), Piece(), Piece(), Piece(), Piece()}};
16     Move m = Move_classic(Coord(1, 0), Coord(3, 0));
17     board.move(m);
18     return !(board(3, 0).get_type() == TypePiece::PAWN &&
19             double_equal(board.get_proba(Coord(1, 0)), 0.) &&
20             board(1, 0).get_type() == TypePiece::EMPTY);
21 }
```

test-move-promotion-with-capture.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_promotion_with_capture(int argc, char **argv)
9  {
10     Board<3> board{
11         {W_ROOK, Piece(), Piece()},
12         {B_PAWN, Piece(), Piece()},
13         {Piece(), W_QUEEN, Piece()}};
14     Move m = Move_promote(Coord(1, 0), Coord(2, 1), TypePiece::BISHOP);
15     board.move(m);
16     return !(board(2, 1).get_type() == TypePiece::BISHOP &&
17             board(2, 1).get_color() == Color::BLACK &&
18             double_equal(board.get_proba(Coord(1, 0)), 0.) &&
19             board(1, 0).get_type() == TypePiece::EMPTY);
20 }
```

test-move-promotion-with-split-before.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_promotion_with_split_before(int argc, char **argv)
9  {
10     Board<3> board{
11         {W_ROOK, Piece(), Piece()},
12         {B_PAWN, Piece(), Piece()},
13         {Piece(), W_QUEEN, Piece()}};
14     Move m1 = Move_split(Coord(2, 1), Coord(2, 0), Coord(2, 2));
15     Move m2 = Move_promote(Coord(1, 0), Coord(2, 0), TypePiece::BISHOP);
16     board.move(m1);
17     board.move(m2);
18     bool res = !((board(2, 0).get_type() == TypePiece::BISHOP &&
19         board(2, 0).get_color() == Color::BLACK &&
20         board(1, 0).get_type() == TypePiece::EMPTY &&
21         double_equal(board.get_proba(Coord(2, 2)), 1.)) ||
22         (board(2, 0).get_type() == TypePiece::QUEEN &&
23         board(2, 0).get_color() == Color::WHITE &&
24         board(1, 0).get_type() == TypePiece::PAWN &&
25         double_equal(board.get_proba(Coord(2, 2)), 0.)));
26     return res;
27 }
```

test-move-split-with-mesure.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_split_with_mesure(int argc, char* *argv)
9  {
10     Board<5> board{
11         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
12         {Piece(), Piece(), Piece(), Piece(), Piece()},
13         {Piece(), Piece(), Piece(), Piece(), Piece()},
14         {Piece(), Piece(), Piece(), Piece(), Piece()},
15         {Piece(), Piece(), Piece(), B_ROOK, Piece()}};
16     Move m = Move_split(Coord(4,3), Coord(4, 1), Coord(4,4));
17     Move m1 = Move_split(Coord(0, 0), Coord(0, 1), Coord(0 ,4));
18     Move m2 = Move_classic(Coord(0,1), Coord(4,1));
19     Move m3 = Move_classic(Coord(4, 4), Coord(0, 4));
20     board.move(m);
21     board.move(m1);
22     board.move(m2);
23     board.move(m3);
24     std::size_t acc{0};
25     for(std::size_t i{0}; i<5; i++)
26     {
```

Retour au début

```
27         for(std::size_t j{0}; j<5; j++)
28         {
29             if(board(i, j).get_type() == TypePiece::ROOK)
30             {
31                 acc++;
32             }
33         }
34     }
35     return acc != 2 && acc != 1;
36 }
```


test-split-in-non-empty-case.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Move.hpp>
4  #include <Coord.hpp>
5  #include <math_utility.hpp>
6
7  int test_split_in_non_empty_case(int argc, char** argv)
8  {
9      Board<1, 4> board {
10         {W_ROOK, Piece(), Piece(), Piece()}
11     };
12
13     Move m1 {Move_split(Coord(0, 0), Coord(0, 1), Coord(0, 3))};
14     Move m2 {Move_split(Coord(0, 1), Coord(0, 2), Coord(0, 3))};
15
16     board.move(m1);
17     board.move(m2);
18
19     bool ret {
20         board(0, 1).get_type() == TypePiece::ROOK &&
21         board(0, 2).get_type() == TypePiece::ROOK &&
22         board(0, 3).get_type() == TypePiece::ROOK &&
23         double_equal(board.get_proba(Coord(0, 1)), 0.5) &&
24         double_equal(board.get_proba(Coord(0, 2)), 0.25) &&
25         double_equal(board.get_proba(Coord(0, 3)), 0.25)
26     };
27 }
```

Retour au début

```
27     return !ret;  
28
```

test-slide-merge-move-through-a-split-piece.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_slide_merge_move_through_a_split_piece(int argc, char **argv)
9  {
10     Board<2, 5> board{
11         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
12         {Piece(), Piece(), B_ROOK, Piece(), Piece()}};
13     Move m = Move_split(Coord(1, 2), Coord(0, 2), Coord(1, 4));
14     Move m1 = Move_split(Coord(0, 0), Coord(0, 1), Coord(0, 4));
15     Move m2 = Move_merge(Coord(0, 1), Coord(0, 4), Coord(0, 3));
16     board.move(m);
17     board.move(m1);
18     board.move(m2);
19     bool res{board(0, 2).get_type() == TypePiece::ROOK &&
20         board(0, 2).get_color() == Color::BLACK &&
21         board(0, 1).get_type() == TypePiece::ROOK &&
22         board(0, 1).get_color() == Color::WHITE &&
23         double_equal(board.get_proba(Coord(0, 1)), 0.5) &&
24         board(0, 3).get_type() == TypePiece::ROOK &&
25         board(0, 3).get_color() == Color::WHITE &&
26         double_equal(board.get_proba(Coord(0, 3)), 0.5) &&
```

Retour au début

```
27         board(0, 4).get_type() == TypePiece::EMPTY};  
28     return !res;  
29
```

test-get-proba-move-slide-capture.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_get_proba_move_slide_capture(int argc, char **argv)
9  {
10     Board<3, 5> board{
11         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
12         {Piece(), W_ROOK, W_ROOK, W_ROOK, Piece()},
13         {Piece(), Piece(), Piece(), Piece(), B_ROOK}};
14     Move m1 = Move_split(Coord(1, 1), Coord(0, 1), Coord(2, 1));
15     Move m2 = Move_split(Coord(1, 2), Coord(0, 2), Coord(2, 2));
16     Move m3 = Move_split(Coord(1, 3), Coord(0, 3), Coord(2, 3));
17     Move m4 = Move_split(Coord(2, 4), Coord(0, 4), Coord(1, 4));
18     Move m5 = Move_classic(Coord(0, 4), Coord(0, 0));
19     board.move(m1);
20     board.move(m2);
21     board.move(m3);
22     board.move(m4);
23     bool res{double_equal(board.get_proba_move(m5), 1./16)};
24     return !res;
25 }
```

test-get-proba-move-slide-on-same-color.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_get_proba_move_slide_on_same_color(int argc, char **argv)
9  {
10     Board<3, 5> board{
11         {Piece(), Piece(), Piece(), Piece(), Piece()},
12         {B_QUEEN, W_ROOK, W_ROOK, W_ROOK, Piece()},
13         {Piece(), Piece(), Piece(), Piece(), B_ROOK}};
14     Move m1 = Move_split(Coord(1, 1), Coord(0, 1), Coord(2, 1));
15     Move m2 = Move_split(Coord(1, 2), Coord(0, 2), Coord(2, 2));
16     Move m3 = Move_split(Coord(1, 3), Coord(0, 3), Coord(2, 3));
17     Move m4 = Move_split(Coord(2, 4), Coord(0, 4), Coord(1, 4));
18     Move m = Move_split(Coord(1, 0), Coord(0, 0), Coord(2, 0));
19     Move m5 = Move_classic(Coord(0, 4), Coord(0, 0));
20     board.move(m1);
21     board.move(m2);
22     board.move(m3);
23     board.move(m4);
24     board.move(m);
25     bool res{double_equal(board.get_proba_move(m5), 1./2)};
26     return !res;
27 }
```

test-get-proba-move-slide-without-target.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_get_proba_move_slide_without_target(int argc, char **argv)
9  {
10     Board<3, 5> board{
11         {Piece(), Piece(), Piece(), Piece(), Piece()},
12         {Piece(), W_ROOK, W_ROOK, W_ROOK, Piece()},
13         {Piece(), Piece(), Piece(), Piece(), B_ROOK}};
14     Move m1 = Move_split(Coord(1, 1), Coord(0, 1), Coord(2, 1));
15     Move m2 = Move_split(Coord(1, 2), Coord(0, 2), Coord(2, 2));
16     Move m3 = Move_split(Coord(1, 3), Coord(0, 3), Coord(2, 3));
17     Move m4 = Move_split(Coord(2, 4), Coord(0, 4), Coord(1, 4));
18     Move m5 = Move_classic(Coord(0, 4), Coord(0, 0));
19     board.move(m1);
20     board.move(m2);
21     board.move(m3);
22     board.move(m4);
23     bool res{double_equal(board.get_proba_move(m5), 1.)};
24     return !res;
25 }
```

test-get-proba-move-jump-same-color.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_get_proba_move_jump_same_color(int argc, char **argv)
9  {
10     Board<3> board{
11         { W_KING, Piece(), Piece() },
12         { Piece(), Piece(), Piece() },
13         { Piece(), Piece(), W_KNIGHT } };
14     Move m1 = Move_split(Coord(2, 2), Coord(0, 1), Coord(1, 0));
15     Move m5 = Move_classic(Coord(0, 0), Coord(0, 1));
16     board.move(m1);
17     bool res{double_equal(board.get_proba_move(m5), 1./2)};
18     return !res;
19 }
```


test-get-proba-move-jump-capture.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_get_proba_move_jump_capture(int argc, char **argv)
9  {
10     Board<3> board{
11         { W_KING, Piece(), Piece() },
12         { Piece(), Piece(), Piece() },
13         { Piece(), Piece(), B_KNIGHT } };
14     Move m1 = Move_split(Coord(2, 2), Coord(0, 1), Coord(1, 0));
15     Move m5 = Move_classic(Coord(0, 0), Coord(0, 1));
16     board.move(m1);
17     bool res{double_equal(board.get_proba_move(m5), 1.)};
18     return !res;
19 }
```

test-capture-slide-move-true.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <Color.hpp>
7  #include <math_utility.hpp>
8
9  int test_capture_slide_move_true(int argc, char **argv)
10 {
11     Board<3, 5> board{
12         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
13         {Piece(), W_ROOK, W_ROOK, W_ROOK, Piece()},
14         {Piece(), Piece(), Piece(), Piece(), B_ROOK}};
15     Move m1 = Move_split(Coord(1, 1), Coord(0, 1), Coord(2, 1));
16     Move m2 = Move_split(Coord(1, 2), Coord(0, 2), Coord(2, 2));
17     Move m3 = Move_split(Coord(1, 3), Coord(0, 3), Coord(2, 3));
18     Move m4 = Move_split(Coord(2, 4), Coord(0, 4), Coord(1, 4));
19     Move m5 = Move_classic(Coord(0, 4), Coord(0, 0));
20     board.move(m1);
21     board.move(m2);
22     board.move(m3);
23     board.move(m4);
24     board.move(m5, true);
25     bool res{board(0, 0).get_type() == TypePiece::ROOK && board(0, 0).get_color() == Color::BLACK };
26     return !res;
27 }
```

test-capture-slide-move-false.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <Color.hpp>
7  #include <math_utility.hpp>
8
9  int test_capture_slide_move_false(int argc, char **argv)
10 {
11     Board<3, 5> board{
12         {W_ROOK, Piece(), Piece(), Piece(), Piece()},
13         {Piece(), W_ROOK, W_ROOK, W_ROOK, Piece()},
14         {Piece(), Piece(), Piece(), Piece(), B_ROOK}};
15     Move m1 = Move_split(Coord(1, 1), Coord(0, 1), Coord(2, 1));
16     Move m2 = Move_split(Coord(1, 2), Coord(0, 2), Coord(2, 2));
17     Move m3 = Move_split(Coord(1, 3), Coord(0, 3), Coord(2, 3));
18     Move m4 = Move_split(Coord(2, 4), Coord(0, 4), Coord(1, 4));
19     Move m5 = Move_classic(Coord(0, 4), Coord(0, 0));
20     board.move(m1);
21     board.move(m2);
22     board.move(m3);
23     board.move(m4);
24     board.move(m5, false);
25     bool res{board(0, 4).get_type() == TypePiece::ROOK && board(0, 4).get_color() == Color::BLACK };
26     return !res;
27 }
```

test-move-merge-after-split.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_move_merge_after_split(int argc, char **argv)
9  {
10     Board<4> board{
11         {B_KING, Piece(), Piece(), Piece()},
12         {B_ROOK, Piece(), Piece(), Piece()},
13         {Piece(), Piece(), Piece(), Piece()},
14         {W_QUEEN, Piece(), Piece(), W_KING}};
15     Move m1 = Move_split(Coord(3, 0), Coord(3, 1), Coord(3, 2));
16     Move m2 = Move_merge(Coord(3, 1), Coord(3, 2), Coord(3, 0));
17     board.move(m1);
18     board.move(m2, true);
19     bool res{double_equal(board.get_proba(Coord(3,0)), 1.)};
20     return !res;
21 }
```

test-split-after-split.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_split_after_split(int argc, char **argv)
9  {
10     Board<4> board{
11         {Piece(), Piece(), Piece(), Piece()},
12         {Piece(), B_KING, B_ROOK, Piece()},
13         {Piece(), Piece(), Piece(), Piece()},
14         {W_QUEEN, Piece(), Piece(), W_KING}};
15     Move m1 = Move_split(Coord(3, 0), Coord(2, 1), Coord(3, 2));
16     Move m2 = Move_split(Coord(3, 2), Coord(2, 1), Coord(1, 0));
17     board.move(m1);
18     board.move(m2, true);
19     bool res{double_equal(board.get_proba(Coord(3,3)), 1.)&&
20     double_equal(board.get_proba(Coord(2,1)), 1./2) &&
21     double_equal(board.get_proba(Coord(1,0)), 1./2)};
22     return !res;
23 }
```

test-split-after-split2.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_split_after_split2(int argc, char **argv)
9  {
10     Board<4> board{
11         {Piece(), Piece(), Piece(), Piece()},
12         {Piece(), B_KING, B_ROOK, Piece()},
13         {Piece(), Piece(), Piece(), Piece()},
14         {W_QUEEN, Piece(), Piece(), W_KING}};
15     Move m1 = Move_split(Coord(3, 0), Coord(2, 1), Coord(3, 2));
16     Move m2 = Move_split(Coord(3, 2), Coord(1, 0), Coord(2, 1));
17     board.move(m1);
18     board.move(m2, true);
19     bool res{double_equal(board.get_proba(Coord(3,3)), 1.)&&
20 double_equal(board.get_proba(Coord(2,1)), 1./4) &&
21 double_equal(board.get_proba(Coord(1,0)), 1./4) &&
22 double_equal(board.get_proba(Coord(3,2)), 1./2) };
23     return !res;
24 }
```

test-multiple-split.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_multiple_split(int argc, char **argv)
9  {
10     Board<3> board{
11         {Piece(), Piece(), B_BISHOP},
12         {Piece(), B_KING, Piece()},
13         {Piece(), Piece(), Piece()}};
14     Move m1 = Move_split(Coord(1, 1), Coord(2, 1), Coord(2, 2));
15     Move m2 = Move_split(Coord(2, 2), Coord(1, 1), Coord(1, 2));
16     Move m3 = Move_classic(Coord(1, 2), Coord(2, 2));
17     Move m4 = Move_split(Coord(1, 1), Coord(0, 0), Coord(0, 1));
18     board.move(m1);
19     board.move(m2);
20     board.move(m3);
21     board.move(m4);
22     bool res{double_equal(board.get_proba(Coord(0, 2)), 1.)};
23     return !res;
24 }
```

test-split-takes.cpp

Retour au début

```
1  #include <Board.hpp>
2  #include <Piece.hpp>
3  #include <Coord.hpp>
4  #include <TypePiece.hpp>
5  #include <Move.hpp>
6  #include <math_utility.hpp>
7
8  int test_split_takes(int argc, char **argv)
9  {
10     Board<3> board{
11         {Piece(), Piece(), W_BISHOP},
12         {Piece(), B_KING, Piece()},
13         {Piece(), Piece(), Piece()}};
14     Move m1 = Move_split(Coord(1, 1), Coord(2, 1), Coord(2, 0));
15     Move m2 = Move_split(Coord(2, 1), Coord(1, 1), Coord(2, 2));
16     Move m3 = Move_classic(Coord(0, 2), Coord(1, 1));
17     Move m4 = Move_classic(Coord(1, 1), Coord(2, 0));
18     board.move(m1);
19     board.move(m2);
20     board.move(m3);
21     board.move(m4);
22     bool res{double_equal(board.get_proba(Coord(2,0)), 1.)};
23     return !res;
24 }
```